

University of Plymouth

School of engineering,
computing, and mathematics

Final Stage Computing Project
2025/2026

S.I.R.A: Simplifying Cybersecurity for SMEs

Matthew Fish

10855826

BSc (Hons) Computer Science (Cyber Security)

Acknowledgements

I would like to express my gratitude to my supervisor, Nathan Clarke, for his continuous guidance and support throughout this project. His constant availability and willingness to help whenever in need ultimately led to a stronger end product. I would also like to extend my thanks to my family and friends who have supported and encouraged me along the way.

Abstract

Small and medium-sized enterprises (SMEs) face a growing volume of targeted cyberattacks but frequently lack the capital for dedicated Security Operations Centres (SOCs) or enterprise-grade SIEMs. While many implement open-source intrusion detection systems, they often struggle to interpret the complex, raw technical logs generated. The purpose of this project was to develop S.I.R.A. (Security Incident Response Assistant), an affordable, LLM-powered incident response dashboard designed to bridge this cybersecurity expertise gap.

The system integrates a log watcher with a large language model (Google Gemini) and a real-time cloud database. To guarantee data privacy and address cloud security concerns, a rigorous two-tier security pipeline was implemented. Raw logs undergo local regex sanitisation to strip Personally Identifiable Information (PII) prior to AI analysis, and all incident records are subsequently secured via AES encryption before transmission. A React-based frontend then displays this data, utilising dynamic AI personas to tailor the technical depth of mitigation playbooks based on specific user roles.

Rigorous security assessments and User Acceptance Testing (UAT) demonstrated that the localised pre-processing pipeline successfully prevented sensitive data exposure to the LLM. Furthermore, the dynamic AI personas successfully translated complex security alerts into plain-English summaries, accurate risk scores, and actionable mitigation steps. The system allowed non-technical users, such as business owners, to confidently understand and address threats, while still providing sufficient technical depth for IT and SOC analysts.

The S.I.R.A. dashboard successfully simplifies incident management for SMEs. The project proves the viability and effectiveness of combining localised data sanitisation with cloud-based Large Language Models to deliver a secure, accessible, and cost-effective cybersecurity solution.

Contents

Acknowledgements.....	1
Abstract.....	1
List of Tables	4
List of Figures	5
Word Count.....	5
Github Repo	5
1. Introduction	5
1.1 Background and Problem Statement.....	6
1.2 Aims and Objectives.....	6
1.2.1 Project Rationale	6

1.2.2 Aim	7
1.2.3 Objectives	7
1.3 Scope and Constraints	7
1.3.1 In-Scope.....	7
1.3.2 Out-of-Scope (Exclusions).....	8
1.3.3 Technical Constraints	8
2. Literature Analysis.....	8
2.1 The Cybersecurity Landscape for SMEs	8
2.2 Existing Security Solutions and Resource Constraints	9
2.3 The Role of Large Language Models in Cybersecurity	10
2.3.1 Overcoming AI Adoption Barriers.....	10
3. Legal, Social, Ethical, and Professional Issues (LSEP)	11
3.1 Legal	11
3.2 Social	11
3.3 Ethical.....	11
3.4 Professional.....	11
4. Requirements & Specification.....	12
4.1 Target Audience	12
4.2 Functional Requirements.....	13
4.3 Non-Functional Requirements.....	14
4.4 Use Case and User Stories.....	15
4.4.1 System Use Cases	15
4.4.2 User Stories	17
5. Project Management & Methodology	17
5.1 Development Methodology.....	17
5.2 Project Management Technologies	18
5.2.1 The Gantt Chart.....	18
5.2.2 Trello Workspace.....	18
5.3 Project and System Risk Assessment	19
5.3.1 System and Flows	19
5.3.2 Risk Categories and Impact	19
5.3.3 Existing Mitigation Control	19
5.3.4 Risk Register	20
6. System Design & Architecture	20
6.1 Technology Stack Justification	20
6.1.1 Frontend Architecture (React, TypeScript & Tailwind CSS)	20

6.1.2 Backend Processing & API (Python & FastAPI)	20
6.1.3 Database & Authentication (Google Firebase/Firestore)	21
6.1.4 AI & Logic Engine	21
6.1.5 Ingestion Strategy.....	21
6.2 High-Level Architecture	21
6.3 Database Design.....	22
6.4 System Logic & Processing Models	24
6.4.1 Functional Decomposition (HIPO)	24
6.4.2 System Sequence & Interaction	25
6.5 System Resilience & Error Handling.....	26
6.6 User Interface Design.....	27
6.6.1 Initial Low-Fidelity Prototyping.....	27
6.6.2 Agile Feedback & Design Iteration.....	28
7. Development & Implementation	30
7.1 Data Origin & Local Ingestion	30
7.1.1 Continuous Monitoring	30
7.1.2 Initial Triage and Noise Reduction	31
7.2 Privacy & Data Sanitisation	31
7.2.1 Deep Text Sanitisation (Regex)	32
7.2.2 Intelligent IP Masking	32
7.3 AI Integration	33
7.3.1 Asynchronous Batching	33
7.3.2 Dynamic AI Personas	33
7.3.3 Cryptographic Storage.....	33
7.4 Frontend Dashboard	34
7.4.1 Real-Time Data Synchronisation.....	35
7.4.2 Analytics and Data Visualisation	35
7.4.3 Rendering the Mitigation Playbooks.....	36
7.5 The Secure API Gateway	36
7.5.1 The Cryptographic Gateway	36
7.6 Overcoming Challenges	37
7.6.1 Intelligent Caching and Quota Management.....	37
7.6.2 AI Rate Limiting and Prompt Evolution.....	37
7.6.3 Frontend Graph and CSS	37
7.6.4 System Integration	37
7.6.5 Modular Architecture and API Key Management.....	38

8. Testing & Evaluation	38
8.1 Testing Strategy	38
8.1.1 Unit testing.....	38
8.1.2 Integration Testing	41
8.1.3 Performance / Stress Testing.....	41
8.2 User Acceptance Testing (UAT).....	42
9. Conclusion & Future Work.....	44
9.1 Project Summary.....	44
9.2 Critical Evaluation & Limitations	44
9.3 Future Enhancements	45
9.4 Final Thoughts.....	45
References	46
Appendices.....	47
Appendix A: Literature Evidence.....	47
Appendix B: Project Management	48
Appendix C: Initial Project Risk Assessment	51
Appendix D: Initial Wireframes.....	55
Appendix E: Final Wireframes.....	58
Appendix F: Unit Tests	61
Appendix G: Integration Tests	67
Appendix H: Performance / Stress Tests.....	68
Appendix I: Blank UAT Questionnaire and Results.	72
Appendix J: Code Snippets.....	93
Appendix K: AI Declaration	100

List of Tables

Table 1: Comparison of existing security solutions against SME resource constraints.	10
Table 2: S.I.R.A. user persona profiles, associated challenges, and dynamic AI system integration....	12
Table 3: Functional Requirements.	13
Table 4: Non-Functional Requirements.	14
Table 5: User Stories	17
Table 6: Initial Firestore database schema design illustrating the conceptualised processing queue and plaintext data fields.	23
Table 7: Final implemented Firestore database schema illustrating the three core collections.....	24
Table 8: Error-Handling matrix detailing system failure domains.	26
Table 9: Backend Unit Test Results	39
Table 10: Frontend API Unit Test Results	40

Table 11: Frontend Analytics Unit Test Results	40
Table 12: Frontend Incidents Unit Test Results	40
Table 13: Frontend Login Unit Test Results	40
Table 14: Frontend Settings Unit Test Results	41
Table 15: Integration Test Results	41
Table 16: Performance Test Results	42

List of Figures

Figure 1: UML Use Case Diagram illustrating User and System interactions within S.I.R.A.	16
Figure 2: High-Level System Architecture diagram illustrating the data flow across the Presentation, Processing, Database, and Local layers.....	22
Figure 3: HIPO Diagram detailing the functional decomposition of system modules.....	25
Figure 4: Sequence Diagram illustrating the chronological message passing between system components.	26
Figure 5: Initial low-fidelity paper prototype for the system dashboard.	27
Figure 6: Initial wireframe for the proposed Admin Management page.	28
Figure 7: Final high-fidelity design for the dedicated 'View Incident' page, featuring the persistent top navigation bar.	29
Figure 8: Code snippet showing the "digital bookmarking" mechanism used to prevent duplicate processing of log files.....	31
Figure 9: Code snippet demonstrating the regex sanitisation used to strip PII.....	32
Figure 10: Implementation of the IP pseudonymisation logic.....	32
Figure 11: Implementation of the crypto.py module, utilizing Fernet for AES-128 payload encryption and decryption.....	34
Figure 12: Screenshot of the S.I.R.A. main analytics dashboard.	35
Figure 13: Screenshot of the unencrypted metadata in firestore.	36

Word Count

10,830

Github Repo

<https://github.com/Matrrfish/Matt-Fish-COMP3000-Computing-Project>

1. Introduction

Small and medium-sized enterprises (SMEs) represent the backbone of the global economy. According to recent UK Government business population estimates (GOV.UK, 2025), SMEs account for 99.9% of the total private sector business population, representing roughly 5.7 million businesses in the UK alone. Despite making up the vast majority of the economy, they are increasingly targeted in a rapidly evolving cyber threat landscape. This project details the design, development, and evaluation of S.I.R.A. (Security Incident Response Assistant), an AI-driven dashboard designed to empower SMEs to remediate cyber threats without the need for an expensive in-house security team.

1.1 Background and Problem Statement

Cyber threats are escalating globally at an unprecedented rate, with over 2,000 major data breaches reported annually, incurring an average organisational cost exceeding \$3.9 million per incident (Fadziso et al., 2023). As digital technology expands the attack surface, the continuous maturation of enterprise-level security has forced threat actors to strategically pivot towards softer, more vulnerable targets—most notably, small and medium-sized enterprises (SMEs) (Saleem et al., 2017). According to the latest industry data, 59% of SMEs experienced a cyberattack within a single twelve-month period, often leading to severe operational downtime, financial loss, and in many cases, complete business failure (Hiscox Group, 2025).

Despite this escalating threat environment, SME management often suffers from a pervasive cognitive bias, identified in cyberpsychology research as the "illusion of invulnerability"—the flawed belief that their limited financial turnover renders them "too small" to be targeted (Troublefield, 2025). Threat actors, however, increasingly target smaller businesses specifically because they are perceived as soft targets with weaker defences (Tetteh, 2024; Junior et al., 2025).

This dangerous false sense of security is compounded by severe operational constraints. Unlike enterprise-level organisations, SMEs typically lack the capital required to fund dedicated Security Operations Centres (SOCs) or employ experienced, full-time cybersecurity analysts to monitor their networks (Saleem et al., 2017). Without professional monitoring or robust incident response plans, malicious network intrusions frequently go entirely unnoticed until the point of critical impact (Tetteh, 2024).

To mitigate these risks on a restricted budget, some SMEs attempt to implement free, consumer-grade commercial tools or open-source Security Information and Event Management (SIEM) software. However, the overarching lack of internal technical expertise renders these efforts largely ineffective. Non-technical staff often fail to understand how to properly configure these systems or interpret the complex security alerts they generate, creating a dangerous illusion of security (Manzoor et al., 2024). Ultimately, this combination of outdated technology, inadequate training, and overwhelming "technical noise" leads to severe alert fatigue, leaving SMEs highly vulnerable to attack (Lake, 2022). There is, therefore, a critical need for an automated, affordable, and accessible incident response solution capable of translating complex threat data into actionable intelligence for non-technical users.

1.2 Aims and Objectives

1.2.1 Project Rationale

Recent academic literature emphasises that Artificial Intelligence (AI) offers transformative capabilities for cyber defence, enabling automated detection and response without the need for intensive manual oversight (Femi et al., 2025). However, despite this potential, AI adoption among SMEs remains critically low. Femi et al. (2025) highlight that SMEs are frequently deterred by the high costs, technical complexity, and the opaque, "black box" nature of enterprise-grade AI tools. To successfully bridge the cybersecurity expertise gap, resource-constrained organisations urgently require frameworks that are not only affordable and automated but also highly transparent. Systems must provide clear, explainable reasoning so that non-technical users can confidently trust and execute the suggested mitigation strategies.

This project directly addresses this identified academic and industry gap through the development of S.I.R.A. (Security Incident Response Assistant). By leveraging a Large Language Model (LLM) to

translate complex security events into plain-English summaries and dynamic, role-specific playbooks, S.I.R.A. aims to deliver a resilient, user-friendly, and cost-effective cybersecurity solution tailored specifically to the operational realities of SMEs.

1.2.2 Aim

The primary aim of this project is to design, develop, and evaluate an affordable, LLM-powered incident response dashboard that bridges the cybersecurity expertise gap for SMEs by automatically translating complex technical logs into actionable, easy-to-understand mitigation strategies.

1.2.3 Objectives

To successfully achieve this aim, the following technical objectives have been established:

- **Log Ingestion and Parsing:** Develop a robust backend system capable of continuously monitoring and parsing raw log files from standard open-source detection tools (e.g., Snort, Winlogbeat).
- **Data Privacy and Sanitisation:** Implement a strict, localised pre-processing pipeline to strip Personally Identifiable Information (PII) from logs, ensuring no sensitive company data is exposed to the external LLM.
- **AI Integration and Analysis:** Integrate the Google Gemini 1.5 Flash API to analyse the sanitised logs, accurately calculate risk scores, and formulate step-by-step mitigation strategies.
- **Dynamic AI Personas:** Engineer the system to adjust the technical depth of its AI-generated summaries based on the user's specific role (e.g., providing high-level business impact summaries for Business Owners, and explicit CLI commands for SOC Analysts).
- **Secure Cloud Synchronisation:** Utilise Google Firestore for real-time data storage, ensuring all AI insights and incident records are secured via Fernet AES encryption prior to cloud transmission.
- **User Interface Development:** Design and build an intuitive, low latency React frontend dashboard tailored for non-technical users, complete with automated email notifications for critical threat alerts.

1.3 Scope and Constraints

To ensure the project remains achievable and aligned with SME operational realities, clear system boundaries have been established.

1.3.1 In-Scope

The S.I.R.A. dashboard is designed strictly as an intelligent incident management and advisory platform. The core scope encompasses the passive ingestion and parsing of raw logs from existing detection tools (e.g., Snort or Windows Event Viewer). It features a localised pre-processing pipeline utilising regular expressions (regex) to strip Personally Identifiable Information (PII) prior to cloud transmission. The system leverages the Google Gemini 1.5 Flash API to translate these sanitised alerts into plain-English summaries and mitigation playbooks, displayed via a role-based React frontend.

1.3.2 Out-of-Scope (Exclusions)

To distinguish S.I.R.A. from enterprise-grade SIEM tools, active threat mitigation is explicitly excluded. The application functions purely in an advisory capacity; it is not an Intrusion Prevention System (IPS) or active firewall. It will not automatically block IP addresses, terminate malicious processes, or alter network configurations. Furthermore, the installation of underlying host detection engines and deep forensic analysis (such as malware sandboxing) fall completely outside the project scope.

1.3.3 Technical Constraints

As an academic endeavour, the development of S.I.R.A. was shaped by several high-level logistical and structural constraints:

- **Financial and Resource Limitations:** The project was entirely self-funded and operated with a zero-budget constraint. Consequently, the architecture was explicitly designed around open-source software and free-tier cloud services (such as the Google Gemini API and Firebase). While this perfectly aligns with the project's goal of delivering a low-cost solution for SMEs, it inherently limited access to enterprise-grade development tools, premium threat-intelligence feeds, and large-scale, paid testing environments. Furthermore, as a solo-developer project, there was a strict limitation on available human resources for quality assurance and peer review.
- **Temporal Constraints:** The project lifecycle was strictly bounded by the academic calendar. The initial requirements engineering and system design phases were compressed into a two-month window (September to October), followed by a primary development and testing phase from November to April. This rigorous timeframe necessitated the strict prioritisation of core functionalities. Because of these time constraints, certain advanced features and exhaustive edge-case testing had to be scoped out to ensure a minimum viable product (MVP) was delivered on schedule. Consequently, there is an acknowledged possibility that some secondary SME operational requirements may have been deferred to future iterations.

2. Literature Analysis

2.1 The Cybersecurity Landscape for SMEs

Recent systematic literature reviews evaluating the state of SME cybersecurity research highlight a critical imbalance in defensive strategies (Chidukwani *et al.*, 2022). When mapped against the National Institute of Standards and Technology (NIST) Cybersecurity Framework, research indicates a disproportionate academic and industry focus on the 'Identify' and 'Protect' functions. Conversely, there is a severe lack of literature and tooling addressing the 'Detect', 'Respond', and 'Recover' phases (see Appendix A, Figure 1).

The evidence suggests that this disparity exists because the later stages of the NIST lifecycle require a level of real-time technical interpretation that most SMEs cannot provide. While tools for protection, such as firewalls or antivirus, are relatively straightforward to deploy, the response and recovery phases depend on the ability to understand complex network telemetry. This means that even when a small business is aware of an intrusion, they often lack the specific technical guidance needed to contain the threat. This indicates that the primary challenge for SMEs is not just the

presence of a threat, but the difficulty of translating raw security data into a practical recovery plan without expert assistance.

This technical deficit is further compounded by a dangerous false sense of security. Research by Lake (2022) highlights a significant 'confidence gap' among small businesses; while 62% of SME owners feel confident in their ability to quickly respond to a cyberattack, only 34% have a formal incident response plan in place. This indicates that many SMEs are relying on assumptions rather than actionable strategies, leaving them entirely unprepared for a real-world breach.

Lake (2022) further notes that when these businesses do attempt to improve their posture by implementing network monitoring, they frequently succumb to "alert fatigue." Because standard open-source detection tools generate thousands of highly technical alerts, small IT teams or solo administrators quickly become overwhelmed by the constant stream of technical noise. Under this cognitive load, genuine indicators of compromise are often missed or ignored. Ultimately, this proves that the primary challenge for SMEs is not a lack of network data, but the absence of automated filtering and translation. Without a tool to distil raw alerts into clear, manageable recovery steps, small businesses remain highly vulnerable, even with detection software running.

2.2 Existing Security Solutions and Resource Constraints

To combat the escalating threat landscape, the cybersecurity industry heavily promotes the adoption of Security Information and Event Management (SIEM) systems. These platforms aggregate and correlate network telemetry to provide a unified view of an organisation's security posture. However, a recent comprehensive evaluation by Manzoor *et al.* (2024) highlights that the SIEM market is fundamentally bifurcated into two categories (commercial and open source) neither of which adequately serves the operational realities of an SME.

Proprietary, enterprise-grade SIEM solutions, such as Splunk Enterprise, IBM QRadar, and LogRhythm, historically dominate the market and provide exceptional threat visibility. However, Manzoor *et al.* (2024) note that these products are accompanied by "huge licensing costs," making them entirely unaffordable for most small businesses. Furthermore, they operate under the assumption that the purchasing organisation employs a dedicated Security Operations Centre (SOC) staffed by analysts who can actively monitor dashboards and triage alerts.

Consequently, budget-constrained SMEs frequently attempt to deploy open-source or free-tier alternatives, such as Wazuh, AlienVault OSSIM, or Elastic Security. While these solutions successfully circumvent initial licensing costs, Manzoor *et al.* (2024) explicitly warn that selecting and maintaining them is a highly challenging task. Effective deployment demands an "in-depth understanding of security features, performance parameters, and a specialised knowledge base." The technical overhead required to operate these open-source systems is immense; administrators must manually configure Syslog forwarding, deploy host-based agents for File Integrity Monitoring (FIM), and continuously tune custom correlation rules to manage high volumes of Events Per Second (EPS) without dropping data.

For an SME relying on a small IT team or a single generalist administrator, this level of manual configuration, performance tuning, and alert triage is fundamentally unsustainable. To contextualise S.I.R.A. within this landscape, Table 1 evaluates prominent existing solutions against the core SME constraints identified in the literature.

To contextualise S.I.R.A. within this landscape, Table 1 evaluates prominent existing solutions against core SME constraints identified in the literature.

Table 1: Comparison of existing security solutions against SME resource constraints.

Security Solution	Target Market	Licensing Cost	Hardware / Resource Overhead	Technical Expertise Required
Splunk / IBM QRadar	Enterprise	High	High (requires dedicated servers)	High (Dedicated SOC)
Wazuh / Elastic	Enterprise	Free	High (Heavy Local Processing)	High (complex config)
S.I.R.A	SMEs	Free / Low-Cost	Low (Cloud-based Processing)	Low (Non-technical)

As demonstrated in Table 1, this visual dichotomy exposes a stark operational gap: enterprise solutions are financially unviable, while open-source alternatives are technically overwhelming. This synthesis of literature and market reality provides a strong academic basis for the development of S.I.R.A. By functioning as a lightweight translation layer, it delivers SIEM-like insights without the associated financial burden or complex technical overhead.

2.3 The Role of Large Language Models in Cybersecurity

Research suggests that Artificial Intelligence (AI) has become a significant asset for cyber defence, yet its implementation within the SME sector remains limited. This low adoption is primarily driven by significant barriers such as high implementation costs, technical complexity, and difficulties with system integration (Femi *et al.*, 2025). While the literature highlights that AI could greatly improve the cybersecurity posture of smaller organisations, most businesses still lack the necessary funding and technical resources to adopt these tools effectively. This resource gap appears to be the most substantial challenge identified across current research.

Femi *et al.* (2025) further note that even when cloud-based or AI-enhanced solutions are available, they are often overlooked by SMEs due to their inherent complexity. This suggests that for AI to be a viable solution for this demographic, it must be delivered through a framework that is both affordable and technically accessible to non-specialist users.

However, recent industry data suggests that this complexity gap is compounded by an emerging trust gap. A survey by Hiscox (2025) reveals a notable dichotomy: while 65% of SMEs recognise AI as a defensive asset rather than a vulnerability, over 57% of these organisations have already fallen victim to an attack involving an AI-driven vulnerability. This indicates that while the theoretical value of AI is understood, the practical reality of AI-powered threats has created a climate of apprehension. For many SMEs, the decision to avoid AI adoption may be less about a lack of interest and more about a fear of the unknown; without a clear understanding of the risks or the internal expertise to manage them, the perceived danger of the technology currently outweighs its defensive benefits.

2.3.1 Overcoming AI Adoption Barriers

To overcome these barriers, AI deployment must align with SME resource constraints. Localised LLMs require expensive infrastructure and maintenance, reintroducing enterprise-level barriers. Alternatively, leveraging a managed, cloud-based LLM via an API (such as Google Gemini 1.5 Flash) entirely abstracts hardware requirements and costs.

Therefore, the literature dictates a clear architectural requirement. This project proposes integrating a cloud-hosted LLM to translate complex technical alerts into step-by-step mitigation playbooks. By acting strictly as an accessible advisory tool, this framework eliminates prohibitive barriers, empowers rapid incident response, and educates users on their cyber environment.

3. Legal, Social, Ethical, and Professional Issues (LSEP)

3.1 Legal

From a legal perspective, the main concern for any security monitoring tool is complying with the UK General Data Protection Regulation (UK GDPR) and the Data Protection Act 2018 (gov.uk, 2011). Because S.I.R.A. reads network logs, it handles potentially identifiable information (PII) such as user IP addresses and internal usernames. For example, if an external actor probes the SME's network using a scanning tool like Nmap, the resulting security logs will capture those specific IP addresses. To stay legally compliant when sending this data to a cloud-hosted Large Language Model (the Gemini API), S.I.R.A. utilises a strict data sanitisation pipeline. Personal and sensitive data, such as IP addresses, are automatically redacted and removed before the telemetry leaves the SME's internal network.

Additionally, the system must adhere to the Computer Misuse Act 1990 (Legislation.gov.uk, 1990), specifically concerning network scanning. Under the Act, unauthorised active scanning or probing of third-party networks constitutes a criminal offence. To ensure strict legal compliance, S.I.R.A. is designed as a purely passive, read-only tool. It does not actively execute network scans; rather, it only monitors pre-existing local logs. Finally, strong access controls are implemented to ensure the tool cannot be used by unauthorised people to view this sensitive network data.

3.2 Social

Socially, this project helps solve a major problem: the cybersecurity skills gap. As cyber threats increase, small businesses are frequently targeted because they cannot afford to hire full-time, expensive security experts. S.I.R.A. provides a clear social benefit by giving these smaller businesses an affordable, accessible way to defend themselves. Furthermore, by translating confusing technical alerts into plain English, the system helps non-technical staff understand their security posture. This empowers business owners to take control of their own networks and builds better cyber awareness across the wider business community.

3.3 Ethical

Ethically, using Artificial Intelligence for security raises concerns about "hallucinations" (where the AI generates incorrect information) and who is ultimately responsible if things go wrong. If the AI gives bad advice and a business gets hacked because they followed it, the ethical and financial blame can be complicated. To prevent this, S.I.R.A. is designed using a "Human-in-the-Loop" approach. It acts purely as an advisor, not an automated system. By ensuring the AI cannot automatically change network settings or firewall rules, the human user always keeps final control and responsibility. The tool will also clearly warn users that its advice is AI-generated, ensuring they do not blindly rely on it without human judgement.

3.4 Professional

Professionally, building this system requires a strong duty of care. The project strictly follows the British Computer Society (BCS) Code of Conduct, specifically the rules regarding working in the public

interest and maintaining professional competence (BCS, 2022). By basing the AI's incident response playbooks on standard industry frameworks, such as the NIST lifecycle, the project ensures that the guidance provided is professional and trustworthy. Finally, the software will be thoroughly tested against modern threats to ensure it meets the high standards expected of a professional software engineering project.

4. Requirements & Specification

Having established the theoretical and ethical foundations of the project in previous chapters, this chapter defines the specific requirements for the S.I.R.A. system. It outlines the target users through defined personas, details the methods used to gather system requirements, and establishes the strict functional and non-functional specifications necessary to deliver an accessible, AI-driven incident response tool for SMEs.

4.1 Target Audience

S.I.R.A. is primarily targeted towards SME business owners who lack formal technical expertise but are committed to protecting their organisations and fostering a proactive cybersecurity culture. However, the system recognises that the target audience encompasses individuals with varying degrees of technical proficiency. For instance, a small business may be run by someone with previous IT experience, or they may employ a solo generalist IT administrator, yet still lack the financial resources to maintain a dedicated Security Operations Centre (SOC).

To accommodate this broad spectrum of expertise, S.I.R.A. incorporates tailored user personas. This architectural feature dynamically adjusts the AI's response generation to match the specific skill level of the active user. By categorising users into distinct profiles, the system ensures that incident data is neither too complex for a beginner nor too simplistic for an expert. The system is designed around three core target personas:

Table 2: S.I.R.A. user persona profiles, associated challenges, and dynamic AI system integration.

Personas	Profile	Challenge	System Integration
Business Owner	The CEO or founder of the SME with minimal to no formal IT knowledge.	They hold the financial and legal risk of a breach but cannot understand raw security alerts to gauge the severity of an incident.	The AI provides a brief, plain-English executive summary of the threat, explains the potential business impact (e.g., data loss or downtime), and offers highly simplified, high-level mitigation advice.
IT Admin	A generalist IT staff member responsible for day-to-day technology operations (e.g., managing networks, printers, and basic software).	They understand basic networking but suffer from "alert fatigue" when faced with thousands of complex Intrusion Detection System (IDS) logs.	The AI explains the nature of the alert and provides specific, intermediate-level instructions, such as how to isolate an infected machine or block a malicious IP address on the firewall.
SOC Analyst	A highly technical security professional, potentially working for an external	They need to rapidly verify threats across multiple clients and require deep technical	The AI delivers advanced threat intelligence, specific command-line execution scripts for rapid containment, and preserves access

	Managed Service Provider (MSP) overseeing the SME's network.	data to ensure the AI is not hallucinating.	to the raw network telemetry (e.g., JSON logs) for manual verification and forensic analysis
--	--	---	--

4.2 Functional Requirements

The functional requirements define the specific behaviours, interactions, and data processing capabilities of the S.I.R.A. system. To ensure a structured development lifecycle and prioritise core features within the project's time constraints, these requirements have been categorised using the MoSCoW prioritisation methodology.

Must Have (Critical Core Functionality): These requirements form the minimum viable product (MVP) necessary to prove the project's core concept. **FR 1, 2, 3, 4, 6, 7, 12, 16.**

Should Have (Important Enhancements): These features add significant value to the user experience and operational security but are not strictly required for the core AI translation to function. **FR 8, 9, 10, 13, 14**

Could Have (Desirable Additions): **FR 15.**

Won't Have (Out of Scope): As defined in the project constraints (Section 1.4), the system will not feature active Intrusion Prevention System (IPS) capabilities, such as automatically modifying firewall rules or blocking IP addresses.

Table 3: Functional Requirements.

FR1	The system shall ingest log files from an IDS or a controlled test log source.
FR2	The system shall parse each log line to extract: timestamp, source IP, destination IP, ports, protocol, event type, severity, and message text.
FR3	The system shall convert all parsed log data into JSON format for further processing.
FR4	The system shall allow users to securely provide their own API key for an external LLM provider.
FR5	The system shall allow users to select an LLM model from supported online providers.
FR6	The system shall send parsed JSON log data to the chosen LLM for summarisation, risk scoring, and recommended mitigation steps.
FR7	The system shall store all generated summaries, risk scores, and related metadata in Firebase.
FR8	The system shall use Firebase Authentication to control access to the dashboard.
FR9	The system shall display an analytics dashboard as the default landing page after login.

FR10	The analytics dashboard shall show metrics such as: • number of incidents over time • distribution of severity levels • most common attack types • trends or spikes in activity
FR11	The system shall allow users to navigate from the analytics dashboard to a detailed incident log page.
FR12	The incident log page shall list all processed incidents with risk score, attack type, summary, and mitigation steps.
FR13	The system shall allow users to filter and search incidents by severity, timestamp, and attack type.
FR14	The system shall generate notifications for high-risk incidents.
FR15	The system shall allow users to acknowledge, comment on, or mark incidents as resolved.
FR16	The system shall allow the user to enter their technical experience level, and the generate incident summaries tailored to that level.

4.3 Non-Functional Requirements

While functional requirements dictate what the system does, the non-functional requirements (NFRs) dictate how well the system performs these tasks, with a strict emphasis on security, privacy, and performance.

Table 4: Non-Functional Requirements.

NFR1	Incident summaries shall be available within 5–10 seconds of log ingestion, depending on LLM API latency.
NFR2	Only authenticated users shall access the dashboard via Firebase Authentication.
NFR3	API keys shall be stored securely and never logged, transmitted, or exposed.
NFR4	Data shall only be sent to the selected LLM provider and Firebase services.
NFR5	The Firebase database shall use encryption at rest and least-privilege access controls.
NFR6	All log inputs shall be validated and sanitised before processing.
NFR7	The analytics dashboard shall present information that's suitable to the type of user.
NFR8	The user interface shall use colour-coded risk indicators (red = critical, amber = medium, green = low).

NFR9	Navigation between pages shall be simple and intuitive.
NFR10	The system shall use a modular architecture to allow future enhancements to be added without changing core logic.
NFR11	The system shall include documentation for setup, API key configuration, and troubleshooting.
NFR12	The system shall use encryption in transit for all data transmitted between the client, Firebase services, and external LLM APIs.

4.4 Use Case and User Stories

To bridge the gap between high-level system requirements and the technical system design, Use Case modelling and User Stories were utilised. These tools translate the strict MoSCoW specifications into practical, user-centric interactions, ensuring the final architecture aligns with the operational realities of the target personas.

4.4.1 System Use Cases

A comprehensive Use Case diagram was developed to map the boundaries of the S.I.R.A. system and define how external entities interact with it. The system architecture is driven by two primary actors: the User (representing the SME personas defined in Section 4.1) and the System (representing the automated backend infrastructure).

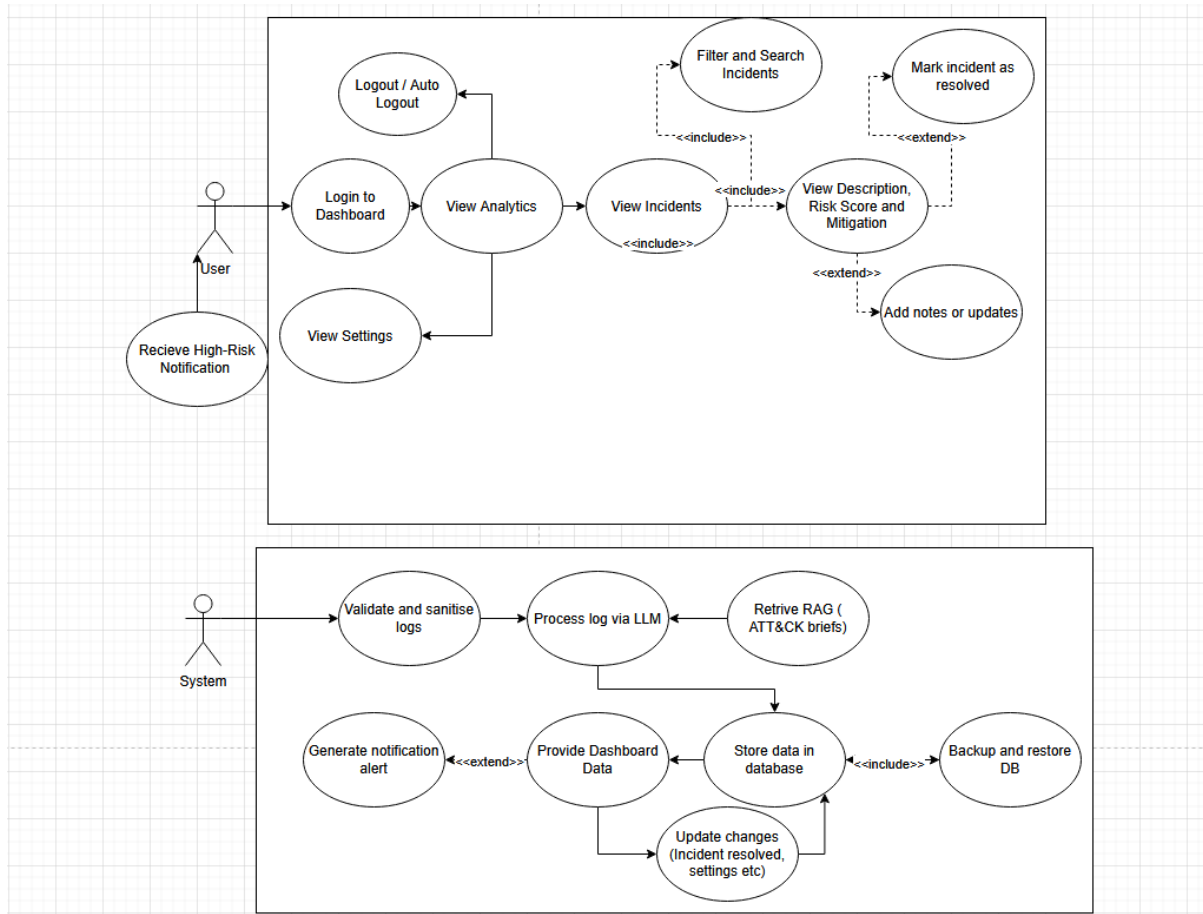


Figure 1: UML Use Case Diagram illustrating User and System interactions within S.I.R.A.

As illustrated in the diagram, the interactions are separated into two distinct workflows:

1. **Frontend User Workflow:** The User actor interacts primarily with the React dashboard. After securely logging in, they can view top-level analytics or navigate to specific incidents. The core interaction involves viewing incident descriptions, risk scores, and mitigation steps, which can optionally be extended (<<extend>>) by marking incidents as resolved or adding manual notes.
2. **Backend System Workflow:** The System actor operates autonomously to manage the data pipeline. It is responsible for validating and sanitising raw logs before processing them via the LLM. The diagram highlights several advanced architectural features, including an initial design concept to retrieve MITRE ATT&CK briefs via Retrieval-Augmented Generation (RAG) to enrich the LLM prompt. However, during the development and testing phases, it was determined that the selected LLMs possessed sufficient inherent training data to generate highly accurate and contextualised summaries out-of-the-box. Consequently, the RAG component was deemed unnecessary and excluded from the final build to streamline system processing and reduce latency. The remaining autonomous features, such as securely storing the parsed data and dynamically triggering high-risk notification alerts to the User, operate exactly as mapped.

4.4.2 User Stories

To further refine the user experience and ensure strict traceability against the defined Functional (FR) and Non-Functional Requirements (NFR), a structured set of User Stories was established. These stories dictate the necessary capabilities of the dashboard from the perspective of the end-user.

Table 5: User Stories

1. Authentication & Security	
US 1.1	As a user, I want to log in securely using my credentials so that only authorised staff can access the dashboard (Maps to: FR8, NFR2).
US 1.2	As a user, I want to provide my own API key for the external LLM provider securely, ensuring it is never logged or exposed (Maps to: FR4, NFR3).
US 1.3	As a user, I want to be automatically logged out after a period of inactivity to ensure my session remains secure (Maps to: FR8).
2. Configuration & Ingestion	
US 2.1	As a user, I want to upload raw log files (e.g., IDS logs) so the system can automatically parse and analyse them (Maps to: FR1, FR2, FR3).
US 2.2	As a user, I want to select which AI model is used for analysis so I can balance cost versus performance (Maps to: FR5).
US 2.3	As a user, I want to set my technical experience level so that the generated summaries are easy for me to understand (Maps to: FR16, NFR7).
US 2.4	As a user, I want the system to validate and sanitise my uploaded logs to prevent corrupted or malicious files from breaking the analysis (Maps to: NFR6).
3. Dashboard & Analytics	
US 3.1	As a user, I want to see an analytics dashboard immediately after login that displays key metrics like incident trends and attack types (Maps to: FR9, FR10).
US 3.2	As a user, I want to view incidents categorised by risk level using clear colour coding (Red, Amber, Green) (Maps to: FR10, NFR8).
US 3.3	As a user, I want to filter and search incidents by severity, timestamp, and attack type to quickly find specific threats (Maps to: FR13).
4. Incident Analysis & Response	
US 4.1	As a user, I want to click on a specific incident to view a detailed report, including the AI-generated summary, risk score, and mitigation steps (Maps to: FR11, FR12, FR6).
US 4.2	As a user, I want to mark incidents as 'Resolved' or add comments so I can track the status of active threats (Maps to: FR15).
US 4.3	As a user, I want to receive immediate notifications for 'High Risk' alerts so I can respond to critical threats without delay (Maps to: FR14, NFR1).
5. Reliability & Support	
US 5.1	As a user, I want my data to be encrypted while it is being sent to the database or LLM provider so that sensitive log information remains private (Maps to: NFR4, NFR12).
US 5.2	As a user, I want access to clear documentation and troubleshooting guides so I can configure my API keys and solve setup issues (Maps to: NFR11).

5. Project Management & Methodology

This chapter outlines the development methodology, the tools used to facilitate it, and the proactive risk assessment undertaken to mitigate potential points of failure.

5.1 Development Methodology

The development lifecycle of this project was governed by an Agile project management methodology. For a solo developer, Agile was identified as the optimal framework over traditional, rigid methodologies, as it allows for continuous iteration, rapid prototyping, and the flexibility to pivot based on testing outcomes. To accelerate this process, Generative AI was used assistively to outline report structures and generate high-level pseudocode to help conceptualise system architecture.

The project was structured into two-week development 'sprints'. At the conclusion of each sprint, fortnightly review meetings were held with the project supervisor, who acted as a proxy stakeholder. These meetings provided a structured environment to review the progress of the previous sprint, highlight technical challenges or blockers, and strategically plan the deliverables for the subsequent weeks.

Crucially, this Agile approach allowed for significant architectural refinements during development. For example, as noted in Section 4.5.1, the initial system design included a Retrieval-Augmented Generation (RAG) module. By employing an iterative, Agile testing approach, it was quickly identified that the LLM performed adequately without this module. Agile allowed the project scope to be rapidly adapted, cutting the redundant feature to save development time without derailing the overall project schedule.

5.2 Project Management Technologies

To facilitate the Agile framework and ensure strict adherence to academic deadlines, a dual-tiered approach to project management was employed, utilising both high-level scheduling and granular sprint tracking.

5.2.1 The Gantt Chart

While the initial baseline (Appendix B, Figures 1 & 2) provided a roadmap, it was formulated before the full literature review and prototyping were complete. Using Agile methodology, the schedule was dynamically updated to reflect two major shifts in strategy:

Research revealed that SMEs face significant hardware constraints. To avoid unnecessary technical complexity, the planned local stack (Llama 3 and PostgreSQL) was substituted for lightweight, cloud-based solutions (Google Gemini API and Firebase).

Iterative testing proved that the Gemini model could contextualize security logs without a custom RAG module. This task was scoped out to reduce system latency and streamline the architecture.

By treating the Gantt chart as a flexible baseline rather than a rigid contract, the project ensured the final system remained practical and aligned with the end-user's operational reality.

5.2.2 Trello Workspace

While the Gantt chart dictated the overarching timeline, Trello was utilised to manage the day-to-day micro-execution of the Agile sprints. The Trello workspace was explicitly structured to support weekly sprint planning and issue tracking across several dedicated boards:

- **The Sprint Plan:** Rather than a traditional state-based Kanban board, this board was structured chronologically, featuring dedicated columns for each week of development. Tasks were proactively assigned to specific weeks based on the MoSCoW priorities, and were systematically ticked off upon completion. This provided clear, immediate visibility into weekly progress.
- **Deadlines:** A chronological tracker dedicated solely to upcoming academic and project deliverables, ensuring critical submission windows were never missed.
- **Queries and Questions:** A dedicated board used to log technical blockers, bugs, or unresolved questions as they arose during coding, preventing them from halting active development.

- **Meetings:** A repository for documenting the outcomes, feedback, and specific action items discussed during the fortnightly supervisor sprint meetings.

(See Appendix B, Figures 4–7 for screenshots of the active Trello workspace)

5.3 Project and System Risk Assessment

5.3.1 System and Flows

The first stage of the assessment involved defining the strict boundaries and data flows of the system. The initial scope mapped the journey of the data from ingestion to display, identifying the privacy risks at each node:

1. **Source to LLM:** Ingestion of structured JSON network logs containing potentially identifiable IP addresses (Assessed Risk: Low–Medium).
2. **LLM Processing:** The AI model receiving JSON input and returning text-based summaries (Assessed Risk: Low).
3. **Database Storage:** The structured storage of processed insights (Assessed Risk: Medium, requiring encryption).
4. **Web Dashboard:** The frontend display of the processed data (Assessed Risk: Low, reliant on authentication).
5. **Test Environment (Scoped Out):** Initially, local Virtual Machines (VMs) were planned to generate test logs. To streamline development and reduce security overhead, this flow was scoped out in favour of pre-existing, open-source log datasets (e.g., alert.ids).

(See Appendix C, Figure 1)

5.3.2 Risk Categories and Impact

To quantify the threats to these data flows, risks were grouped into specific technical and ethical categories, including Privacy Violations, Bias & Fairness, Security, and Operational Reliability.

The overall risk severity was calculated using a quantitative matrix. The Likelihood of a vulnerability being exploited and the potential Impact on the SME was scored on a scale of 1 to 10. These combined metrics generated an overall priority rating of Low, Medium, High or Significant.

(See Appendix C, Figure 2)

5.3.3 Existing Mitigation Control

Before finalizing the risk register, existing mitigation controls inherent to the proposed architecture were evaluated. The primary controls established to protect the system included:

- **Input Sanitisation & Pseudonymisation:** Truncating or hashing IP addresses before they are stored or processed by the LLM.
- **Access Control & Basic Authentication:** Enforcing Role-Based Access Control (RBAC) and the principle of least privilege on the API and web dashboard.
- **Database Security:** Utilizing least-privilege service accounts and ensuring encryption at rest.
- **System Isolation:** Initially planned via Docker container isolation, though ultimately mitigated by the Agile pivot to isolated cloud environments (Google Gemini and Firebase).

(See Appendix C, Figure 3)

5.3.4 Risk Register

The final stage of the assessment involved compiling the identified threats, their calculated impact scores, and the proposed mitigation strategies into a formal Risk Register. Rather than serving purely as a theoretical exercise, this register acted as a critical project management tool throughout the development lifecycle.

It systematically ensured that all 'High' and 'Significant' priority threats were addressed before core development began. By explicitly mapping specific engineering controls to each vulnerability, the register formally demonstrates how the project's initial security risks were successfully reduced to an acceptable, operational 'Low' residual score.

(See Appendix C, Figure 4 for the complete Risk Register)

6. System Design & Architecture

6.1 Technology Stack Justification

To ensure the system remained lightweight, secure, and highly responsive, a modern, decoupled technology stack was selected.

6.1.1 Frontend Architecture (React, TypeScript & Tailwind CSS)

To deliver an accessible user experience for non-technical SME owners, a web-based dashboard was selected over a standalone desktop application.

- **React & TypeScript:** React was chosen as the core frontend framework, as its component-based structure allows for a highly modular dashboard design (e.g., separating charts, incident logs, and settings) that can be easily updated independently. Furthermore, the frontend was developed using TypeScript. Selecting TypeScript over standard JavaScript enforces strict type safety, significantly reducing runtime errors and making the codebase far easier to maintain and debug as complexity increases.
- **Tailwind CSS & Recharts:** Tailwind CSS was utilised to guarantee a modern, highly responsive user interface. Additionally, the Recharts library was integrated to handle dynamic data visualisations without the overhead of building custom graphical components from scratch.

6.1.2 Backend Processing & API (Python & FastAPI)

To efficiently manage the computational load of parsing network telemetry alongside serving web requests, a decoupled backend architecture was implemented.

Python was selected as the core language for the backend. Its extensive native libraries make it exceptionally efficient at handling continuous file I/O operations and complex Regular Expression (Regex) matching for data sanitisation. Furthermore, Python seamlessly integrates with Google's Generative AI SDK, allowing the script to independently batch sanitised logs, query the Gemini API, and encrypt the resulting insights using the cryptography.fernet library before pushing them to the database.

FastAPI was implemented as a secure, high-performance intermediary between the React frontend and the database. It was selected over Flask for its superior speed and native Pydantic integration, which automatically validates JSON payloads to prevent injection attacks.

6.1.3 Database & Authentication (Google Firebase/Firestore)

A critical architectural decision was the selection of the data persistence layer. Initially, a traditional relational database (PostgreSQL) was considered for its JSON support. However, it was ultimately rejected to reduce infrastructure complexity. Firestore was selected as the optimal alternative:

- **Real-Time Synchronisation:** Firestore operates as a NoSQL document database with built-in WebSocket capabilities via onSnapshot listeners. This allowed the React dashboard to subscribe to the database and automatically update the UI the exact moment the Python backend pushes a new AI analysis. This perfectly satisfied the critical Non-Functional Requirement (NFR1) for real-time alerting without the immense engineering overhead of building a custom WebSocket server.
- **Schema-less Flexibility:** Because network logs (such as Snort or Winlogbeat alerts) vary wildly in their JSON structure, Firestore's schema-less NoSQL collections inherently suit the storage of unstructured threat data far better than rigid SQL tables.
- **Delegated Security & Authentication:** Rather than manually engineering a custom login system (which introduces severe risks regarding password hashing and salt storage) Firebase Authentication was implemented. This abstracts the immense security overhead of identity management, enabling secure, token-based user sessions that strictly control access to the dashboard (NFR2).

6.1.4 AI & Logic Engine

The system's core analytical engine is powered by the Google Gemini (Flash-Lite) API. As previously discussed, locally hosted models were ruled out due to hardware limitations. By offloading the heavy analytical processing to the cloud, the application remains exceptionally lightweight and accessible on standard business hardware. Furthermore, enterprise cloud models provide vastly superior reasoning capabilities for analysing and correlating complex security threats compared to hardware-constrained local alternatives.

6.1.5 Ingestion Strategy

Rather than building a tight, rigid integration with a single Intrusion Detection System (such as Snort or Suricata), the backend Python script (`log-forwarder.py`) was designed to autonomously poll and monitor a designated local directory for file system changes. This architectural choice ensures the system is entirely IDS-agnostic. Any security tool, firewall, or custom script capable of outputting log files to a specific local folder is automatically supported. This guarantees maximum flexibility and ease of integration across diverse SME IT environments.

6.2 High-Level Architecture

To visualise the end-to-end data pipeline and establish the boundaries between local processing and cloud infrastructure, a high-level system architecture diagram was developed. The architecture is explicitly designed to isolate sensitive local network telemetry from the external cloud environment using a strict, unidirectional data flow.

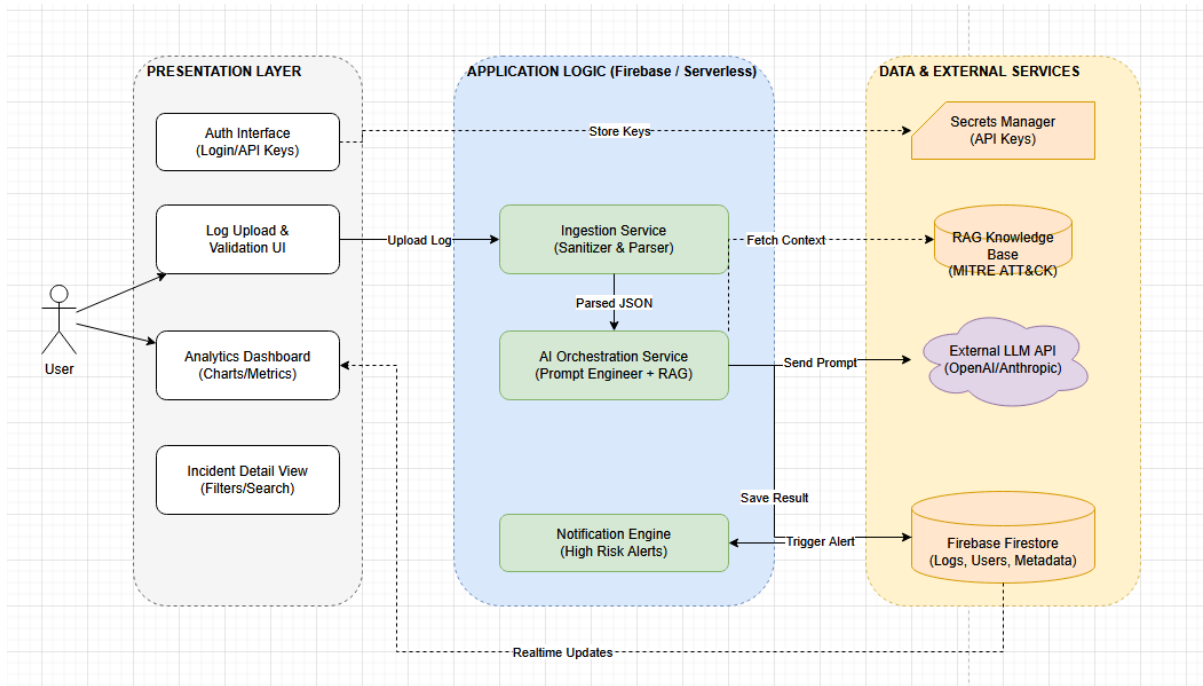


Figure 2: High-Level System Architecture diagram illustrating the data flow across the Presentation, Processing, Database, and Local layers.

As illustrated in the diagram, the S.I.R.A. data flow operates through four chronological stages:

- 1. Local Ingestion & Pre-Processing:** A local IDS (e.g., Snort) writes raw alerts to alert.ids. The Python log-forwarder.py detects these entries and applies a Regex sanitisation pipeline to strip all Personally Identifiable Information (PII), such as IP and MAC addresses, ensuring complete data anonymisation prior to transmission.
- 2. Cloud AI Processing:** Sanitised logs are batched and transmitted via HTTPS to the Google Gemini API, injected alongside a dynamic, role-based persona prompt (Business Owner, IT Support, or SOC Analyst). The LLM processes this context and returns a structured JSON payload containing a plain-English summary, a risk score (1–10), and mitigation steps.
- 3. Cryptographic Storage:** Upon receiving the LLM JSON response, the Python backend encrypts the AI insights via AES (using the cryptography.fernet library). The ciphertext payload is then pushed to Firestore, ensuring all cloud incident data is securely encrypted at rest.
- 4. Real-Time Presentation:** The React dashboard utilises Firestore onSnapshot listeners to establish a persistent WebSocket connection, eliminating manual API polling. When new ciphertext enters the database, Firestore actively pushes it to the client. The data is instantly decrypted and rendered on the UI, simultaneously triggering any configured email notifications.

6.3 Database Design

S.I.R.A. utilises Google Firestore, a NoSQL document-based database. Data is structured into discrete 'Collections' containing individual JSON-like 'Documents'.

Table 6: Initial Firestore database schema design illustrating the conceptualised processing queue and plaintext data fields.

Collection:	Users	Collection:	processing_queue
Purpose:	Stores user profile and settings (Linked to firebase auth UID)	Purpose:	Handles the "circuit breaker" and "Async" logic for uploaded files. Backend puts the jobs here; frontend watches the status.
Document ID:	user_uid	Document ID:	Auto-generated UID
Fields:	email: Google Email, role: analyst, // Future-proofing for RBAC experience_level: beginner, // [FR16] For tailoring LLM summaries created_at: Timestamp, settings: notifications_enabled: true, theme: dark LLM_API_Key:	Fields:	file_name: og_001.json, upload_timestamp: Timestamp, status: processing, // Options: pending, processing, completed, failed error_message: null, // Populated if parsing fails [Error Handling] user_id: user_uid
Collection:	Incidents	Collection:	audit_logs
Purpose:	Stores the processes logs, LLM summaries, and status.	Purpose:	Record of who did what
Document ID:	Auto Generated UID	Document ID:	Auto-generated UID
Fields:	status: open, // Options: open, investigating, resolved [FR15] severity: high, // Options: low, medium, high, critical [FR10] timestamp_detected: timestamp, source_ip: 192.168.1.50, attack_type: Brute Force, // Extracted by Parser summary: "Multiple failed login attempts detected...", // [FR6] From LLM mitigation_steps: // [FR6] From LLM Block IP 192.168.1.50, Reset Admin Password, risk_score: 8.5, // 0-10 raw_log_reference: C://bucket_name/logs/file_123.log", // Traceability assigned_to: user_uid // Optional	Fields:	actor_id: user_uid", action: resolved_incident, // or "deleted_log", exported_report target_resource_id: incident_id, timestamp: Timestamp, metadata: { previous_status: new, new_status: resolved }

The database schema underwent significant refactoring between the initial design phase and final implementation. The original design proposed a processing_queue collection to manage asynchronous parsing jobs. However, to optimise Firestore read/write quotas and reduce cloud costs, this was refactored; batching and queuing are now handled entirely in-memory by the Python orchestrator. Additionally, an integrity_hash field was initially conceptualised to cryptographically verify log authenticity. During development, this was evaluated as computationally redundant for the MVP's scope) given Firestore's inherent TLS transmission and internal security rules (and was subsequently scoped out to streamline database writes.

Table 7: Final implemented Firestore database schema illustrating the three core collections.

Field Name	Data Type	Security / Notes
name	String	Plaintext - User's display name
email	String	Plaintext - Google OAuth email
role	String	Plaintext - e.g., 'admin', 'analyst' (RBAC)
tech_level	String	Determines Gemini API persona prompt
notification_level	String	Threshold for email alerts (e.g., 'critical')
app_notification_level	String	Threshold for in-app UI alerts
cleared_notifications	Array	Tracks which alerts the user has dismissed
created_at	Timestamp	Server-side account creation time
Collection:	audit_logs	
Purpose:	An immutable forensic ledger tracking all API interactions.	
Document ID:	Auto-generated Firestore UID	
Field Name	Data Type	Security / Notes
client_ip	String	The IP address making the HTTP request
method	String	e.g., 'GET', 'POST', 'PATCH'
path	String	The specific FastAPI endpoint accessed
status_code	Integer	e.g., 200 (Success), 403 (Forbidden)
process_time	Float	Server execution time (for performance monitoring)
user_agent	String	Browser/Client software identification
timestamp	Timestamp	Server-side execution time

Field Name	Data Type	Security / Notes
is_encrypted	Boolean	Flag to trigger frontend decryption logic
data	String (Ciphertext)	[AES Encrypted] The raw sanitised log data
ai_insights	Array (Ciphertext)	[AES Encrypted] The LLM summary and mitigation steps
user_notes	Array (Ciphertext)	[AES Encrypted] Manual notes added by analysts
risk_score	Integer	Plaintext (1-10) - Used for quick Dashboard UI sorting
analysis_status	String	Plaintext - e.g., 'AI_Analysis_Complete', 'resolved'
assigned_to	String	Plaintext - User UID of the assigned analyst
completed_steps	Array [Integers]	Plaintext - Tracks mitigation progress checkboxes
timestamp	Timestamp	Server-side time of log ingestion

As visualised in Table 7, the final data model consists of:

- incidents:** This is the primary collection for storing processed alerts. To enforce strict data-at-rest encryption, sensitive fields (such as data, ai_insights, and user_notes) are stored purely as AES-encrypted ciphertext blobs. Only non-sensitive metadata required for database querying and dashboard analytics (e.g., risk_score, timestamp, analysis_status, assigned_to) is stored in plaintext. This hybrid schema ensures the React frontend can quickly generate charts without decrypting the entire database.
- users:** This collection manages Role-Based Access Control (RBAC) and user preferences. Crucially, it stores the user's tech_level (which dictates the specific persona prompt injected into the Gemini API) alongside their notification_level (which dictates the threshold for triggering automated email alerts).
- audit_logs:** To ensure strict system accountability, this collection operates as an immutable ledger. Populated autonomously by the FastAPI backend middleware, it records the client_ip, HTTP method, path, status_code, and process_time for every single request made to the API, establishing a comprehensive forensic trail against unauthorised dashboard access.

6.4 System Logic & Processing Models

To accurately map the behavioural logic and chronological data flow of the S.I.R.A. system, two distinct processing models were developed during the design phase.

6.4.1 Functional Decomposition (HIPO)

A Hierarchy plus Input-Process-Output (HIPO) diagram was constructed to break down the system into modular functional units. This established strict boundaries between core processes, such as

Log Ingestion (reading and validating .ids files), LLM Processing (batching and API interaction), and Analytics (aggregating metrics for the dashboard). Defining these distinct input-process-output boundaries ensured the system adhered to the principle of separation of concerns during development.

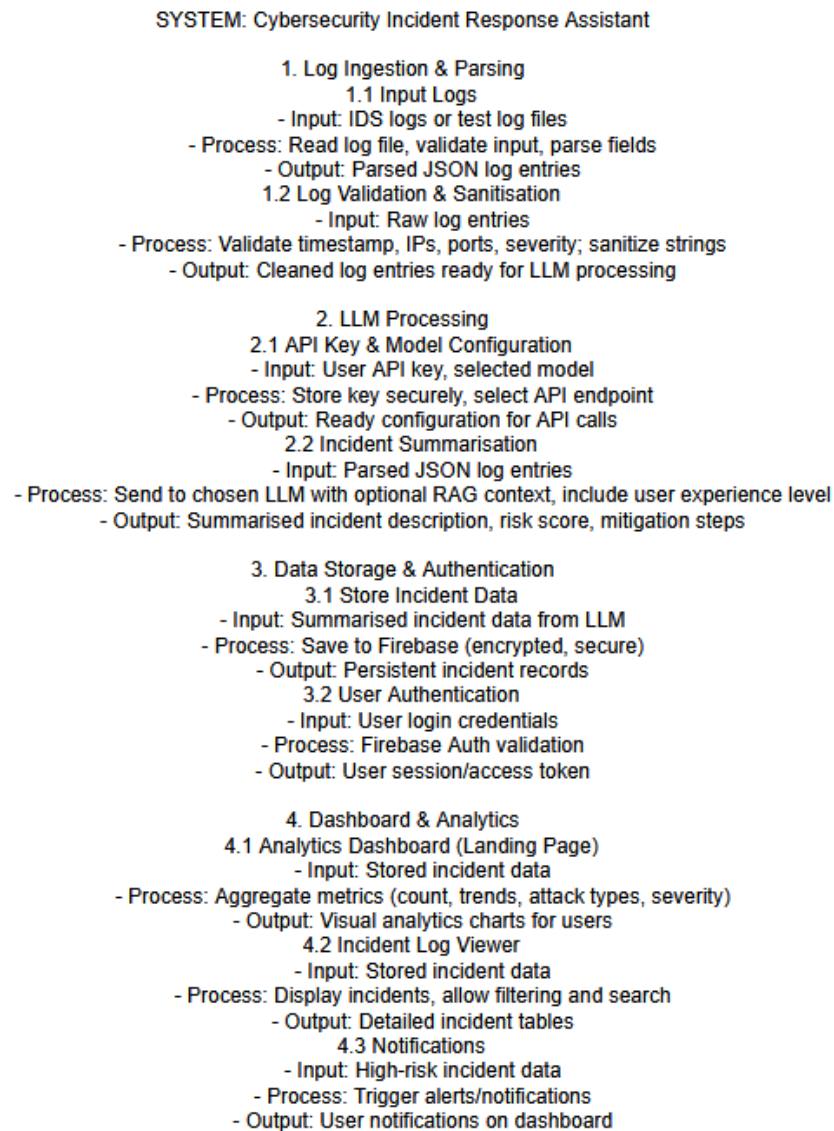


Figure 3: HIPO Diagram detailing the functional decomposition of system modules.

6.4.2 System Sequence & Interaction

While the HIPO diagram established what the system does, a Sequence Diagram was developed to map when it happens. The diagram illustrates the chronological lifecycle of a threat event, specifically highlighting the asynchronous interactions between the local Python orchestrator, the external Gemini API, and the React frontend. Crucially, it visualizes the non-blocking nature of the architecture: while the backend awaits the 10–30 second LLM response, the React dashboard remains fully interactive, only updating once the Firestore onSnapshot listener receives the final ciphertext payload.

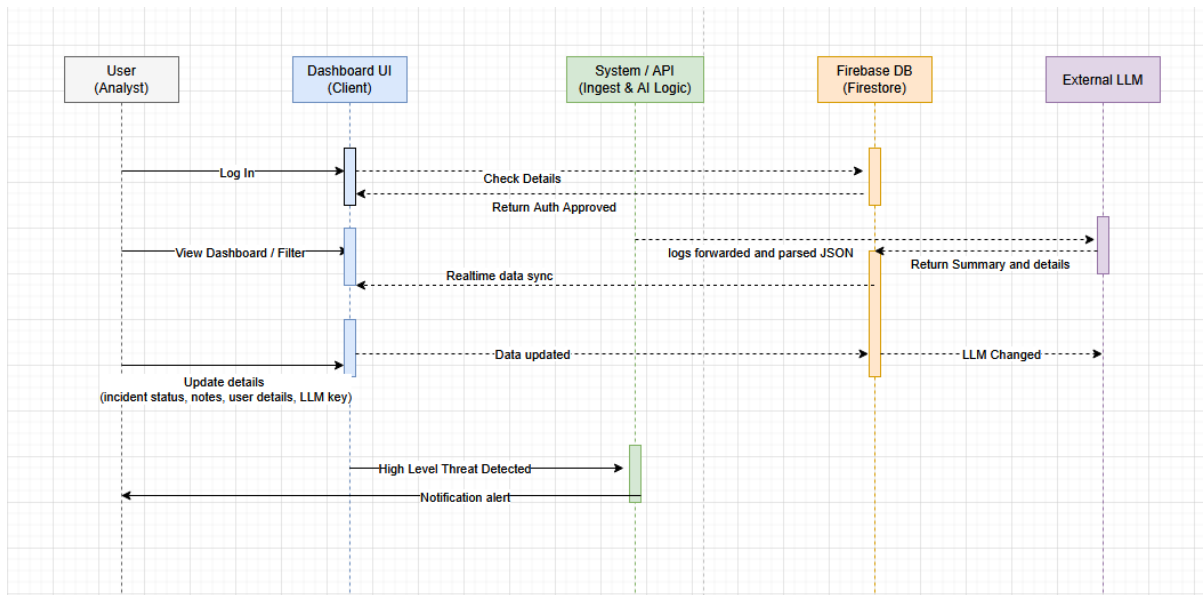


Figure 4: Sequence Diagram illustrating the chronological message passing between system components.

6.5 System Resilience & Error Handling

A critical component of the system design involved mapping potential failure domains. Rather than allowing the application to crash upon encountering edge cases, a comprehensive error-handling matrix was developed to ensure system resilience and graceful degradation.

Table 8: Error-Handling matrix detailing system failure domains.

Failure Domain	Scenario	Backend	Frontend
Ingestion	User uploads a corrupted file or unsupported format (e.g., PDF).	IngestionService checks MIME type and magic bytes. Returns 415 Unsupported Media Type or 422	Show a red "Toast" notification: "Invalid file format. Please upload .log or .txt files only."
Ingestion	watchdog detects a file, but it is locked (still being written to by IDS).	Implement a try/except loop (wait 1s, then 2s, then 4s) before attempting to read.	N/A (Background process). Log the retry attempt to system logs.
LLM Logic	The OpenAI/Anthropic API Key is invalid or expired.	Catch AuthenticationError from the LLM SDK. Update the incident status in Firebase to "Analysis Failed".	Dashboard shows an alert on the specific incident card: "Analysis Failed: Check your API Key settings."
LLM Logic	The LLM API times out (takes > 30s) or returns 503 Service Unavailable.	Catch TimeoutError. Do not retry immediately (to avoid rate limits). Mark incident as "Pending Retry" and queue a background task to retry in 1 minute.	UI shows a "Spinner" with status: "AI Provider is busy. Retrying in 60s..."
Database	Firebase Quota Exceeded or Permission Denied.	Catch FirebaseError. If it's a quota issue, stop processing new logs to prevent data loss.	Show a global banner: "System is in Offline Mode. Logs are queued locally."
Parsing	Log line does not match the expected Regex (e.g., weird timestamp format).	Partial Failure Strategy: Log the specific line number that failed to a "Dead Letter Queue" (error log) but continue processing the rest of the file.	Show a warning after upload: "Processed 98/100 lines. 2 lines were skipped due to formatting errors."

As detailed in Table 8, the system architecture was designed to anticipate failure across both local and cloud environments. The matrix maps how backend faults (such as network timeouts or API authentication errors) should safely queue data locally rather than causing permanent data loss.

However, while these robust failure domains were thoroughly mapped during the design phase, applying Agile MoSCoW prioritisation dictated that the complex frontend UI feedback mechanisms (such as global offline banners and specific toast notifications) be scoped out of the Minimum Viable Product (MVP). Documenting these strategies ensures the architecture is prepared for future enterprise iterations, while allowing core development to remain strictly focused on the primary analytical pipeline.

6.6 User Interface Design

To make the S.I.R.A. system accessible for non-technical users (FR8), the UI and UX were designed to be highly intuitive. The interface was built iteratively using Agile methods, relying on continuous feedback rather than rigid upfront planning.

6.6.1 Initial Low-Fidelity Prototyping

The initial design phase involved sketching paper wireframes and low-fidelity mockups to establish the core layout. (Initial designs can be found in Appendix D).

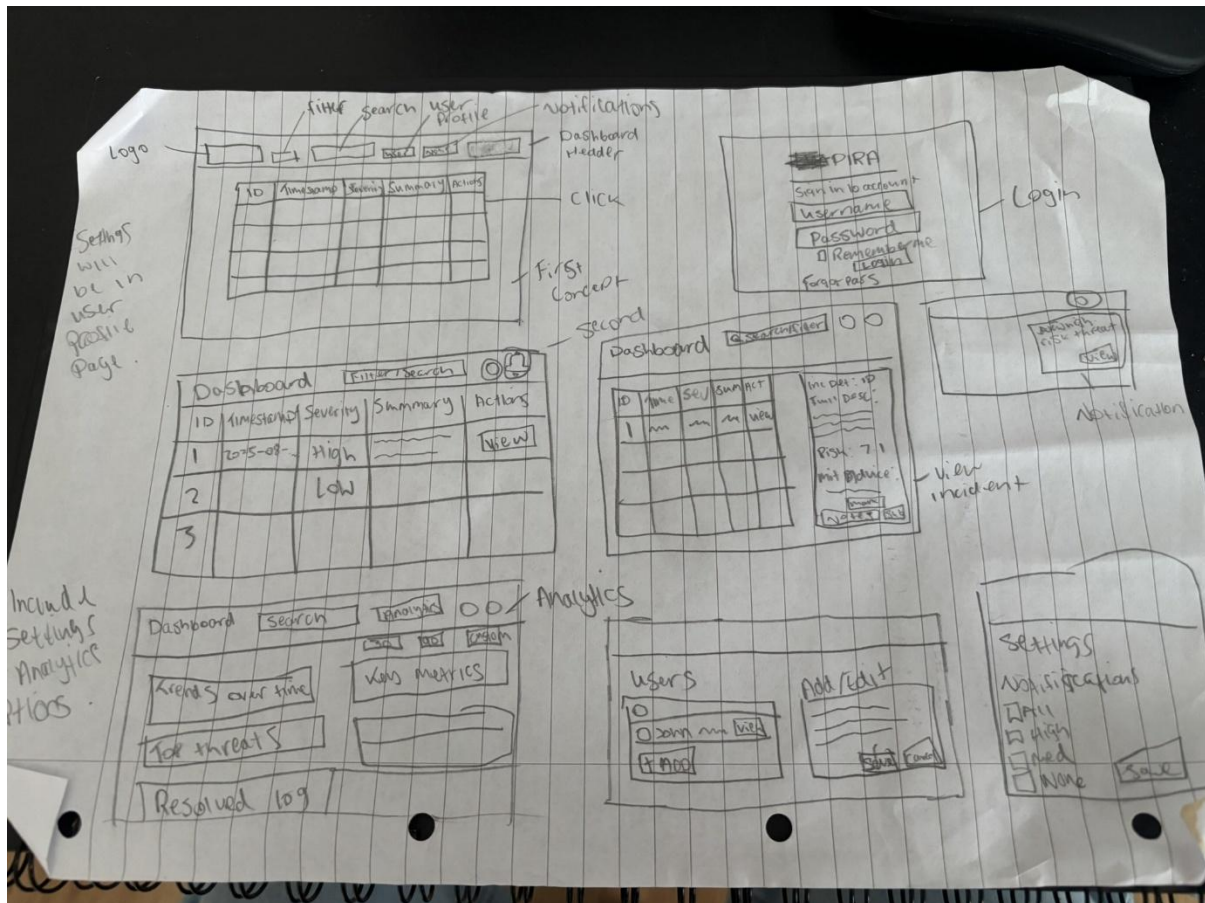


Figure 5: Initial low-fidelity paper prototype for the system dashboard.

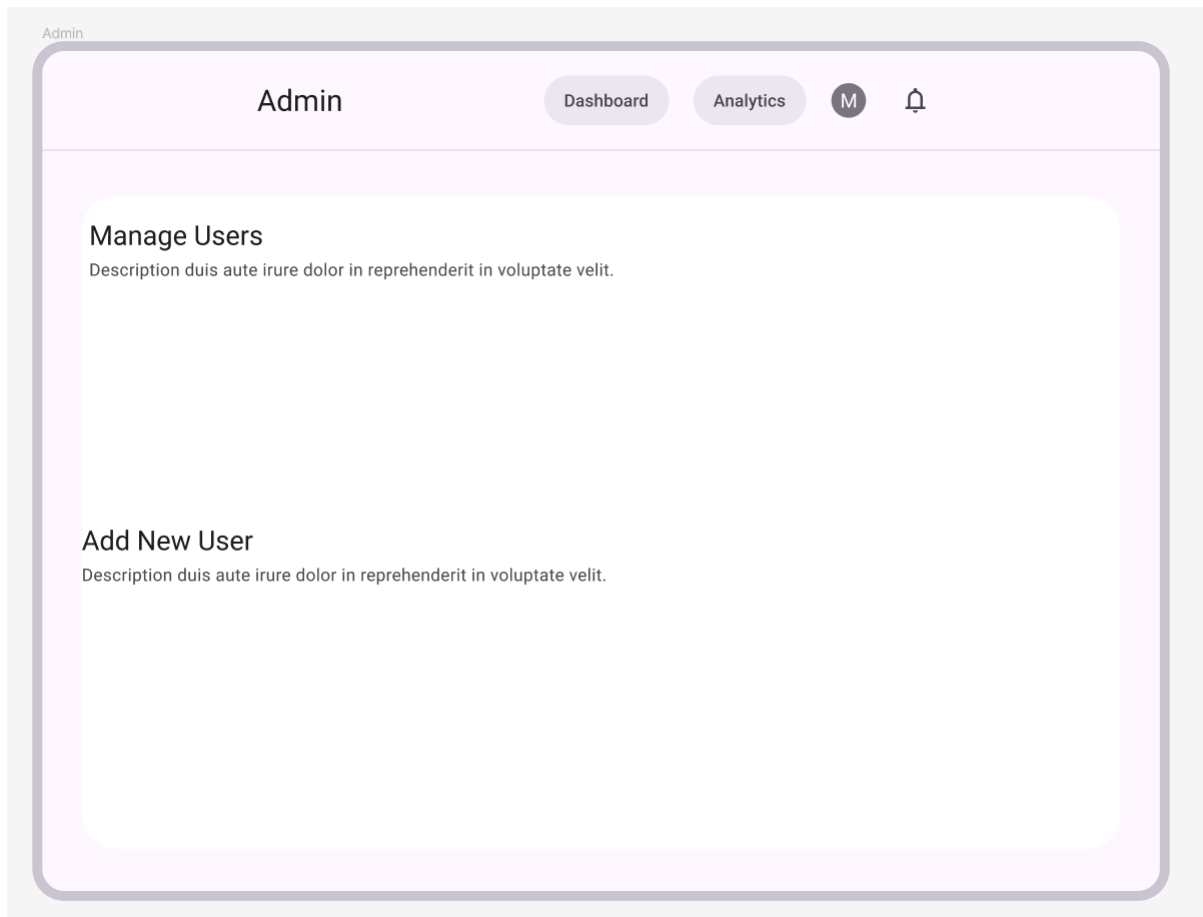


Figure 6: Initial wireframe for the proposed Admin Management page.

As seen in the initial designs, the core structure included an Analytics page for key metrics, an Incidents log, and a Settings page. The original design also included a dedicated 'Admin Page', which was intended to allow business owners to manually create and manage employee accounts. Furthermore, viewing an incident was initially designed to open as a small pop-up window over the main dashboard.

6.6.2 Agile Feedback & Design Iteration

Following supervisor feedback and Agile self-evaluation, several critical changes were made to the UI to reduce cognitive load and improve system usability. The wireframes were subsequently upgraded to high-fidelity designs.

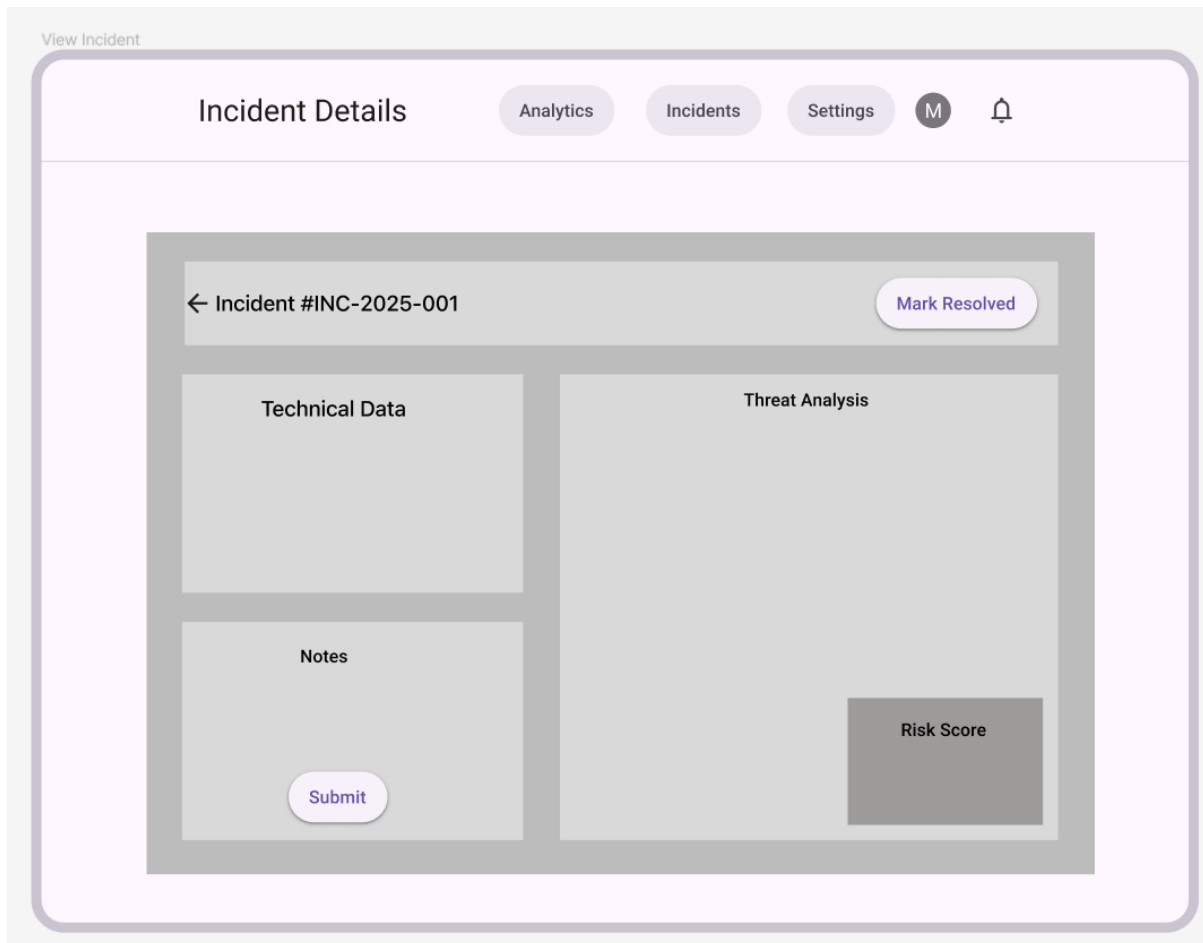


Figure 7: Final high-fidelity design for the dedicated 'View Incident' page, featuring the persistent top navigation bar.

(See Appendix E for the full suite of final high-fidelity UI screenshots).

The key justifications for the design evolution are as follows:

- **Removal of the Admin Page:** The dedicated user management page was completely scoped out of the final UI. Since the backend utilises Google Firebase, user accounts and role management are handled entirely via the secure Firebase Console. Removing this feature from the React frontend eliminated unnecessary complexity and saved significant development time.
- **Consolidation of Resolved Logs:** Originally, 'Resolved Logs' were going to have their own separate page. To reduce UI clutter, this was replaced with a simple status filter on the main Incidents page, creating a single, centralized repository for all threat data.
- **Redesigning 'View Incident':** The initial idea of using a pop-up modal to view incident details was abandoned. Because the Gemini LLM generates highly detailed summaries and step-by-step mitigation advice, a small pop-up felt too restrictive. Instead, it was moved to a dedicated, full-screen page to provide maximum screen real estate for reading technical advice comfortably.
- **Expanded Settings & Navigation:** A permanent top navigation bar was implemented to allow users to switch pages seamlessly. Furthermore, the Settings page was heavily expanded based on early feedback. Rather than hardcoding API keys, the UI was updated to allow users to securely input their own Gemini API keys and dynamically select their

'Technical Persona' (e.g., Business Owner vs. IT Support), ensuring the system remains highly customisable.

7. Development & Implementation

This chapter outlines the translation of core system requirements into functional code, highlighting the key technical mechanisms driving the localised log-parsing pipeline, the AI integration, and the React frontend, while detailing how critical technical challenges were overcome.

7.1 Data Origin & Local Ingestion

The operational lifecycle of the S.I.R.A. system begins at the local network level. The architecture relies on external detection engines, such as Snort (for network intrusion) or Winlogbeat (for Windows system events), to generate raw logs. The primary engineering challenge for this stage was capturing this continuous stream of data efficiently without overwhelming the host machine.

7.1.1 Continuous Monitoring

To ingest these logs, a custom Python backend service was developed. Rather than reading entire log files from start to finish every time the script executes, the `log_watcher` function was engineered to be "stateful". By utilising `os.path.getsize()`, the system acts as a digital bookmark. It records the exact byte position where it last finished reading in a local `log_progress.json` file. Consequently, when Snort or Winlogbeat appends a new alert to a file, the Python script jumps directly to that new line, ensuring rapid, deduplicated data ingestion.

```

# Get current file size
try:
    current_size = os.path.getsize(src_path)
except OSError:
    continue # File might be locked or deleted

# Get the last known position (default to 0 if new file)
last_pos = file_progress.get(file, 0)

# If the file is smaller than before, it was likely truncated/rotated. Reset to 0
if current_size < last_pos:
    last_pos = 0

# If there is NEW data (current size > last position)
if current_size > last_pos:
    try:
        with open(src_path, "r", encoding="utf-8", errors="replace") as f:
            f.seek(last_pos) # JUMP to where we left off
            new_lines = f.readlines()

            if new_lines:
                log_sanitiser(new_lines, file) # Process ONLY the new lines
                new_data_found = True

            # Update "bookmark"
            file_progress[file] = f.tell()
            save_file_progress(file_progress)

    except Exception as e:
        print(f"Error reading {file}: {e}")

```

Figure 8: Code snippet showing the "digital bookmarking" mechanism used to prevent duplicate processing of log files.

7.1.2 Initial Triage and Noise Reduction

Once a new log line is ingested, it undergoes immediate local triage. Because the architecture relies on a free-tier external LLM, it is financially and computationally unviable to process every piece of benign background traffic. To solve this, the script implements a two-tier filtering mechanism:

- **Noise Reduction:** The logs are first checked against a KNOWN_SAFE_PATTERNS array. This aggressively discards safe, high-volume events (such as standard DHCP requests or successful Windows Event 4624 logons) locally, preventing them from consuming further system resources.
- **Threat Identification:** If the log bypasses the noise filter, it is evaluated against a custom ThreatDictionary. This dictionary contains known indicators of compromise categorised by attack type (e.g., SQL injection strings like ' OR '1'='1').

By implementing this, the system guarantees that only genuinely suspicious lines advance to the next stage of the pipeline, significantly optimising performance. (See Appendix J, Figures 1 and 2).

7.2 Privacy & Data Sanitisation

Once a log entry is flagged as suspicious by the local triage phase, it enters the pre-processing pipeline. Because the S.I.R.A. system relies on sending data to a third-party cloud API (Google Gemini), transmitting raw logs directly would pose a severe security risk. Raw network logs

frequently contain Personally Identifiable Information (PII) and sensitive internal network layouts, which would violate data protection laws like the UK GDPR if leaked or stored externally.

7.2.1 Deep Text Sanitisation (Regex)

To guarantee data privacy, the Python script executes a rigorous local sanitisation process before any internet connection is made. Using complex Regular Expressions (Regex), the system actively scans the text to find and scrub sensitive data points. This includes stripping plain-text passwords, email addresses, MAC addresses, and specific Windows user directory paths, replacing them with safe, generic tags (such as [EMAIL_REDACTED] or [PASSWORD_REDACTED]).

```
#sanitisation
# remove mac address entirely for same reason as ip
macs = re.findall(pattern_mac, text_to_analyze) # find and store before redacting the text, same
sanitised_line = re.sub(pattern_mac, "[MAC_REDACTED]", text_to_analyze) # replace all mac occurances
# New System-Level Sanitisation
sanitised_line = re.sub(pattern_email, "[EMAIL_REDACTED]", sanitised_line)
sanitised_line = re.sub(pattern_win_user_path, "[WINDOWS_USER_DIR]", sanitised_line)
sanitised_line = re.sub(pattern_passwords, "[PASSWORD_REDACTED]", sanitised_line)
```

Figure 9: Code snippet demonstrating the regex sanitisation used to strip PII.

7.2.2 Intelligent IP Masking

A major engineering challenge during this phase was dealing with IP addresses. If the system simply deleted all IP addresses, the AI would lose critical context—it would not be able to tell if one attacker was scanning ten machines, or if ten attackers were scanning one machine.

To solve this, an intelligent anonymisation function was built. The script extracts all unique IPv4 and IPv6 addresses from the log and checks their prefixes to categorise them as internal (e.g., starting with 192.168. or 10.) or external. It then replaces them in the text with sequential placeholders, such as [EXTERNAL_IP_0] and [INTERNAL_IP_1]. It ensures the external LLM never learns the SME's actual network layout, whilst accurately track an attacker's movements across the network.

```
# Extract both IPv4 and IPv6 addresses
ips_v4 = re.findall(pattern_ip, sanitised_line)
ips_v6 = re.findall(pattern_ipv6, sanitised_line)

# Use a set to get unique IPs, then sort them to ensure consistency
# and combine both ips into a single unique list
unique_ips = sorted(list(set(ips_v4 + ips_v6)))
# Define all internal prefixes (both v4 and v6)
internal_prefixes = ("192.168.", "10.", "172.", "fc", "fd", "fe80", "::1", "127.0.0.1")
# Separate into internal and external lists by checking the prefix
internal_ips = [ip for ip in unique_ips if ip.lower().startswith(internal_prefixes)]
external_ips = [ip for ip in unique_ips if ip not in internal_ips]

# External IPs now starting from 0
for i, ip in enumerate(external_ips):
    sanitised_line = sanitised_line.replace(ip, f"[EXTERNAL_IP_{i}]")
# Internal IPs now starting from 0 (separate to external ips)
for i, ip in enumerate(internal_ips):
    sanitised_line = sanitised_line.replace(ip, f"[INTERNAL_IP_{i}]")
```

Figure 10: Implementation of the IP pseudonymisation logic.

7.3 AI Integration

With the log data completely sanitised, it is ready to be analysed by the Gemini LLM. However, interacting with cloud APIs and databases introduces new challenges regarding network efficiency and data security.

7.3.1 Asynchronous Batching

During early development, the script sent a new API request for every single suspicious log. Testing quickly revealed that this approach instantly triggered the Gemini API's rate limits (HTTP 429 errors), causing the system to crash. To resolve this, an in-memory queueing system was developed. The script now collects up to 50 logs into a single 'batch' before making a single, consolidated call to the LLM. A timeout mechanism was also added to ensure that if only a few logs are generated, they do not get stuck waiting in the buffer indefinitely. (See Appendix J, Figures 3–5).

7.3.2 Dynamic AI Personas

Before the batch is sent to the LLM, the backend queries the user's settings to determine their technical level. It then injects a specific prompt—either 'Business Owner', 'IT Support', or 'SOC Analyst'—alongside the log data. Furthermore, the prompt strictly commands the LLM to return its analysis in a highly structured JSON format. Enforcing JSON output ensures the React frontend can reliably map the AI's response to the dashboard UI without errors. (See Appendix J, Figures 6–7).

7.3.3 Cryptographic Storage

Once the LLM returns the mitigation playbook, the data must be stored in Firestore so the web dashboard can read it. To protect this data at rest and during transit, the Python backend uses the `cryptography.fernet` library to encrypt the AI's insights via AES before uploading them to the database. Initially, this stage also included a cryptographic hashing feature designed to verify the integrity of logs on the frontend. However, it was concluded that it was out of scope and was not necessary.

```

backend > src > security > crypto.py > ...
1  #crypto.py used for encrypting the incident data and ai summary within log-fowarder and decrypting it when retrieving from firestore
2  import os, json
3  from cryptography.fernet import Fernet
4  from dotenv import load_dotenv
5
6  # Uses a relative path so can be reused and works on any machine
7  ENV_PATH = os.path.join(os.path.dirname(__file__), "..", ".env")
8  load_dotenv(ENV_PATH)
9
10 def get_or_create_key():
11     key = os.getenv("ENCRYPTION_KEY")
12     if not key:
13         new_key = Fernet.generate_key().decode()
14         with open(ENV_PATH, "a") as f:
15             f.write(f"\nENCRYPTION_KEY={new_key}") # adds a new key to .env if one doesnt exist
16         return new_key
17     return key
18
19 # Initialize once imported
20 raw_key = get_or_create_key()
21 cipher = Fernet(raw_key.encode()) # fernet uses AES-128 in CBC mode with HMAC for auth. encode converts key to bytes
22
23 def encrypt_payload(data_dict: dict) -> str:
24     json_data = json.dumps(data_dict, default=str).encode() # converts a dict to JSON bytes, encrypts it and returns a string
25     return cipher.encrypt(json_data).decode()
26
27 def decrypt_payload(encrypted_string: str):
28     # If fit gets a list instead of a string, catch it here
29     if not isinstance(encrypted_string, str):
30         print(f"Error: Expected string for decryption, got {type(encrypted_string)}")
31         return None
32
33     try:
34         decrypted_bytes = cipher.decrypt(encrypted_string.encode())
35         return json.loads(decrypted_bytes.decode())
36     except Exception as e:
37         return None

```

Figure 11: Implementation of the crypto.py module, utilizing Fernet for AES-128 payload encryption and decryption.

7.4 Frontend Dashboard

The final stage of the S.I.R.A. architecture involves retrieving the processed, encrypted data from the cloud and presenting it to the user in an accessible format. The frontend dashboard was developed using React and Tailwind CSS, ensuring a responsive and modern user interface that caters specifically to non-technical SME owners.

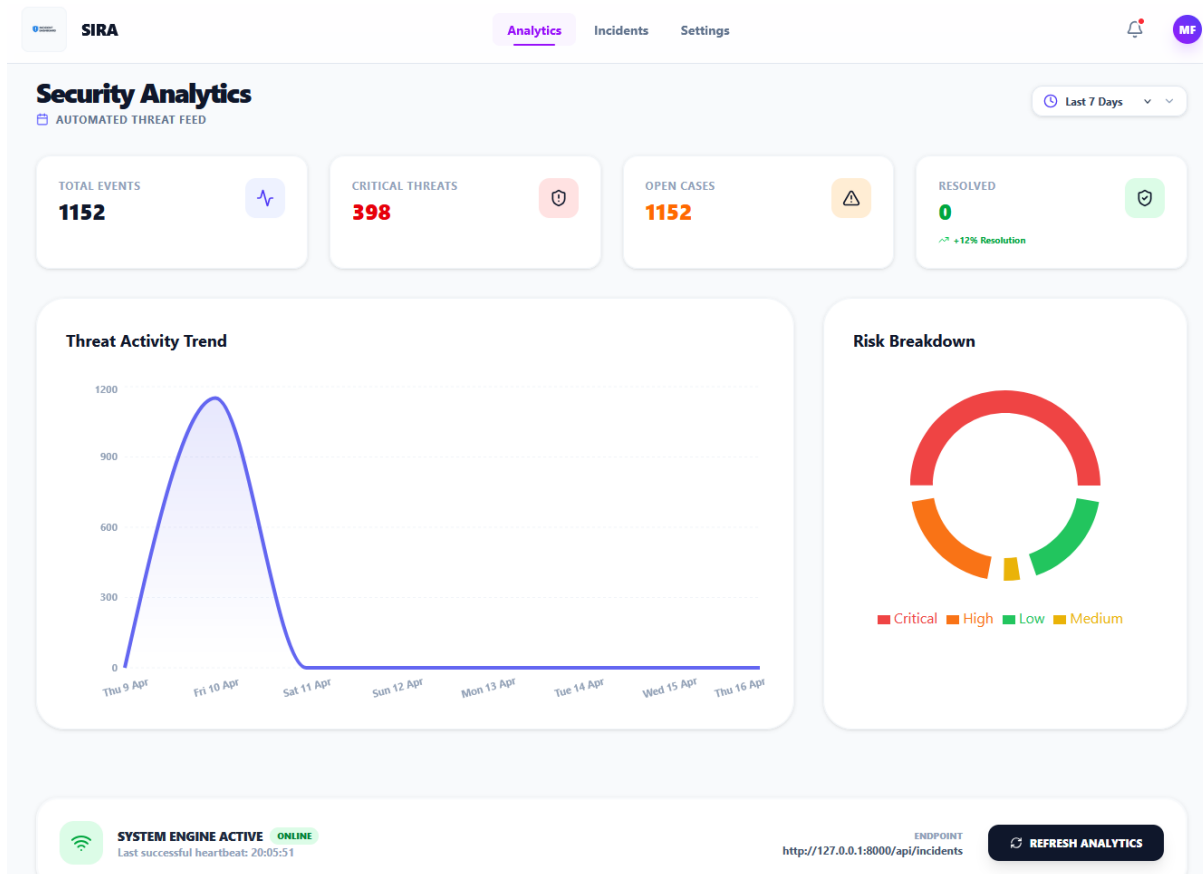


Figure 12: Screenshot of the S.I.R.A. main analytics dashboard.

7.4.1 Real-Time Data Synchronisation

A traditional web application often requires the user to manually refresh the page or relies on inefficient 'polling' (constantly asking the server if new data exists). To provide a seamless experience, the dashboard utilises Firebase onSnapshot listeners. This establishes a persistent WebSocket connection between the React frontend and the Firestore database. When the Python backend uploads a newly analysed threat, Firestore actively pushes this update to the React client. This guarantees that the dashboard UI updates instantly and automated email notifications are triggered without any manual input from the user, ensuring critical threats are surfaced in real-time. (See Appendix J, Figures 8–10).

7.4.2 Analytics and Data Visualisation

To help business owners quickly understand their security posture, the dashboard features a dedicated analytics page generating visuals like the 'Threat Activity Trend' and 'Risk Breakdown' charts. A critical architectural decision here was the implementation of a hybrid database schema. While the sensitive AI threat summaries are stored securely as encrypted ciphertext, basic metadata is deliberately stored in plaintext. By allowing the Recharts library to read this directly, the system avoids the massive computational overhead of downloading and decrypting everything just to count incidents. This ensures the analytics dashboard loads instantly, even when processing thousands of logged events.

```
analysis_status: "AI_Analysis_Complete"

data: "gAAAAABp2MhDOU3koPYF5YCV0IK6uLKtN1B-
3ywla5CZ3nrA2SCMXPO-
ReDs6UzFY01znoxRg8Mvyh8c8waWDNaQrfEk5F_rS
LMd5n5HjPY_BGR7nFdV1M8PlZ7IkUIXl4BjEa7jdgGLg
POD5Jux2ppfiHGiBt-
LlGwwK1GxMngmoaBpv2xfQyM5DhuDMpW8LYMER
vCQqi81SE0tFFaYkwz7ecA9xS3esX1jLK9COOmYxcC
WeQe1s9lXld6T5-
kSS9wDA7Ejh-
6VlcVWAwdOG7ytpC8ghHPnod0qpf9aKFt52jeH_xV
8wa4ha9VNX2SacSejqLYc7vWKGdQ3zwdlFj1Ad7vy8
LJuHrYqf8aOXP9MWSrudyDmEOF9obW356SDV7U6l

is_encrypted: true

risk_score: 7

timestamp: 10 April 2026 at 10:51:45 UTC+1
```

Figure 13: Screenshot of the unencrypted metadata in firestore.

7.4.3 Rendering the Mitigation Playbooks

When a user clicks on a specific incident, the React frontend retrieves the heavily encrypted `ai_insights` payload. Once securely decrypted, the structured JSON generated by the Gemini LLM is mapped directly into the UI components. The system dynamically renders the 'Incident Overview', a colour-coded 'Risk Score', and the step-by-step 'Mitigation Plan'. This final mapping process is the core realization of the project's primary aim. It successfully translates complex, raw technical telemetry into a highly readable, actionable interface, empowering users of any technical background to confidently understand and remediate cyber threats. (See Appendix J, Figures 11 – 12).

7.5 The Secure API Gateway

While the React dashboard relies on Firestore's `onSnapshot` listeners for real-time update notifications, retrieving and rendering the actual content required a more robust handling mechanism. To manage this secure data flow, a custom REST API was developed using FastAPI.

7.5.1 The Cryptographic Gateway

Because the Python ingestion script encrypts all sensitive AI insights before uploading them to Firestore, passing this ciphertext directly to the browser would require exposing the AES decryption keys in the frontend code. To mitigate this, FastAPI was deployed as a strict cryptographic middleman to handle two-way data translation:

- **Secure Decryption:** When the dashboard fetches an incident, the API retrieves the encrypted data from Firestore, decrypts the AI insights using its secure server-side keys, and passes the readable plaintext to the React UI for rendering.
- **Secure Encryption:** Conversely, if the frontend were permitted to write data directly to the database, sensitive user notes could be intercepted. When a user submits an investigation

note, the frontend sends the raw text to the API. FastAPI validates the JSON payload using Pydantic, encrypts the text, and pushes the resulting ciphertext to the database.

7.6 Overcoming Challenges

7.6.1 Intelligent Caching and Quota Management

A critical challenge was the rapid exhaustion of cloud database read quotas, caused by the frontend requesting full data reads on every page load. To overcome this, AI was utilised similarly to Stack Overflow, to explore syntax and debug state management strategies. Using these insights, a local caching mechanism was engineered into the API layer. When the frontend requests data, the FastAPI server now checks if any new incidents have been ingested. If the database state remains unchanged, the API serves the data directly from its cache. This significantly reduces latency, eliminates unnecessary database reads, and successfully protects the free-tier quotas. (See Appendix J, Figures 13 – 15).

7.6.2 AI Rate Limiting and Prompt Evolution

While the batching mechanism (detailed in Section 7.3.1) successfully resolved the API rate-limiting issues, the next major challenge was ensuring the AI's output was useful for the target audience.

A major challenge was ensuring the AI's output was actionable for non-technical users. Initially, generic prompts produced highly technical, jargon-heavy responses, rendering the mitigation steps unusable for business owners without IT teams.

For the 'Business Owner' persona, the LLM is explicitly commanded to "provide absolute clarity without technical jargon." Crucially, to accommodate micro-SMEs, the prompt was further updated to warn the AI that "not all businesses will have an IT team." This ensures the model generates practical mitigation steps the owners can execute independently, rather than unhelpfully advising them to escalate the issue. This iterative engineering guarantees that complex threat intelligence is consistently translated into accessible, operational steps. [See Appendix J, Figures 16 - 18]

7.6.3 Frontend Graph and CSS

Initially, the 'Threat Activity Trend' graph rendered completely empty. Debugging revealed this was caused by timezone offsets between the backend server and the client's browser, which recorded the day of an event differently. This was resolved by standardising a strict YYYY-MM-DD date format across both the backend and frontend, ensuring the data grouped correctly.

Additionally, Tailwind CSS initially failed to load on the frontend. While the packages were correctly installed, the global stylesheet had not been imported into the application's root file (main.tsx). Adding this missing import successfully applied the styling across the dashboard.

7.6.4 System Integration

Attempting to run the open-source Snort IDS natively on Windows failed repeatedly, as the default configuration files were strictly formatted for Linux directory structures. The .conf files required massive manual modification, and a custom .bat script was written to forcefully generate the alert.ids files required by the Python watcher.

Also, Hardcoded file paths for log directories and environment variables (.env) broke instantly when moving the project between development machines. This was resolved by implementing `os.path.dirname(__file__)` in Python, ensuring the backend dynamically resolves relative paths regardless of the host machine's specific file structure.

7.6.5 Modular Architecture and API Key Management

During the later stages of development, an attempt was made to refactor the AI integration into a highly modular architecture. The objective was to isolate the LLM logic into a dedicated file, which would enable future support for multiple AI providers (e.g., OpenAI and Gemini) and allow users to securely input their own personal API keys via the React settings dashboard.

While the frontend interface was successfully built and the key could be transmitted to the backend, a critical architectural limitation arose regarding backend state management. I could only retrieve and apply the custom API key globally across the entire application, rather than securely isolating it on a strict per-user basis. The key would also be exposed in the frontend which I would need more time to secure and prevent.

8. Testing & Evaluation

A comprehensive testing strategy was implemented to verify the system's functionality, security, and overall resilience. This focussed on unit testing of individual components, end-to-end integration testing of the complete data pipeline, and User Acceptance Testing (UAT) to assess the dashboard's efficacy from the perspective of the target end-users.

8.1 Testing Strategy

8.1.1 Unit testing

To ensure the Security Incident Response Assistant (S.I.R.A.) operates securely and reliably, a comprehensive full-stack testing strategy was implemented. During this phase, AI tools were used assistively to review code readability and suggest complex edge cases for validation. Backend stability was verified through automated Python unit tests for core logical functions, alongside manual API endpoint testing via Swagger UI to confirm data validation and security rules. Frontend integrity was validated using automated component testing via Vitest and React Testing Library. By mocking the live database connection, the frontend tests deterministically proved that all user interfaces, mathematical aggregations, and error boundaries function flawlessly without risking the production environment. (See Appendix F for the Actual Unit Test)

Table 9: Backend Unit Test Results

UT-02	log_sanitiser() -> Hardware & Email Scrubbing	To verify that the function successfully identifies and redacts MAC addresses and corporate email addresses to prevent physical and personnel identity exposure.	Alert triggered by matthew.fish@students.plymouth.ac.uk on device 00:1B:44:11:3A:B7	Alert triggered by [EMAIL_REDACTED] on device [MAC_REDACTED]	Pass.
UT-03	log_sanitiser() -> Sensitive Path & Credential Scrubbing	To ensure that plaintext passwords captured in raw HTTP/SQL logs, alongside identifiable Windows user directories, are forcefully redacted.	Execution in C:\Users\MattFish\app.exe with password=Secret123!	Execution in C:\Users\WINDOWS_USER_DIR\app.exe with password=[PASSWORD_REDACTED]	Initial Fail -> Iterative Pass. The re.sub engine misinterpreted the C:\Users\ file path replacement string as an invalid Unicode escape sequence (\U), causing a PatternError crash. The script was immediately refactored to utilise a Python raw string literal (r"..."), resolving the crash and successfully redacting the target directory.
UT-04	encrypt_data() and decrypt_data()	To verify the functional integrity of the AES cryptographic module by testing if a plaintext string can be encrypted into a ciphertext blob, and subsequently decrypted back to its exact original state without corruption.	plaintext = "Critical SQL Injection Attempt Detected"	decrypted_text = "Critical SQL Injection Attempt Detected"	Pass.
UT-05	get_ai_persona() (Dynamic Prompt Generation Logic)	To verify that the role-based logic engine successfully intercepts the user's technical proficiency setting and maps it to the correct predefined prompt dictionary before dispatching telemetry to the Google Gemini API.	"business_owner", "soc_analyst", and an invalid edge-case input "unknown_role".	The function correctly returns the simplified executive summary instructions for the business owner, the advanced technical instructions for the SOC analyst, and safely defaults to the business owner profile when an invalid role is provided to prevent API crashes.	Pass.
UT-06	log_sanitiser() -> JSON Extraction Logic	To verify the system's ingestion flexibility. The test evaluates whether the ingestion pipeline can dynamically identify and parse structured JSON logs (e.g., Winlogbeat) to extract EventID and message fields, while safely falling back to plain-text reading for unstructured logs (e.g., Snort.ids) via exception handling without crashing.	Two distinct inputs: A structured JSON string ({ "message": "Failed login", "winlog": { "event_id": "4625" }}) and an unstructured plaintext Snort alert.	For the JSON input, the script successfully navigates the dictionary to output "EventID 4625: Failed login". For the plaintext input, the JSONDecodeError exception is safely caught, returning the raw string unaltered.	Pass.

UT-07	is_suspicious(line)	To verify the accuracy of the threat-hunting heuristic engine. The test ensures that logs containing known Indicators of Compromise (IoCs), such as SQL injections or brute-force attempts defined in the ThreatDictionary, correctly flag the is_suspicious boolean to trigger LLM analysis.	Two strings: "Failed password for user admin" and "User loaded the homepage".	The engine correctly returns True for the malicious string and False for the benign string.	Pass
UT-08	is_noise(line)	To verify the efficiency of the system's noise-reduction logic. By filtering out high-volume, low-risk telemetry (e.g., Windows Event 4624 "Successful Logon"), this function prevents the system from exhausting Gemini API rate limits and incurring unnecessary cloud processing costs.	"WinEventLog: 4624 Successful Logon"	The function correctly identifies the substring and returns True, ensuring the log bypasses the LLM processing queue.	Pass
UT-09	save_file_progress() and get_file_progress()	To verify the fault tolerance and state persistence of the file ingestion engine. The test ensures that the exact byte-read position of a given log file is successfully serialized into a local JSON tracker and can be accurately deserialized. This guarantees that if the Python backend restarts, it will not re-ingest duplicate data.	A dictionary mapping file names to byte integers: {"apache_attack.log": 1054}.	The JSON file is successfully created on the disk, and subsequent read operations extract an exact match of the dictionary.	Pass
UT-10	should_notify(pref, risk)	To verify the mathematical logic determining when critical email alerts are dispatched. The test ensures that notifications strictly obey the user's predefined threshold settings (e.g., bypassing a medium-risk alert if the user requested only critical alerts).	Matrix of preferences and AI scores (e.g., pref="critical", risk=7).	The boolean logic returns False for alerts beneath the threshold, and True for alerts meeting or exceeding it, preventing alert fatigue.	Pass
UT-11	process_batch() -> LLM JSON Parsing	To verify system resilience against LLM hallucinations. The test evaluates whether the ingestion pipeline will crash if the Gemini API returns a malformed JSON object missing the required mitigation_plan array.	A simulated, incomplete JSON dictionary simulating a degraded LLM response.	Instead of raising a KeyError, the system's .get() methods safely extract default values and trigger the hardcoded "Review logs manually" string to ensure database continuity.	Pass
UT-12	process_batch() -> Live LLM API Integration	To verify the end-to-end functionality of the external Gemini API connection. The test evaluates whether the system correctly constructs a multi-log string batch, successfully transmits it to the external API, and accurately enforces prompt-engineering constraints to prevent "Data Coalescence" (a known LLM hallucination where multiple logs are merged into a single summary).	A simulated batch array containing 3 distinct threat logs, pushed to the live Gemini API endpoint.	A successful 200 OK HTTP response from Google's servers, returning a cleanly formatted JSON array containing exactly 3 analysis objects mapped precisely to the input event_ids.	Pass

Table 10: Frontend API Unit Test Results

API-UT-01	Global Authentication Dependency (get_api_key)	To prove the API successfully blocks any user or system from accessing the database if they do not provide a valid API key	Attempted to view the /api/users data via Swagger UI without logging in (no authorization token provided).	Server returns HTTP 401 403 Forbidden.	Pass
API-UT-02	NoteRequest String Validation	To prove the API protects the database from being overloaded or crashed by users submitting maliciously large blocks of text.	Submitted a new incident note to the /api/incidents/{id}/notes endpoint that intentionally exceeded the strict 1000-character limit.	Server returns HTTP 422 Unprocessable Entity.	Pass
API-UT-03	Mitigation Progress Type Validation	To prove the API strictly enforces data types, preventing invalid text strings from being injected into fields that are only supposed to hold numbers.	Attempted to submit a word ("hello") into the /api/incidents/{doc_id}/mitigate endpoint, which specifically requires an array of numbers.	Server returns HTTP 422 Unprocessable Entity.	Pass
API-UT-04	resolve_incident() Endpoint Logic	To prove that when an authorised user provides the correct API key and properly formatted data, the API successfully processes the request and updates the Firestore database.	Dispatched a valid, authorised request to the /api/incidents/{id}/resolve endpoint using a real incident ID.	Server returns HTTP 200 OK with {"status": "success"}.	Pass
API-UT-05	audit_logging_middleware()	To prove the custom logging system successfully records when users alter the database to maintain accountability, but safely ignores requests that only view data to reduce database costs.	Executed a "Read" request (GET) followed by an "Update" request (PATCH) while monitoring the backend server terminal.	GET operates silently. PATCH outputs [AUDIT] Action 'PATCH' securely logged.	Pass

Table 11: Frontend Analytics Unit Test Results

AL-UT-01	Loading Screen	Prove the dashboard doesn't show a broken, blank screen while waiting for the database to load.	Intentionally delayed the mocked API response to freeze the component.	The screen successfully displays the "Connecting to Local API..." loading spinner.	Pass
AL-UT-02	Total Events Math	Prove the component correctly counts the total number of items in the database.	Injected a fake database array containing exactly 4 incidents.	The "Total Events" UI card updates to display the number "4".	Pass
AL-UT-03	Open Cases Math	Prove the component correctly filters and counts incidents that are not yet resolved.	Injected an array containing exactly 3 "pending" incidents.	The "Open Cases" UI card updates to display the number "3".	Pass
AL-UT-04	Critical Threats Math	Prove the AI Risk Score logic works (only counting open cases with a score of 8, 9, or 10).	Injected items with various scores, including exactly 2 critical ones.	The "Critical Threats" UI card updates to display the number "2".	Pass
AL-UT-05	Resolved Cases Math	Prove the component accurately calculates how many cases have been closed.	Injected an array containing exactly 1 "resolved" incident.	The "Resolved" UI card updates to display the number "1".	Pass
AL-UT-06	Server Crash Handling	Prove the website safely warns the user instead of crashing if the backend goes offline.	Forced the mocked API network to simulate a 500 Internal Server Error.	The footer successfully turns red and displays "Backend Disconnected / OFFLINE".	Pass
AL-UT-07	Default Time Filter	Prove the dashboard applies a sensible default timeframe when the user first logs in.	Rendered the component and checked the state of the Time Range dropdown.	The dropdown defaults to "Last 7 Days" (week).	Pass
AL-UT-08	Custom Date Filter	Prove the interface dynamically changes shape when the user interacts with it.	Simulated a user clicking the dropdown and selecting "Custom Range".	The hidden Start Date and End Date input boxes magically appear on the screen.	Pass
AL-UT-09	Refresh Button	Prove the manual refresh button actually requests fresh data from the server.	Simulated a user clicking the "Refresh Analytics" button.	The system successfully triggers a second network request to the database.	Pass

Table 12: Frontend Incidents Unit Test Results

IT-UT-01	Loading Screen	Prove the page shows a loading spinner while waiting for the database so the user doesn't see a blank screen.	Purposely froze the fake database connection.	The "Loading incidents..." spinner successfully appears on screen.	Pass
IT-UT-02	Table Rendering	Prove the table successfully reads data and builds the rows correctly.	Fed two fake security incidents into the dashboard.	Both incident IDs (#EVT-123 and #EVT-999) successfully show up in the table.	Pass
IT-UT-03	Search Bar Filter	Prove the search box instantly filters out irrelevant incidents.	Typed "EVT-999" into the search bar.	The table instantly updates to hide #EVT-123 and only show #EVT-999.	Pass
IT-UT-04	Empty Search State	Prove the UI gives a clear warning when a search finds absolutely nothing.	Typed random gibberish into the search bar.	The table vanishes and the "No Incidents Found" warning appears.	Pass
IT-UT-05	Investigation Panel	Prove the user can click into an incident to view its specific details.	Simulated a user clicking the "INVESTIGATE" button on a row.	The table disappears and the detailed "Incident Investigation" view opens.	Pass
IT-UT-06	Add Note Function	Prove the app successfully sends newly typed notes to the server to be saved.	Typed a test note and clicked the "Add Note" button.	The app successfully triggers a "Save" request (POST) to the backend.	Pass
IT-UT-07	Resolve Case Button	Prove the app can securely tell the database to close an incident.	Clicked the green "Mark as Resolved" button.	The app successfully triggers an "Update" request (PATCH) to the backend.	Pass
IT-UT-08	Mitigation Playbook	Prove the AI mitigation checkboxes correctly save your progress when clicked.	Opened the playbook and clicked on a specific mitigation step.	The app successfully triggers an "Update" request (PATCH) to the backend.	Pass
IT-UT-09	PDF Generation	Prove the "Generate Report" button triggers the document engine without crashing the webpage.	Clicked the "Generate Report" button.	The PDF engine successfully starts processing the data without breaking the page.	Pass

Table 13: Frontend Login Unit Test Results

LG-UT-01	Default State	Prove the component loads the "Login" interface first, protecting the system by default.	Rendered the component without any user interaction.	The "Welcome Back" text is visible, and the "Full Name" input box is safely hidden.	Pass
LG-UT-02	UI Mode Toggle	Prove the user can seamlessly switch between logging in and creating a new account.	Clicked the "Don't have an account? Sign up" text button.	The form smoothly expands to include the "Full Name" field and updates the button to "Sign Up".	Pass
LG-UT-03	Form Validation	Prove the component physically blocks users from submitting blank data to the database.	Clicked "Log In" while leaving the email and password fields completely empty.	The submission is blocked, Firebase is ignored, and a "fill in all fields" warning appears.	Pass
LG-UT-04	Valid Login API	Prove the component correctly packages user credentials and sends them to Firebase Auth.	Typed fake credentials and clicked "Log In".	The mock Firebase signIn function is successfully triggered with the exact typed credentials.	Pass
LG-UT-05	Valid Sign Up API	Prove the component successfully creates an account and a new database profile simultaneously.	Switched to Sign Up mode, filled out all details, and submitted the form.	The mock Firebase createUser and Firestore setDoc functions are both successfully triggered.	Pass
LG-UT-06	Firebase Error UI	Prove the interface catches raw database errors and translates them into readable warnings.	Forced the mock Firebase server to reject the login with a raw Firebase: auth/user-not-found crash code.	The UI catches the crash, strips out the ugly "Firebase:" text, and displays the clean error.	Pass

Table 14: Frontend Settings Unit Test Results

ST-UI-01	Session Render	Prove the dashboard correctly identifies and displays the currently logged-in user.	Rendered the component using a fake authenticated user profile.	The UI mounts and accurately displays the user's email address (test@company.com).	Pass
ST-UI-02	Fetch Preferences	Prove the component automatically pulls the user's saved settings from the database upon loading.	Rendered the component with the mocked database returning a saved soc_analyst preference.	The component successfully triggers a "Read" (getDoc) request to the database on mount.	Pass
ST-UI-03	Save Preferences	Prove the application can securely package and overwrite user settings in the database.	Clicked two new preference cards and hit "Save Preferences".	The app safely triggers an "Update" (setDoc) request using a { merge: true } safeguard.	Pass
ST-UI-04	Secure Sign Out	Prove the logout button securely terminates the active session and flushes the application state.	Simulated a click on the "Secure Sign Out" button.	The app triggers the Firebase signOut command and successfully forces a browser window reload.	Pass
ST-UI-05	Deletion Safeguard	Prove the system protects users from accidentally deleting their entire account.	Clicked "Delete Account" but simulated the user clicking "Cancel" on the browser warning prompt.	The deletion functions (deleteDoc and deleteUser) are safely aborted and never run.	Pass
ST-UI-06	Complete Deletion	Prove the system successfully wipes both custom data and authentication profiles when explicitly commanded.	Clicked "Delete Account" and simulated the user clicking "OK" on the warning prompt.	The app successfully deletes the database record, deletes the Auth profile, and forces a logout.	Pass

8.1.2 Integration Testing

While component testing isolated individual features, integration testing verifies that S.I.R.A.'s independent modules communicate securely. These manual tests validate the complete end-to-end data lifecycle: from AI generation via the Gemini API, through the Python API's encryption and decryption middleware, to final rendering on the React frontend.

Table 15: Integration Test Results

Test ID	Integration Link	Objective	Test Execution	Expected Output	Actual Outcome
INT-01	Data Ingestion & AI (Backend → Gemini API → Firestore)	Prove the backend successfully communicates with the Gemini API to generate insights and stores the initial payload in the database.	Trigger a mock security event in the backend processor.	The backend receives the Gemini AI summary and writes the encrypted event data to Firestore.	Fail - 14 events have been sent to the LLM and to firestore, but there are 15 events with 1 not having the AI summary. Iterative Pass: The log progress file was tracking lines from previous deleted data, meaning that some of this new data was skipped over.
INT-02	Secure Retrieval (Firestore → Python API [Decrypt] → Frontend)	Prove the Python API successfully fetches encrypted data, decrypts it in memory, and serves readable JSON to the React UI.	Load the Analytics.tsx or Incidents.tsx dashboard in the browser.	The Python API pulls the ciphertext, decrypts it, and the frontend successfully renders the plaintext data on screen.	Pass
INT-03	Secure Mutation (Frontend → Python API [Encrypt] → Firestore)	Prove that user actions (e.g., adding notes) are securely transmitted, encrypted by the API, and safely stored.	Type a new investigation note in the UI and click "Add Note".	The Python API receives the plaintext note, encrypts it, and writes the new ciphertext permanently into Firestore.	Pass
INT-04	Authentication Flow (Frontend ↔ Firebase Auth)	Prove the React client can successfully negotiate authentication tokens to protect the UI.	Submit valid credentials via the live Login.tsx interface.	Firebase validates the credentials and grants access to the protected dashboards.	Pass. However, the firebase api key was found to be exposed from this test.

During the INT-01 test, the data ingestion test temporarily failed because the Python log forwarder's tracking file remembered read-positions from older, deleted logs. Clearing this hidden progress file forced a fresh read and instantly resolved the issue. Furthermore, network inspection for INT-04 revealed a critical security vulnerability: the Firebase API key was inadvertently exposed in the headers of the authentication POST request. [See Appendix G for the Actual Test Examples]

8.1.3 Performance / Stress Testing

To ensure S.I.R.A. remains resilient in a live enterprise environment, a rigorous suite of performance and stress tests was conducted. This phase evaluated the system's capacity to handle high-volume data ingestion, simultaneous API requests, and complex frontend rendering without performance degradation. By simulating real-world traffic bursts and prolonged usage, these tests validate the application's scalability, ensuring that both the React frontend and Python backend remain stable and responsive under extreme operational loads.

Table 16: Performance Test Results

PERF-01	UI Render Speed (Large Dataset)	Prove the React frontend does not freeze when rendering a massive volume of security logs.	Loaded the Incidents dashboard with a populated database of 1,000+ mock security events.	The DOM mounts and renders the table in under 2 seconds without crashing the browser tab.	Pass
PERF-02	Search Algorithm Latency	Prove the use/memo search filter is optimized for real-time keystrokes.	Rapidly typed complex queries into the search bar while the table contained 1,000+ rows.	The table re-renders instantly (sub-100ms latency) without noticeable input lag or stuttering.	Pass
PERF-03	PDF Generation Engine	Prove the jsPDF library can compile a multi-page report without causing a memory overflow.	Clicked "Generate Report" while 500+ incidents were actively filtered on the screen.	The browser successfully compiles and downloads the PDF in under 3 seconds without a "Page Unresponsive" warning.	Pass
PERF-04	Backend File Parsing (Stress)	Prove the Python backend can efficiently parse extremely large log files at once.	Uploaded a 290,000-char Snort file to the backend processor.	The Python script successfully parses the file, applies regex, queues the alerts and gets an LLM output for each.	FAIL (Initial): The massive payload caused the LLM to cut off mid-response, crashing the JSON parser with an 'Unterminated String' error. PASS (Mitigated): Implemented strict pagination (max 50 logs per request). The system now safely chunks data, preventing all token overflows.
PERF-05	FastAPI Concurrency	Prove the backend server can handle multiple simultaneous requests without bottlenecks.	Fired 50 simultaneous GET requests to the /api/incidents endpoint using a testing script.	FastAPI handles the requests asynchronously, returning 200 OK for all calls with zero dropped connections.	Pass
PERF-06	LLM Rate Limit Recovery	Prove the system can process thousands of logs without triggering the Gemini API's Requests-Per-Minute (RPM) block (429 Too Many Requests).	Processed a 1,000-line log file, sending consecutive 50-log batches to the LLM as fast as the Python script could run.	The backend successfully forwards all batches without the external API rejecting the connection.	FAIL (Initial): The backend outpaced the free-tier RPM limit, causing the API to block requests and drop AI insights. PASS (Mitigated): Implemented an intentional 60-second execution delay (time.sleep) between batch calls. The system now throttles itself, guaranteeing 100% data ingestion with zero rejected API calls.
PERF-07	Firestore Read Efficiency	Prove the application retrieves data efficiently without downloading the entire database unnecessarily.	Navigated between the Analytics and Incidents pages multiple times.	Network inspection shows optimized payload sizes, retrieving data in under 500ms per page load.	Pass
PERF-09	Frontend Memory Leaks	Prove the dashboard cleans up its memory correctly during prolonged use.	Left the Analytics dashboard open for 30+ minutes, refreshing the data manually 20 times.	Browser memory profiling shows stable RAM usage (no exponential memory ballooning) and stable CPU utilization.	Pass
PERF-10	State Persistence	Prove the log forwarder accurately remembers its reading position if the system crashes or is forcefully terminated.	Start processing a 1,000-line log file. Forcefully terminate the script (Ctrl+C) at 50% completion, then restart it.	The script resumes from line 501, ensuring exactly 1,000 total logs are written to the database with zero duplicates.	FAIL: Upon forceful termination, the script loses its read-state for unprocessed logs, restarting them from line 1 and causing redundant API calls and database duplication. However, logs successfully processed by the LLM prior to the crash are safely isolated as tracked JSON files in a 'cleaned' directory, successfully preventing their duplication.

During stress testing, three key limitations were identified. First, processing massive data batches caused the LLM to exceed its token output limits, resulting in unparseable JSON crashes; this was successfully mitigated by implementing strict pagination (capped at 50 log lines per request) to prevent token overflow. Second, the backend ingestion rate outpaced the API's free-tier Requests-Per-Minute (RPM) limits, which was resolved by engineering a 60-second execution delay between batch calls, guaranteeing 100% data retention with zero rejected requests. Finally, testing revealed a partial state persistence issue upon forceful system termination: while logs processed prior to a crash are safely isolated in a 'cleaned' directory to prevent duplication, unprocessed logs lose their read-state and restart from line one, highlighting an area for future database deduplication enhancements. (See Appendix H for the Actual Test Examples)

8.2 User Acceptance Testing (UAT)

To rigorously evaluate the real-world usability and effectiveness of the S.I.R.A. dashboard, User Acceptance Testing (UAT) was conducted. The methodology involved a cohort of seven participants, specifically selected to represent the system's diverse target personas. This group included a mix of individuals with varying levels of technical expertise, ranging from non-technical individuals simulating the 'Business Owner' persona to computer science students acting as proxy 'IT Administrators'.

The evaluation followed a structured, task-based approach. Participants were provided with access to the S.I.R.A. dashboard, populated with pre-ingested threat data, and asked to independently complete core operational workflows (e.g., authenticating, navigating the analytics dashboard, and triaging a critical AI-generated incident).

To provide strong academic support for the approach taken, the evaluation quantified user feedback using a structured post-test questionnaire based on a 5-point Likert scale (1 = Poor/Strongly Disagree, 5 = Excellent/Strongly Agree). (See Appendix I for the blank questionnaire and full results).

Overall, the UAT results strongly validate the core hypothesis of the S.I.R.A. project: that an AI-driven dashboard can effectively simplify cybersecurity incident response for SMEs. Across all seven participants the quantitative scores consistently averaged between 4 and 5 out of 5 across most functional areas.

Key Strengths

- **UI/UX and Accessibility:** The system achieved unanimous maximum scores (5/5) for account creation, authentication, and top-level analytics navigation (e.g., Total Events, Trend Charts, and Pie Charts). Multiple participants explicitly praised the modern, intuitive layout, noting its superiority over traditionally complex SIEM interfaces.
- **The Core AI Engine:** The integration of the LLM was highly successful in filtering out technical noise. Notably, one participant identified an emergent use-case: the system serves as an effective educational tool for junior SOC analysts, enabling them to learn from the AI-generated mitigation steps rather than blindly escalating tickets.
- **Incident Management Workflow:** Core operational features, such as assigning team members, adding investigation notes, resolving incidents, and adjusting configurations, received unanimous 5/5 ratings, demonstrating that the system is robust and frictionless for daily operational use.

Areas for Improvement and Limitations

- **AI Persona Prompting:** While generally effective, the "Business Owner" persona requires further prompt tuning. One non-technical user rated the jargon reduction at 3/5, observing that some summaries were too brief and occasionally recommended "escalating to an IT team"—a directive that contradicts the operational reality of micro-SMEs without dedicated IT staff.
- **Data Redaction Context:** A critical limitation was identified regarding the PII sanitisation pipeline. While stripping IP and MAC addresses is essential for privacy prior to LLM processing, displaying this redacted data separately from the main technical alert created a fragmented experience for technical users. Furthermore, inconsistencies were noted where certain redacted elements (such as usernames) were omitted from the final UI display.
- **UI/UX Refinements:** Minor usability improvements were identified. The notification bell currently requires users to dismiss active alerts to view older ones; implementing a scrollable history was suggested as a scalable solution. Additionally, the formatting of the generated PDF incident reports received slightly lower scores (averaging 3.5/5), indicating a need for a more professional styling template. Finally, the absence of a "Forgot Password" workflow was flagged as a missing standard feature.

Primary Feature Requests

The most consistent feedback across the testing cohort was the desire for advanced, granular analytics. Specific recommendations included:

- Implementing keyword-based threat tracking (e.g., visually isolating SQL injection attempts versus NMAP scans).
- A highly innovative proposal to feed aggregated, long-term analytics back into the LLM. This would allow the system to generate macro-level security posture reports over weeks or months, moving beyond the analysis of isolated, single incidents.

9. Conclusion & Future Work

9.1 Project Summary

The primary aim of this project was to design, develop, and evaluate an affordable, LLM-powered incident response dashboard to bridge the cybersecurity expertise gap for SMEs. Reflecting upon the initial objectives, this aim has been successfully achieved. The S.I.R.A. platform demonstrates a highly efficient, end-to-end data pipeline capable of translating raw network telemetry into accessible, actionable intelligence.

The core technical objectives were met through the development of a stateful Python backend that actively monitors and parses IDS logs. To guarantee data privacy, a rigorous local pre-processing pipeline was implemented, successfully stripping Personally Identifiable Information (PII) and pseudonymising network IPs via regex before cloud interaction.

Crucially, the integration of the Gemini LLM proved highly effective. By engineering an asynchronous batching system and a FastAPI RAM cache, the architecture successfully circumvented cloud rate limits and protected free-tier database quotas. Furthermore, as validated by User Acceptance Testing (UAT), iteratively refining the dynamic AI personas successfully eliminated technical jargon, allowing the system to tailor mitigation playbooks specifically for non-technical business owners. Ultimately, the real-time React dashboard, coupled with AES payload encryption, successfully delivered a secure, intuitive interface that fundamentally reduces the cognitive load associated with traditional SIEM management.

9.2 Critical Evaluation & Limitations

While S.I.R.A. successfully functions as a prototype, testing revealed several key operational and security limitations.

From a security standpoint (INT-04), the Firebase VITE_API_KEY is inadvertently exposed within the frontend headers. Additionally, an attempt to let users securely input their own Gemini API keys was abandoned; the backend architecture could only apply the key globally across the application, rather than securely isolating it per user.

Operationally, the Python ingestion script requires manual execution. It does not yet run as a background service, meaning if the terminal is closed, log ingestion stops entirely. Furthermore, stress testing showed that if the system crashes, unprocessed logs lose their read-state and restart from line one, creating a risk of duplicate data ingestion.

Finally, User Acceptance Testing (UAT) highlighted issues with the AI context and UI. The "Business Owner" persona occasionally provided unhelpful advice by suggesting users "escalate to an IT team"—ignoring the reality of micro-SMEs. Additionally, while the PII sanitisation pipeline is essential for privacy, it fragmented the dashboard for technical users by separating redacted data from the main alerts, occasionally omitting elements like usernames entirely.

9.3 Future Enhancements

To evolve S.I.R.A. from an academic prototype into a highly resilient, enterprise-ready application, several key enhancements have been identified for future iterations:

- **Service Automation:** Convert the Python ingestion script into a dedicated Windows/Linux background service to ensure continuous, autonomous monitoring without manual user execution.
- **Frontend API Security:** Secure the exposed Firebase VITE_API_KEY by routing authentication requests through a secure backend proxy, and implement Firebase JWT Authentication for true zero-trust security.
- **Database Multi-Tenancy:** Refactor the database schema and user account logic to support multi-tenancy. Currently, the database provides a global view; this must be isolated so individual organisations or users only see their specific incident data.
- **Custom API Key Integration:** Develop the frontend settings UI to allow users to securely input and store their own personal LLM API keys, transferring the usage costs and quota management directly to the user.
- **System Resilience:** Enhance the Python script's state management to ensure that if the script is interrupted, any logs that were parsed but not yet processed by the LLM are automatically re-uploaded upon restart.
- **Active Mitigation Integration:** Expand the system's scope by adding automated API integrations with local firewalls, allowing S.I.R.A. to dynamically block malicious IPs based on the AI's recommendations.
- **Advanced Analytics:** Implement broader, macro-level analytics that track long-term threat trends, and feed this aggregated data back into the LLM to generate weekly or monthly security posture reports.
- **Enhanced PII Redaction:** Expand the local regex sanitisation dictionaries to capture a wider array of sensitive data points, ensuring maximum compliance with UK GDPR standards.
- **Alternative Infrastructure:** Explore migrating the backend logic to AWS or supporting free-tier, localised LLMs to prevent cloud API credit exhaustion during periods of high network traffic.
- **UI Quality of Life Features:** Implement standard application features requested during UAT, like a secure "Forgot Password" workflow for account recovery.

9.4 Final Thoughts

Small and medium-sized enterprises remain the backbone of the global economy, yet they are systematically under-resourced and disproportionately targeted in the modern cyber threat landscape. Traditional enterprise security tools are often financially prohibitive and operationally overwhelming, leaving small business owners to rely on a dangerous false sense of security. S.I.R.A. fundamentally bridges this gap. By leveraging the analytical power of Large Language Models within a highly secure, privacy-first pipeline, the system removes the overwhelming technical noise of standard detection engines. It translates complex, daunting network telemetry into clear, educational, and actionable steps. Ultimately, S.I.R.A. empowers non-technical users to take

confident ownership of their digital environments, proving that proactive cybersecurity can be both highly accessible and cost-effective.

References

- Fadziso, T. et al. (2023) *Evolution of the cyber security threat: An overview of the scale of Cyber Threat*, researchgate. Available at: https://www.researchgate.net/publication/374156044_Evolution_of_the_Cyber_Security_Threat_A_n_Overview_of_the_Scale_of_Cyber_Threat (Accessed: 23 March 2026).
- Hiscox Group (2025) *Cyber-attacks leave smes with hefty fines and uncertain futures, Cyber-attacks leave SMEs with hefty fines and uncertain futures | Hiscox Group*. Available at: <https://www.hiscoxgroup.com/news/press-releases/2025/29-09-25> (Accessed: 23 March 2026).
- Saleem, J. et al. (2017) *A state of the art survey - impact of cyber attacks on SME's | proceedings of the International Conference on Future Networks and Distributed Systems*, ACM Digital Library. Available at: <https://dl.acm.org/doi/abs/10.1145/3102304.3109812> (Accessed: 23 March 2026).
- Tetteh, A.K. (2024) *Issues in information systems volume 25, issue 1, pp. 235-246, 2024 235*, International Association for Computer Information Systems. Available at: https://iacis.org/iis/2024/1_iis_2024_235-246.pdf (Accessed: 23 March 2026).
- Lake, K. (2022) *Top SME cybersecurity challenges and how to overcome them*, JumpCloud. Available at: <https://jumpcloud.com/blog/overcome-top-sme-cybersecurity-challenges> (Accessed: 23 March 2026).
- Manzoor, J. et al. (2024) *Cybersecurity on a budget: Evaluating security and performance of open-source Siem solutions for smes*, PLOS ONE. Available at: <https://journals.plos.org/plosone/article?id=10.1371%2Fjournal.pone.0301183> (Accessed: 23 March 2026).
- Femi, A. et al. (2025) *Designing Resilient AI-Based Cybersecurity Frameworks for Small and Medium Enterprises (SMEs)*, British Journal of Cyber Criminology. Available at: <https://www.pubtexto.com/journals/british-journal-of-cyber-criminology/fulltext/designing-resilient-ai-based-cybersecurity-frameworks-for-small-and-medium-enterprises-smes> (Accessed: 23 March 2026).
- Junior, C.R., Becker, I. and Johnson, S. (2025) *Unaware, unfunded and uneducated: A systematic review of SME Cybersecurity*, arXiv.org. Available at: <https://arxiv.org/abs/2309.17186> (Accessed: 23 March 2026).
- Troublefield, T. C. (2025) *The Cyberpsychology of Small and Medium-Sized Enterprises Cybersecurity: A Human-Centric Approach to Policy Development*. *Journal of Information Security*, **16**, 158-183. Available at: [10.4236/jis.2025.161009](https://doi.org/10.4236/jis.2025.161009). (Accessed: 23 March 2026).
- Chidukwani, A., Zander, S. and Koutsakis, P. (2022) *A survey on the cyber security of small-to-medium businesses: Challenges, research focus and recommendations | IEEE Journals & Magazine | IEEE Xplore, IEEE.org*. Available at: <https://ieeexplore.ieee.org/abstract/document/9853515/> (Accessed: 23 March 2026).

BCS, The Chartered Institute for IT (2022) *Code of Conduct for BCS Members*. Available at: <https://www.bcs.org/membership-and-registrations/become-a-member/bcs-code-of-conduct/> (Accessed: 24 March 2026).

gov.uk (2011) The UK's data protection legislation. Available at: <https://www.gov.uk/data-protection> (Accessed: 24 March 2026).

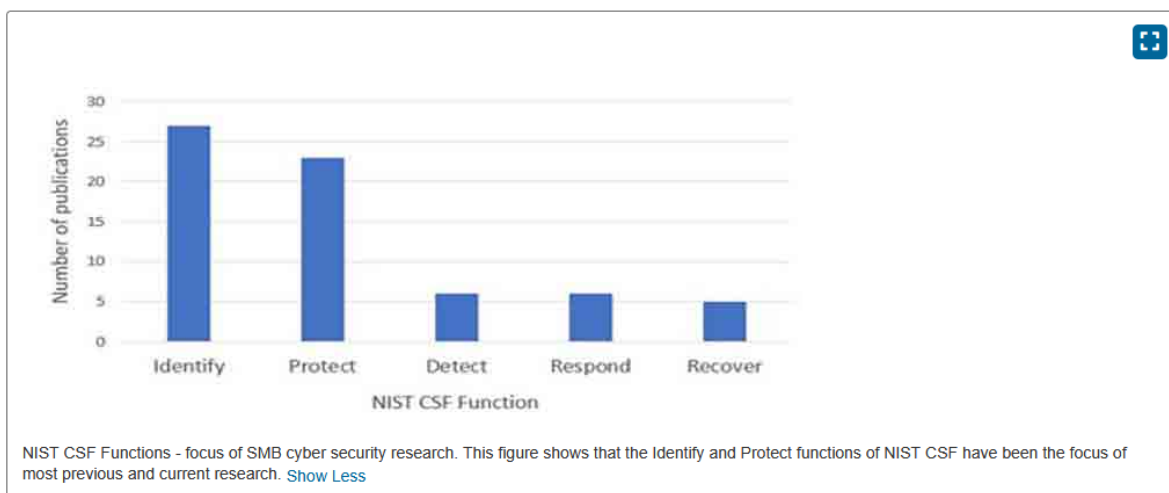
Legislation.gov.uk (1990) *Computer Misuse Act 1990*. Available at: <https://www.legislation.gov.uk/ukpga/1990/18/contents> (Accessed: 24 March 2026).

gov.uk (2025) Cyber security breaches survey 2025. Available at: <https://www.gov.uk/government/statistics/cyber-security-breaches-survey-2025/cyber-security-breaches-survey-2025> (Accessed: 24 March 2026).

Business population estimates for the UK and regions 2025: Statistical release (2025) GOV.UK. Available at: <https://www.gov.uk/government/statistics/business-population-estimates-2025/business-population-estimates-for-the-uk-and-regions-2025-statistical-release> (Accessed: 29 April 2026).

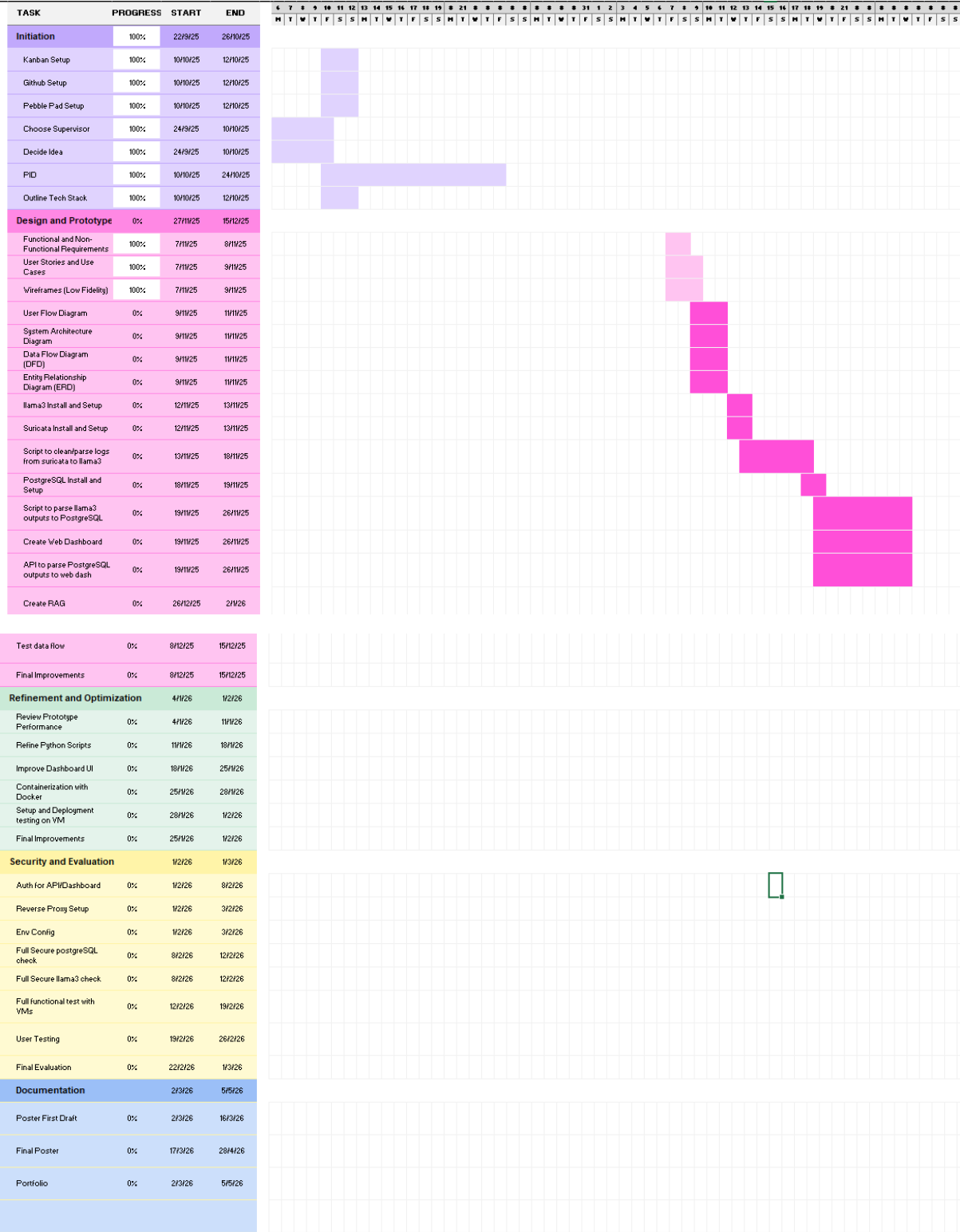
Appendices

Appendix A: Literature Evidence



[Figure 1: A graph from a recent study showing the number of publications per each phase of the NIST CSF.]

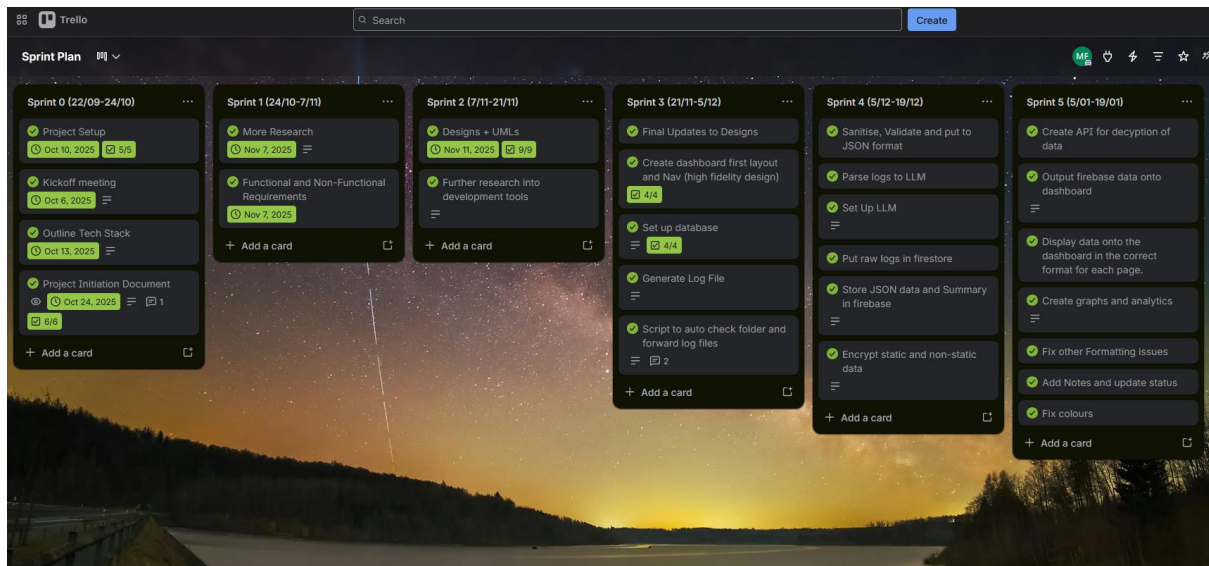
Appendix B: Project Management



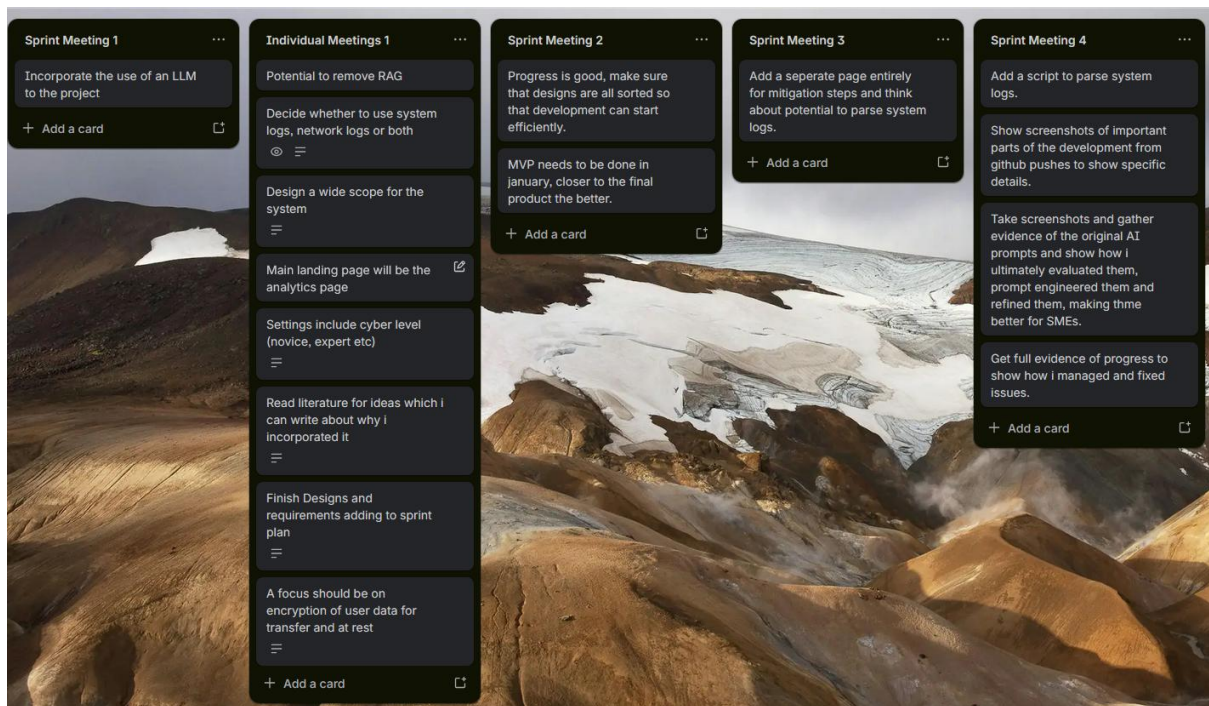
[Figures 1 & 2: Screenshots of the initial Project Gannt Chart.]

Final goes here

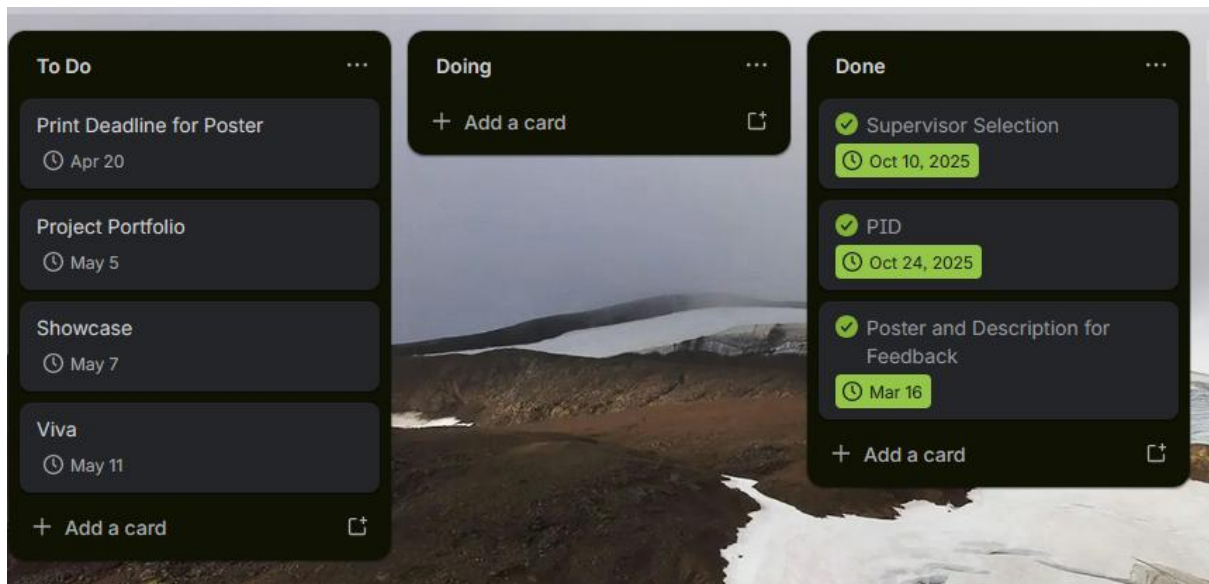
[Figure 3: Screenshot of the final Project Gannt Chart.]



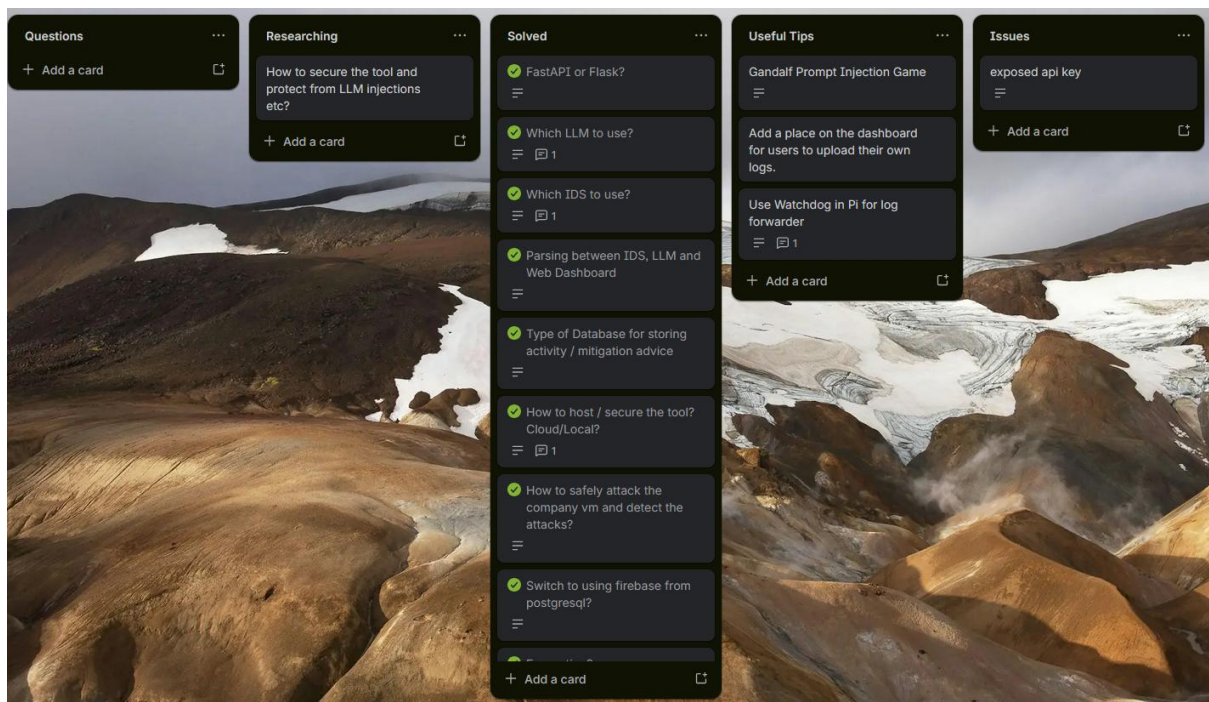
[Figure 4: A Screenshot of the Trello Sprint Plan board weeks 1 – 5.] note: add the final weeks too



[Figure 5: A Screenshot of the Trello Meeting notes board weeks 1-4.]



[Figure 6: A Screenshot of the Trello Deadlines board.]



[Figure 7: A Screenshot of the Trello Queries & Questions Board.]

Appendix C: Initial Project Risk Assessment

System and Scope					
Overview	Scope	Inputs	Outputs	Ownership	Supervisor
LLM-Powered Incident response Assistant for SMEs: Ingests network logs from IDS, passes it to LLM for summarisation, Outputs into a DB, Displays on a Web Dashboard.	Local Deployment on VM for testing., evaluates how LLMs can assist SMEs during incident response.	JSON logs	LLM-generated dashboard data.	Matthew Fish	Nathan Clarke
Data Flow					
Source	Type	Flow	PII Risk	Notes	
Network logs	Structured JSON	JSON Network logs to LLM	Low-Medium	IP addresses may indirectly identify individuals.	
LLM	JSON input/ Text output	Receives JSON, Returns summarised data	Low	Model runs locally so no external data transfer.	
Database	Structured	Stores processes data	Medium	Must ensure encryption and restricted access.	
Web dashboard	Structured	Displays processed data	Low	User authentication needed to prevent exposure.	
Test VMs	Unstructured	Simulated attacks	None	Used for testing only	

[Figure 1: A Screenshot of the System Scope and Data Flow PII Risk]

Potential Risk Categories		Likelihood and Impact						Rules	
Category	Description		Risk	Likelihood	Impact	Overall Risk	Rationale	1 to 10	Low, Medium, High, Significant
Bias & Fairness	LLM may dramatise risk scores.		Model bias in classification	5	7	High	Depends on the data the LLM has been trained with. Will use a RAG to reduce bias as much as possible.		
Privacy Violations	Logs could contain identifiable IPs or metadata.		Privacy leak via IP data	4	8	High	Rare but could have a serious impact. Will try to anonymise IPs where possible.		
Explainability	LLM output may be unclear or untraceable to specific log evidence.		Model hallucination / misinterpretation	5	7	High	Will implement a way to check the processed data.		
Security	Exposure of database, API endpoints, or LLM prompt injection, User tampering.		Database compromise	2	10	Medium	Huge affect on the company as an attacker could do a ransomware attack or use the logs for other		
Regulatory / Compliance	Data handling must align with GDPR.		LLM prompt injection or misuse	2	7	Medium	Stored locally so very unlikely and an implemented check on the data would help with security and		
Operational / Reliability	LLM, IDS, API or Dashboard downtime may stop attacks from being prevented.		Unauthorised access to dashboard, IIm or IDS	2	10	Medium/High	Will all be deployed locally and auth access will be required for dashboard. Access controls would be implemented for only admin to		
Data Integrity	Corrupted or manipulated logs could cause false insights.		Data loss / corruption	2	5	Medium	Could use backups of data but requires further storage and security.		
Supply Chain	Third party risks (docker images, pre-trained LLM)		System downtime	5	8	High	Downtime for updates, internet loss, power loss affects the whole system.		
People	Lack of user training and lack of incident response plan for if the tool got compromised		Non-compliance with GDPR and computer misuse for users	2	10	Medium	Implement secure handling of data, access controls and systems in place to prevent tool being misused.		
			Supply chain risks	4	6	High	Dependencies on third-party vendors or libraries could fail or be compromised, affecting system stability and security.		
			Insufficient Logging	3	6	Medium	Limited logging makes it harder to detect, investigate, or respond to issues		
			User Error or misconfig	4	5	Medium	Misconfigurations or mistakes by users can cause outages or security issues		
			Ethical Misuse of tools	2	8	Medium	Tools could be used unethically (e.g., privacy violations)		

[Figure 2: A Screenshot of the potential risk categories with their likelihood and impact]

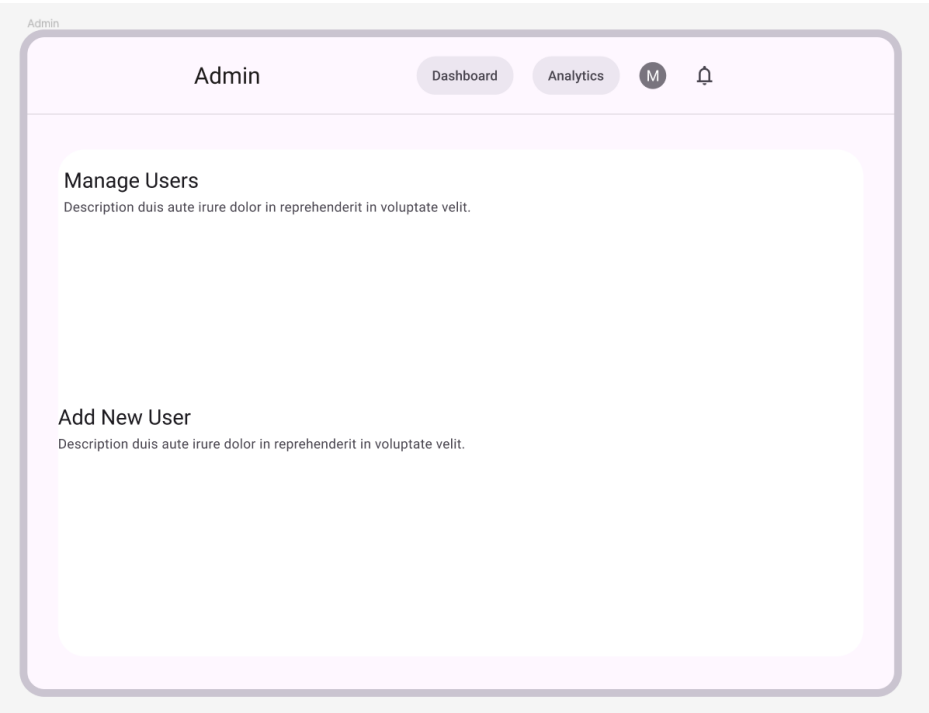
Existing Mitigation Control				
Control	Description	Effectiveness	Enforced?	Works as intended?
Docker container isolation	Each service runs in its own container.	High	To be configured	To be Verified
Everything run locally	No public exposure to data.	High	To be configured	To be Verified
Basic auth on API/dashboard	Restricts access.	Medium	To be configured	To be Tested
Access Control	RBAC for system access. Principle of least privilege.	Medium	To be configured	To be Tested
Pseudonymisation of IPs	Truncate or hash IP addresses before storing.	Medium	To be configured	To be Tested
Database Security	Least-privilege accounts , encryption at rest	Medium	To be configured	To be tested
System updates	Patch Docker images regularly, IDS, LLM, Python.	High	Ongoing	Yes
Input Sanitisation and validation	Validate and sanitise all logs before entering LLM.	Medium	To be configured	To be verified
Logging and monitoring	System healthchecks and logs for user actions.	Medium	To be configured	To be Verified
Backup and Recovery	Regular backups of database and critical files	High	To be configured	To be Verified

[Figure 3: A Screenshot of the pre design and development existing mitigation controls]

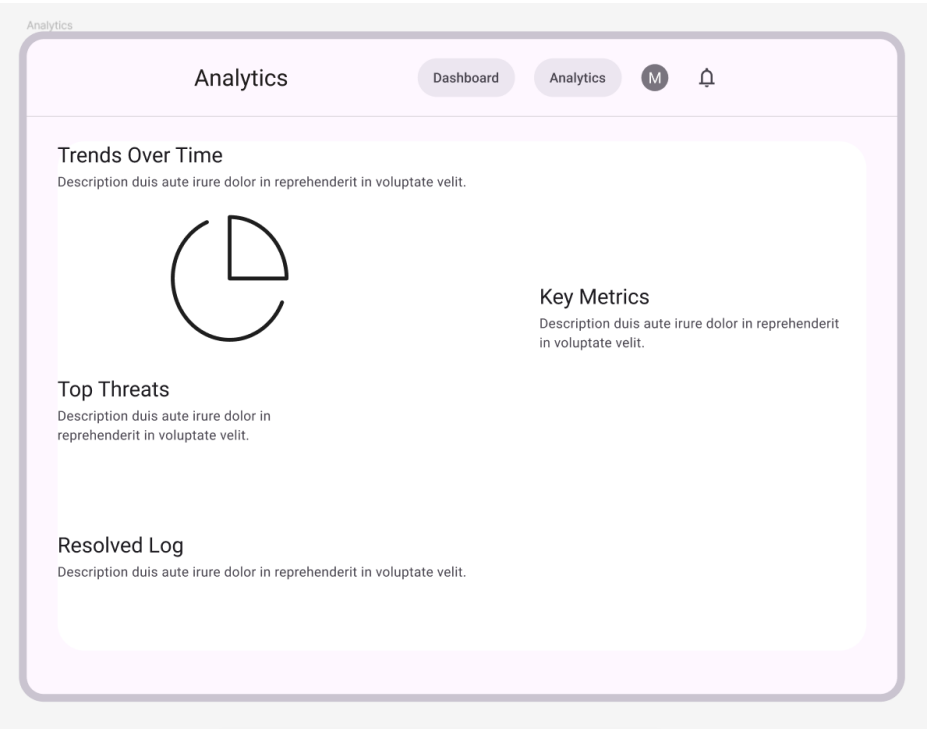
Risk Register							
Risk Description	Likelihood	Impact	Overall	Controls	Owner	Mitigation /	Timeline
LLM produces biased or inconsistent	5	7	High	Will review it manually.	Matthew	Validate LLM outputs vs. known threats. RAG	By Feb
IP data exposes personal info	4	8	High	IP anonymisation, stored	Matthew	Hash Ips	Jan
Unauthorised LLM, IDS, dashboard, db access	2	10	High	Auth / Access Controls, docker isolation	Matthew	Implement JwT/API-key login. Access controls locally.	Jan
Prompt injection or malicious input	2	7	Medium	Sanitise model inputs.	Matthew	Validate all logs before / after LLM processing	Jan
Data loss or corruption	2	5	Medium	Backups	Matthew	Backups	Feb
Downtime or service failure	5	8	High	Monitoring, healthchecks	Matthew	Monitor uptime, add healthchecks, updates scheduled at low	Feb
Non-compliance with GDPR / Computer Misuse	2	10	Medium	Ethical review, documentation	Matthew	Ethical review, Document compliance steps.	Ongoing
Supply chain risks	4	6	High	System updates, vulnerability scanning	Matthew	Trusted docker images, regular scans.	Ongoing
Insufficient Logging	3	6	Medium	Audit Logging	Matthew	Limited logging makes it harder to detect, investigate, or respond to issues. Access to tools are logged to a secure	Ongoing
User Error or misconfig	4	5	Medium	Training and documentation	Matthew	Clear documentation for setup and use. Configuration to avoid manual errors.	Ongoing
Ethical Misuse of tools	2	8	Medium	Access controls, policies	Matthew	Restrict tool access to authorised users. Define and communicate acceptable use	Ongoing

[Figure 4: A Screenshot of the final detailed Risk Register]

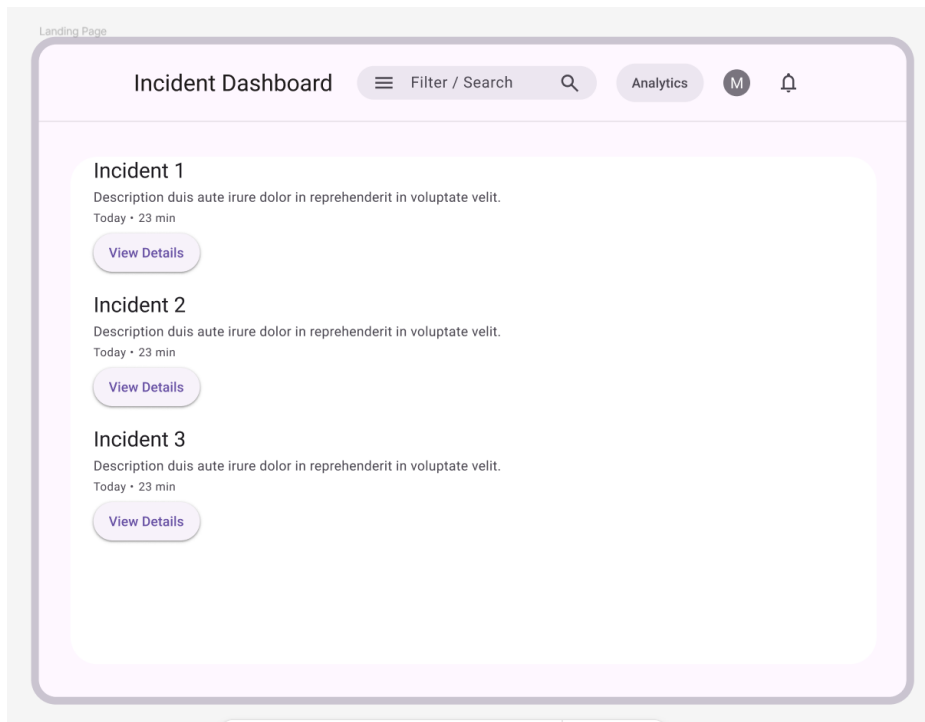
Appendix D: Initial Wireframes



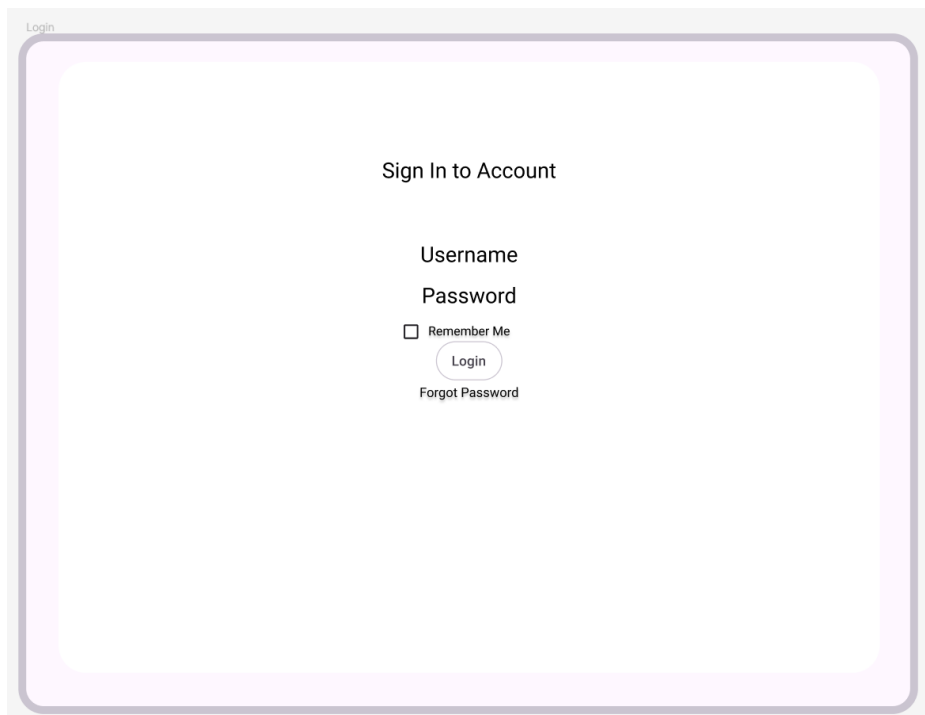
[Figure 1: Admin Page Wireframe]



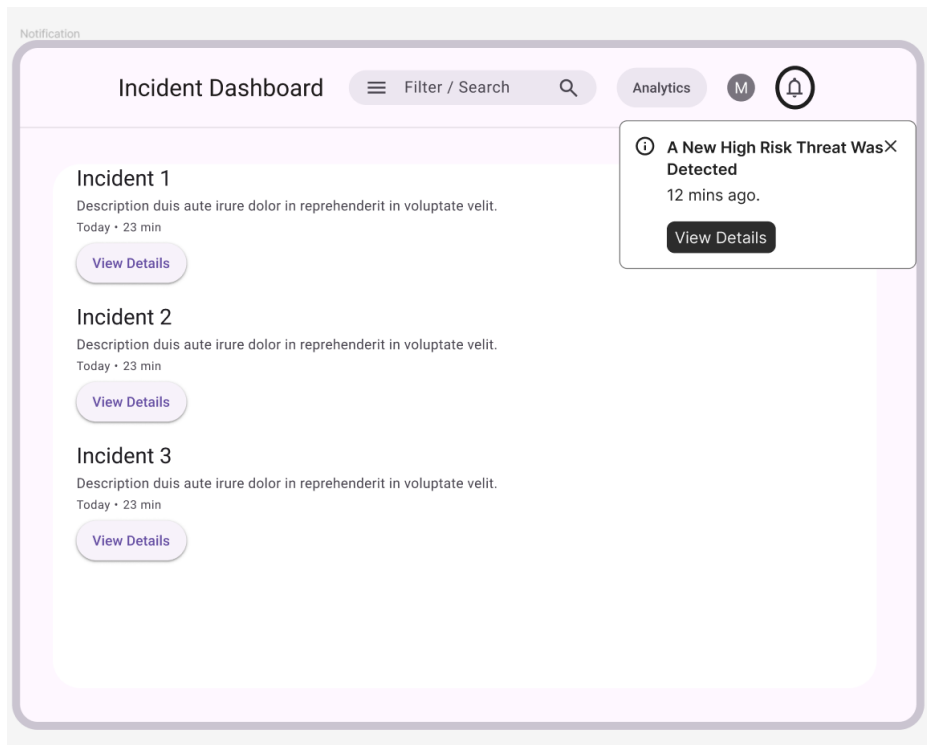
[Figure 2: Analytics Page Wireframe]



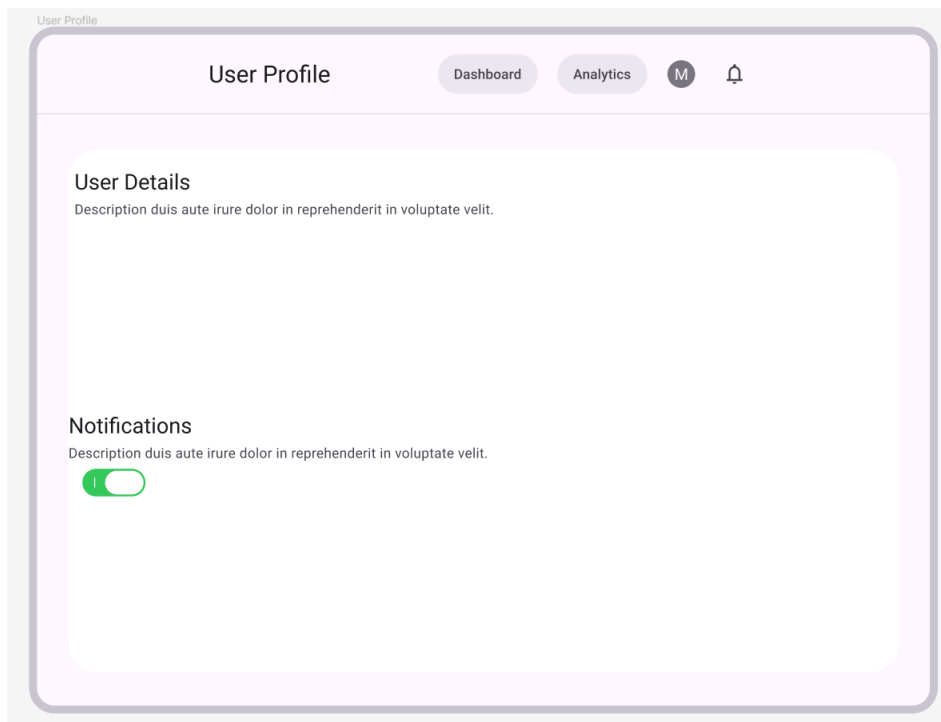
[Figure 3: Incidents Page Wireframe]



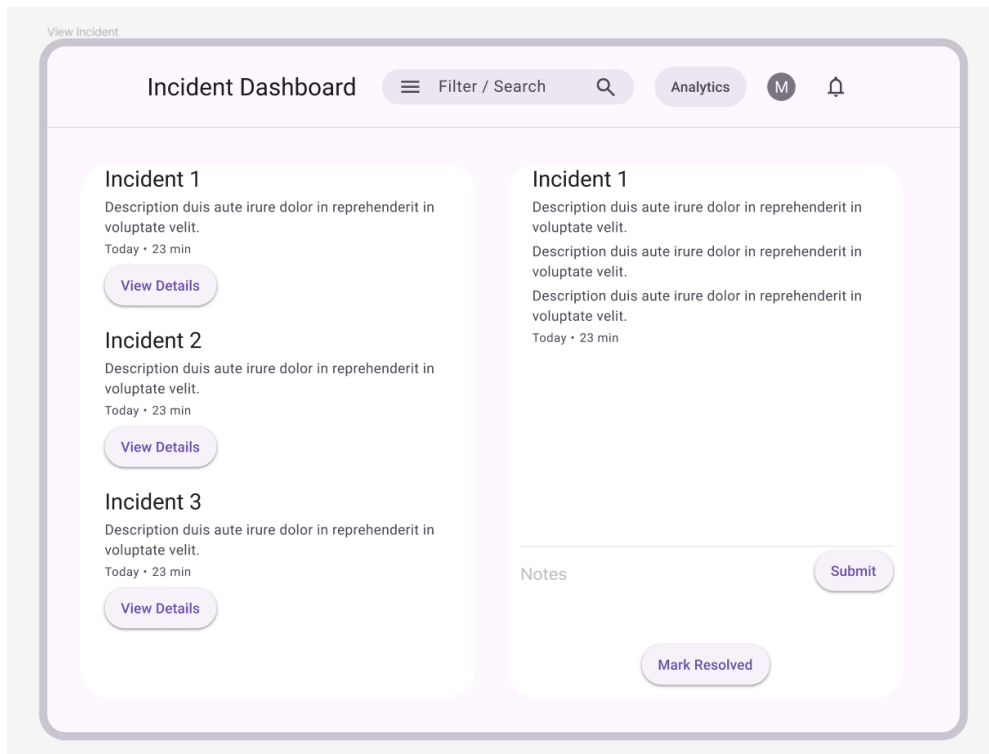
[Figure 4: Login Page Wireframe]



[Figure 5: Notification Popup Wireframe]

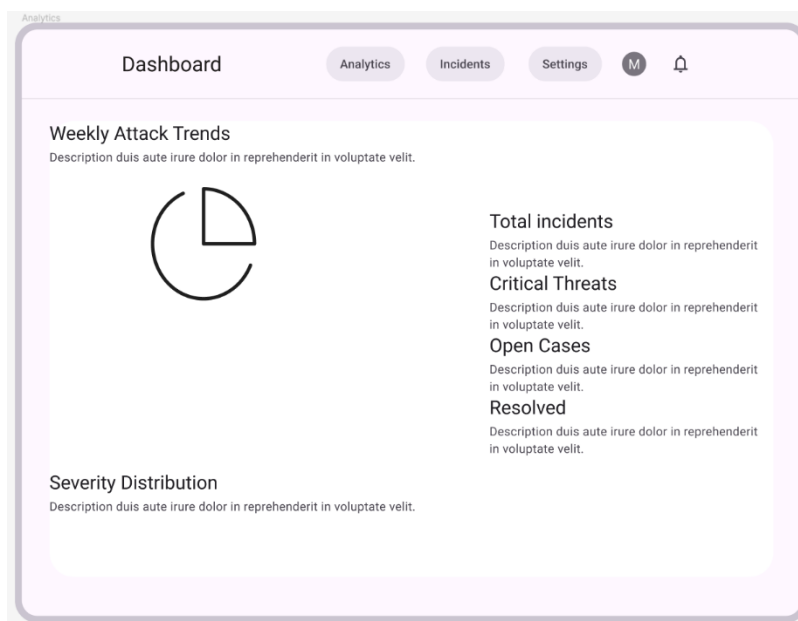


[Figure 6: User Profile Page (Settings) Wireframe]

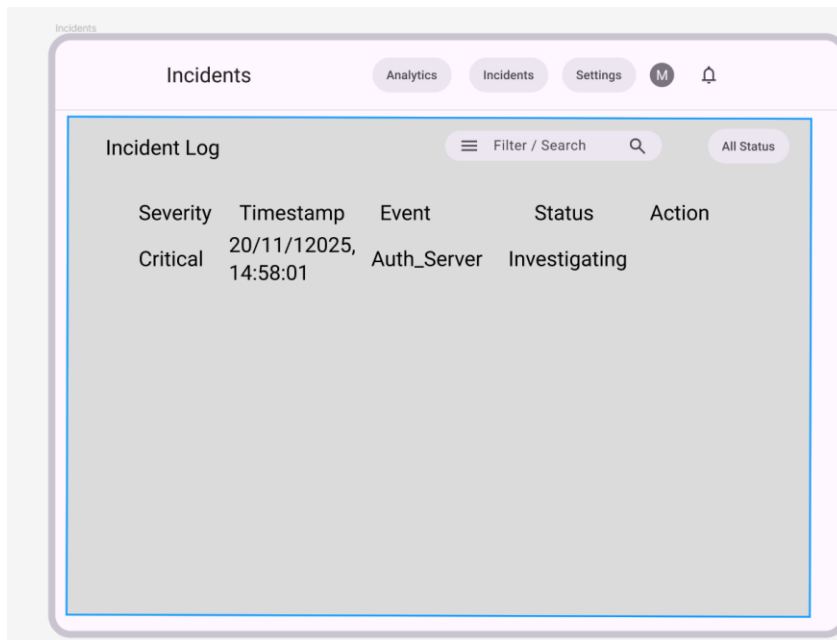


[Figure 7: View Incidents Page Wireframe]

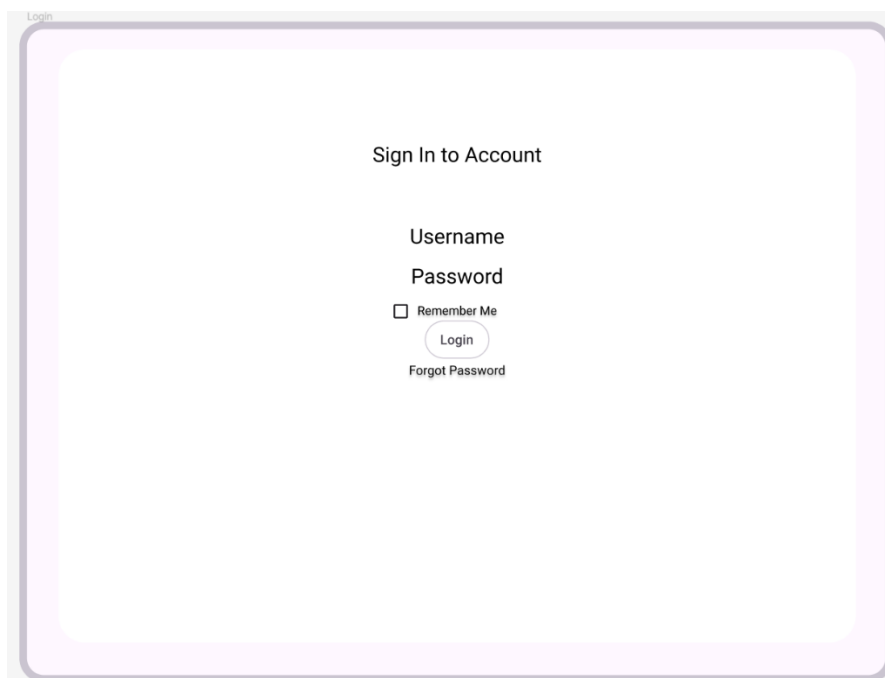
Appendix E: Final Wireframes



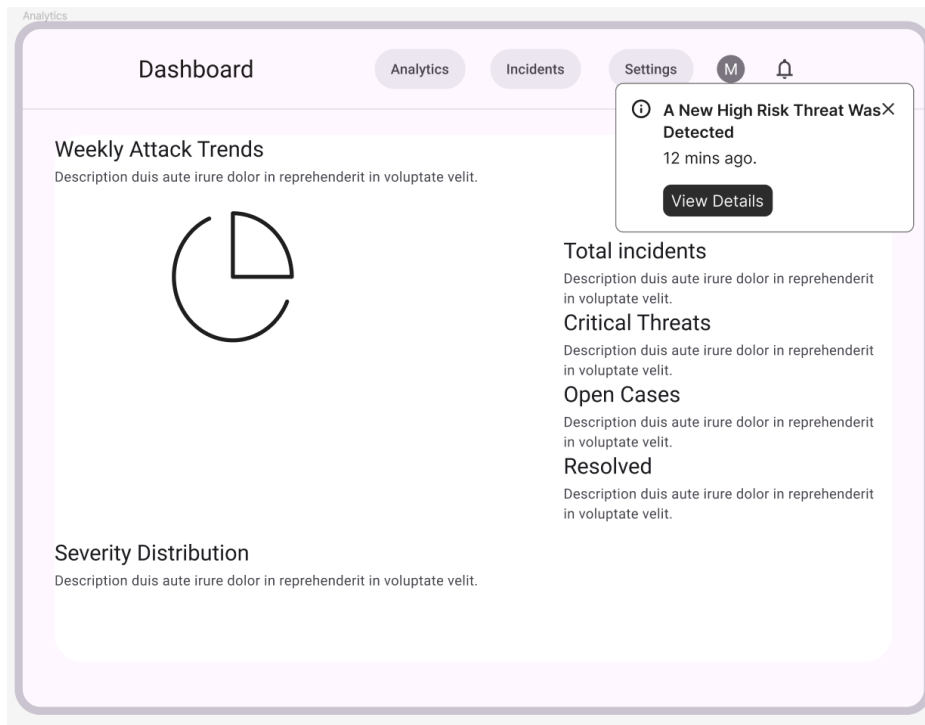
[Figure 1: Analytics Page Wireframe]



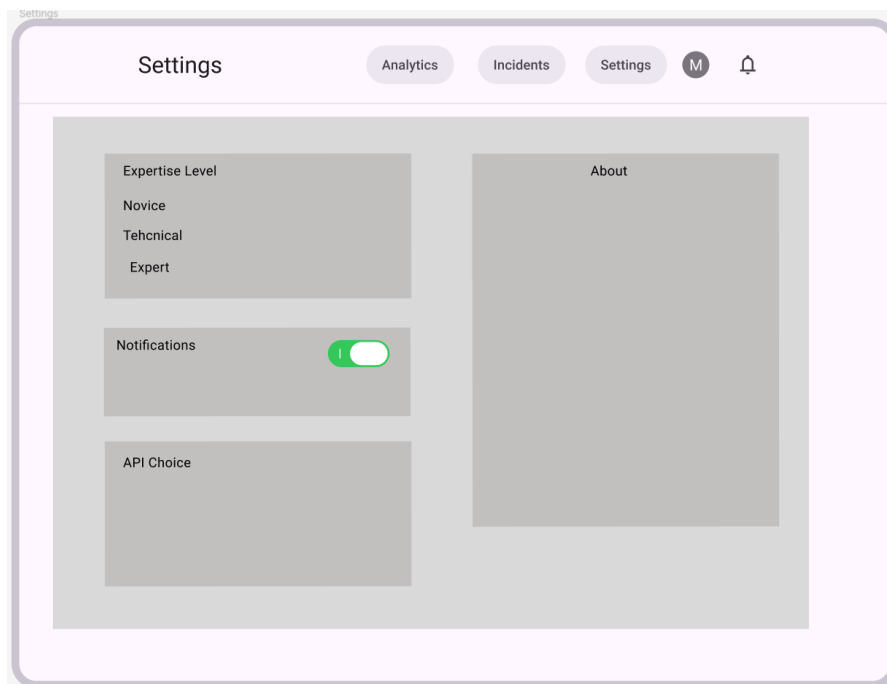
[Figure 2: Incidents Page Wireframe]



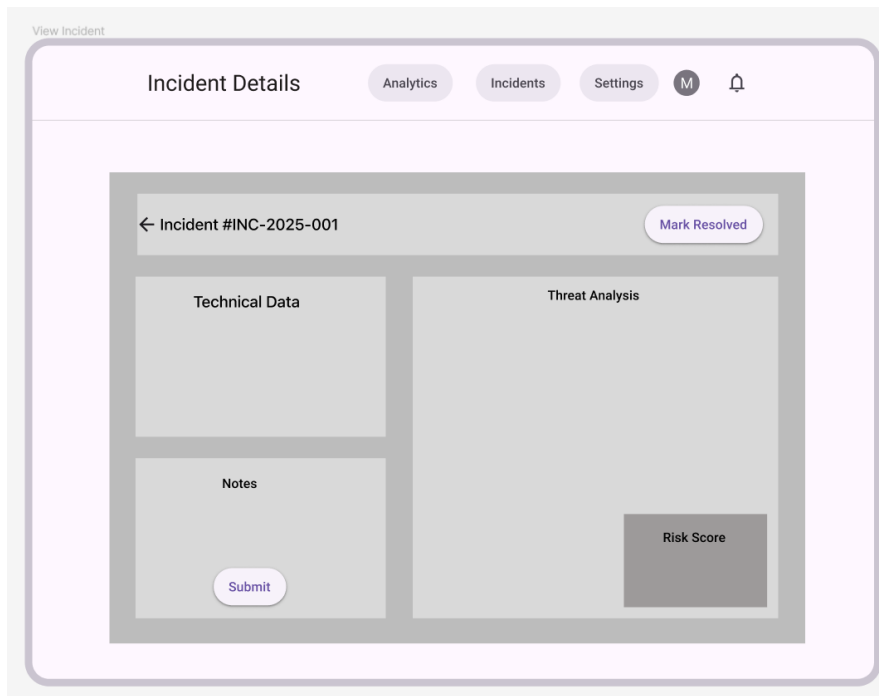
[Figure 3: Login Page Wireframe]



[Figure 4: Notification Popup Wireframe]



[Figure 5: Settings Page Wireframe]



[Figure 6: Incident Details Page Wireframe]

Appendix F: Unit Tests

```

--- STARTING MANUAL UNIT TESTS ---
Original String:
Login failed for matthew.fish@students.plymouth.ac.uk in C:\Users\WattFish\app.exe from 192.168.1.150 (MAC: 00:18:44:11:3A:B7). External Call to 8.8.8.8. password=SuperSecret123!

Sanitized Result:
Login failed for [EMAIL_REDACTED] in C:\Users\[WINDOWS_USER_DIR]\app.exe from [INTERNAL_IP_0] (MAC: [MAC_REDACTED]). External Call to [EXTERNAL_IP_0]. password=[PASSWORD_REDACTED]

--- UT-04: ENCRYPTION & DECRYPTION ---

[3] Payload Ready for Encryption:
('status': 'Processed', 'log_data': 'Login failed for [EMAIL_REDACTED] in C:\Users\[WINDOWS_USER_DIR]\app.exe from [INTERNAL_IP_0] (MAC: [MAC_REDACTED]). External Call to [EXTERNAL_IP_0]. password=[PASSWORD_REDACTED]', 'risk_score': 7)

[4] Ciphertext:
pAAAAABp146S1mYAK4Y6ALNi9pEYhFC8UqA0maHg_APm-isl0j81YgHlvpugd_hQ14HtpGN3bPTf9R03quEDRQeLmWkU8Yp7n103o1ZrZLxov7ZD4NQjOD7moNIB8H6kxz_kokQUE-1qZi0IYPh0s8h89XOLPTc58ICVlaNQ#HjiCcUxbnguACEF60Qq#8Ypgnk8IRpcA3Z6QAlmTk8qd_-057mZfsN2XVtsmbxag30b33B
rFj0b3F6Rmbr2HcfSH1X0dRecn49prfcX2-pc6Ae9W1FNbmBrlzyGcdM#WMTZoyJE5wJDN213-DJzgJEs7cpmDTmt5bbAA_UebMGH95h83r9aJnD8NQ08ErF14kS_VcMEn

[5] Decrypted Payload:
('status': 'Processed', 'log_data': 'Login failed for [EMAIL_REDACTED] in C:\Users\[WINDOWS_USER_DIR]\app.exe from [INTERNAL_IP_0] (MAC: [MAC_REDACTED]). External Call to [EXTERNAL_IP_0]. password=[PASSWORD_REDACTED]', 'risk_score': 7)

[RESULT]: PASS! Sanitized data was successfully encrypted and decrypted without corruption.

--- UT-05: AI PERSONA PROMPT LOGIC ---
[Test A: Requesting 'business_owner' prompt]
Result: Analyze these logs for a non-technical Business Owner (SME). Your goal is to provide absolute clarity without technical jargon.

[Test B: Requesting 'soc_analyst' prompt]
Result: Analyze these logs for a Senior Security Analyst (SOC). Be extremely technical. Focus on attack vectors, payload analysis...

[Test C: Requesting invalid 'hacker' prompt]
Result (Fallback): Analyze these logs for a non-technical Business Owner (SME). Your goal is to provide absolute clarity without technical jargon.

[RESULT]: PASS! The system dynamically loads the correct LLM instruction set based on user role.

--- UT-06: LOG PARSING (JSON vs TEXT) ---
[JSON Parsed Successfully] Extracted: EventID 4625: Failed login attempt
[Text Parsed Successfully] Extracted: 03/24-10:30:05.123 [**] [1:1000001:1] Malicious Traffic [**] (TCP) 192.168.1.50:443

```

[Figure 1: Unit Test Part 1]

```

--- UT-07 & UT-08: THREAT & NOISE FILTERING ---
[UT-08] Malicious Log Suspicious? True (Expected: True)
[UT-08] Benign Log Suspicious? False (Expected: False)
[UT-09] Is 4624 considered Noise? True (Expected: True)
[UT-09] Is 4625 considered Noise? False (Expected: False)

--- UT-09: STATE MANAGEMENT (FILE PROGRESS) ---
[UT-10] Retrieved State: {'apache_attack.log': 1054, 'auth.log': 500}
[RESULT] PASS! State persistence correctly saves and loads byte positions.

--- UT-10: NOTIFICATION THRESHOLDS ---
Test A (Pref: High, Risk: 5) -> Send Email? False (Expected: False)
Test B (Pref: High, Risk: 7) -> Send Email? True (Expected: True)
Test C (Pref: Critical, Risk: 7) -> Send Email? False (Expected: False)
Test D (Pref: Critical, Risk: 9) -> Send Email? True (Expected: True)

--- UT-11: LLM FALLBACK LOGIC ---
Perfect LLM Parsed: (8, ['Step 1: Block IP'])
Broken LLM Parsed (Fallback Triggered): (5, ['Review logs manually'])

--- UT-12: GEMINI BATCH PROCESSING & API TEST ---
Created mock batch of 3 logs.
Sending batch to Gemini API. Awaiting response (10-20 seconds)...

[SUCCESS] Received 3 parsed objects back from Gemini.
-> Processed Event ID: TEST-001 | Risk Score: 7
-> Processed Event ID: TEST-002 | Risk Score: 9
-> Processed Event ID: TEST-003 | Risk Score: 4

[RESULT]: PASS! The LLM successfully processed the batch and returned exactly 1 summary per log without dropping data.

```

[Figure 2: Unit Test Part 2]

Responses

Curl

```
curl -X 'GET' \
'http://127.0.0.1:8000/api/users' \
-H 'accept: application/json'
```

Request URL

```
http://127.0.0.1:8000/api/users
```

Server response

Code	Details
403 <i>Undocumented</i>	Error: Forbidden Response body <pre>{ "detail": "Could not validate credentials" }</pre> Response headers <pre>content-length: 43 content-type: application/json date: Thu, 09 Apr 2026 15:38:05 GMT server: uvicorn</pre>

Responses

Code	Description
200	Successful Response Media type application/json Controls Accept header. Example Value Schema <pre>"string"</pre>

[Figure 3: Api Unit Test 01 Result]

```
Curl
curl -X 'POST' \
'http://127.0.0.1:8000/api/incidents/123/notes' \
-H 'accept: application/json' \
-H 'X-API-Key: ABSR7mqwskt-BUT_jgJ7TRh7Ls0-W0mlw48Wye5A08' \
-H 'Content-Type: application/json' \
-d '{
  "note": "stringaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaddddddddddddddddddddddddddddddddddddddaaaaaaaaaaaaaaaaaaaaaaaaaaa"}'
```

Request URL

http://127.0.0.1:8000/api/incidents/123/notes

Server response

Code	Details
422	Error: Unprocessable Content

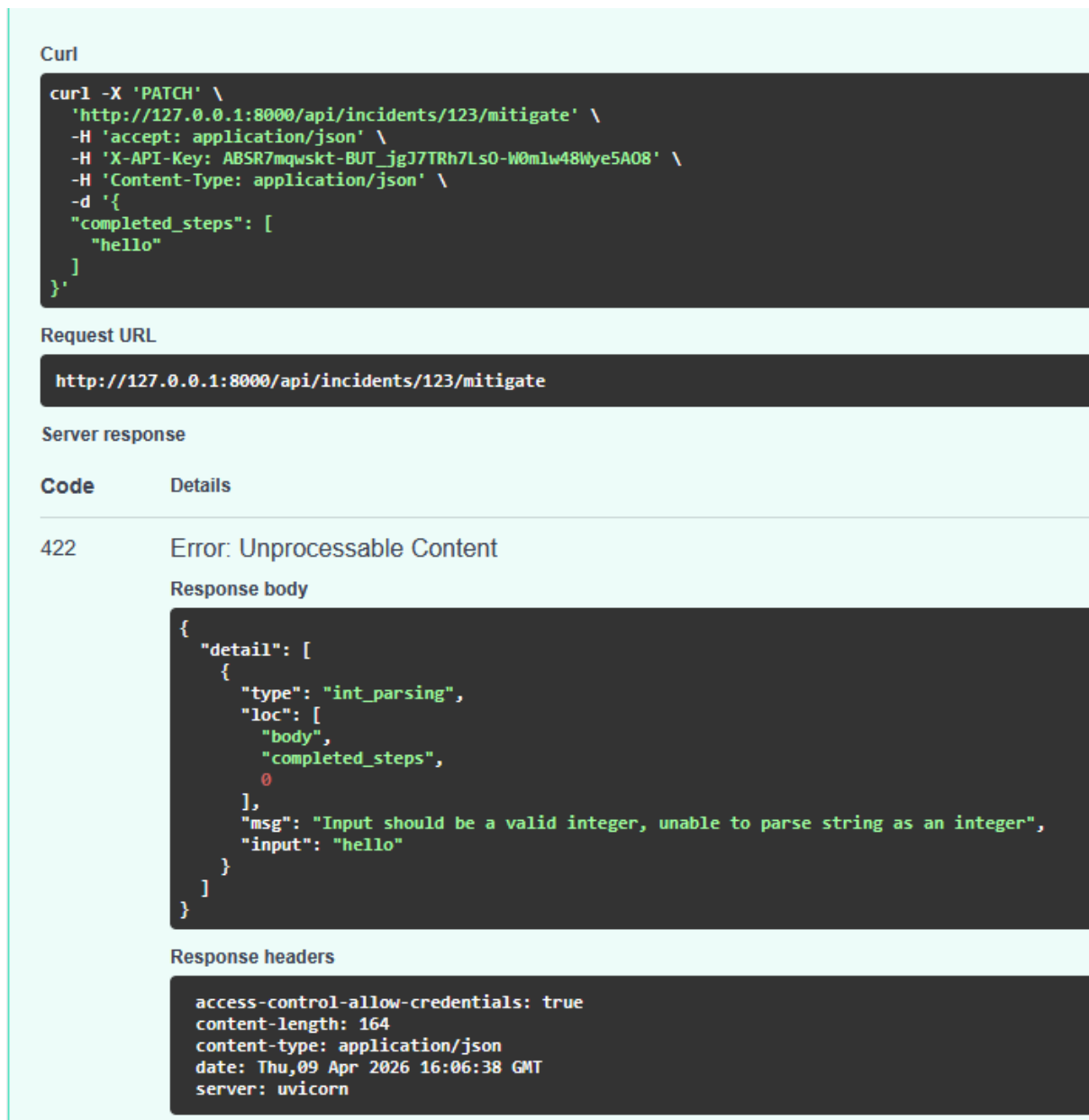
Response body

```
{
  "detail": [
    {
      "type": "string_too_long",
      "loc": [
        "body",
        "note"
      ],
      "msg": "String should have at most 1000 characters",
      "input": "stringaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaddddddddddddddddddddddddddddddddddddddaaaaaaaaaaaaaaaaaaaaaaaaaaa"
    }
  ]
}
```

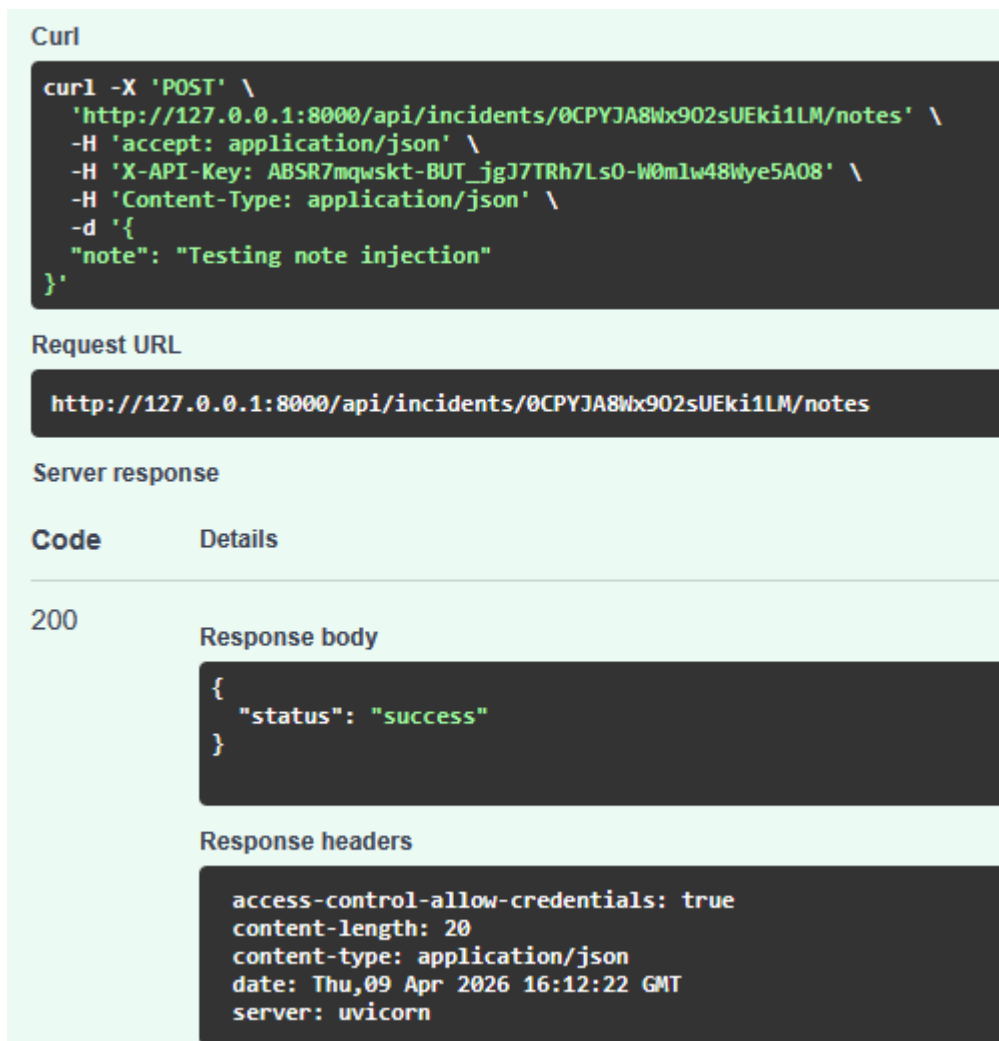
Response headers

```
access-control-allow-credentials: true
content-length: 2015
content-type: application/json
date: Thu, 09 Apr 2026 15:54:59 GMT
server: uvicorn
```

[Figure 4: Api Unit Test 02 Result]



[Figure 5: Api Unit Test 03 Result]



[Figure 6: Api Unit Test 04 Result]

```
INFO:      127.0.0.1:65498 - "GET /api/incidents HTTP/1.1" 200 OK
[SYNC] Firebase pushed an INCIDENT update! Updating RAM cache...
[SYNC] Incident Cache updated successfully.
[AUDIT] Action 'PATCH' securely logged to database.
INFO:      127.0.0.1:59985 - "PATCH /api/incidents/0CPYJA8Wx902sUEki1LM/resolve HTTP/1.1" 200 OK
█
```

[Figure 7: Api Unit Test 05 Result]

```

✓ pages/Analytics.test.tsx (9 tests) 347ms
✓ Analytics Dashboard - Full Component Test Suite (9)
  ✓ [UI-01] Renders the initial loading spinner before data arrives 36ms
  ✓ [UI-02] Calculates the correct Total Events metric 57ms
  ✓ [UI-03] Calculates the correct Open Cases metric 20ms
  ✓ [UI-04] Calculates the correct Critical Threats metric (Score >= 8) 19ms
  ✓ [UI-05] Calculates the correct Resolved cases metric 33ms
  ✓ [UI-06] Displays the red offline boundary if the backend connection fails 20ms
  ✓ [UI-07] Loads with the default "Last 7 Days" time filter selected 98ms
  ✓ [UI-08] Opens the custom date inputs when "Custom Range" is selected 36ms
  ✓ [UI-09] Triggers a new data fetch when the Refresh Analytics button is clicked 23ms

Test Files 1 passed (1)
Tests 9 passed (9)
Start at 08:29:00
Duration 2.21s (transform 123ms, setup 0ms, import 432ms, tests 347ms, environment 1.24s)

```

[Figure 8: Analytics Page Unit Test Result]

```

✓ pages/Incidents.test.tsx (9 tests) 251ms
✓ Incidents Dashboard - Full Component Test Suite (9)
  ✓ [UI-01] Renders the initial loading spinner 33ms
  ✓ [UI-02] Successfully loads and displays incidents in the data table 44ms
  ✓ [UI-03] Filters the table correctly when a search query is entered 22ms
  ✓ [UI-04] Displays the empty state UI when a search finds no results 14ms
  ✓ [UI-05] Opens the Investigation Details view when a table row is clicked 29ms
  ✓ [UI-06] Successfully submits a new investigation note to the API 26ms
  ✓ [UI-07] Successfully marks an incident as resolved via the API 28ms
  ✓ [UI-08] Switches to the Mitigation Playbook and toggles a checkbox 30ms
  ✓ [UI-09] Triggers PDF Generation without crashing the application 22ms

Test Files 1 passed (1)
Tests 9 passed (9)
Start at 08:44:42
Duration 2.13s (transform 164ms, setup 0ms, import 467ms, tests 251ms, environment 1.23s)

```

[Figure 9: Incidents Page Unit Test Result]

```

✓ pages/Login.test.tsx (6 tests) 123ms
✓ Login Component Test Suite (6)
  ✓ [UI-01] Renders the Login form by default 44ms
  ✓ [UI-02] Toggles between Login and Sign Up modes 17ms
  ✓ [UI-03] Displays an error if the user submits an empty form 7ms
  ✓ [UI-04] Successfully calls Firebase Login API with valid credentials 13ms
  ✓ [UI-05] Successfully calls Firebase Auth and Firestore APIs when signing up 20ms
  ✓ [UI-06] Safely catches and displays Firebase Authentication errors 20ms

Test Files 1 passed (1)
Tests 6 passed (6)
Start at 08:50:39
Duration 1.84s (transform 67ms, setup 0ms, import 295ms, tests 123ms, environment 1.24s)

```

[Figure 10: Login Page Unit Test Result]

```

✓ Settings Component Test Suite (6)
  ✓ [UI-01] Renders the Settings dashboard and shows the logged-in user 56ms
  ✓ [UI-02] Fetches existing user preferences from the database on load 29ms
  ✓ [UI-03] Successfully saves new preferences to the database 64ms
  ✓ [UI-04] Successfully triggers the secure Sign Out process 70ms
  ✓ [UI-05] Cancels account deletion if the user declines the warning 16ms
  ✓ [UI-06] Successfully deletes custom data and auth profile when confirmed 97ms

Test Files 1 passed (1)
Tests 6 passed (6)
Start at 08:56:21
Duration 2.20s (transform 93ms, setup 0ms, import 398ms, tests 335ms, environment 1.26s)

```

[Figure 11: Settings Page Unit Test Result]

Appendix G: Integration Tests

```
Processing 25 new lines from alert.ids...

Script stopped manually.
--- [FINAL FLUSH] Processing 14 remaining logs before exit ---

--- [ACTION] BATCH OF 14 READY FOR LLM ---
AI Persona Loaded: Analyze these logs for a non-technical Business Ow...
Waiting 60 seconds for API rate limits...
Consolidated alert sent to mattfish@hotmail.co.uk
Success: Batch processed and notifications sent.
Batch processed and timer reset.
Shutdown complete. Goodbye!
PS C:\Users\mattr\Github\Matt-Fish-COMP3000-Computing-Project\source-code> |
```

[Figure 1: INT 01 Test – Sending data to api, encrypting it, and adding to firestore.]

Panel viewQuery builder

More in Google Cloud

incidents > 16XevfQe9IgwB09rzQtt

(default)

incidents

16XevfQe9IgwB09rzQtt

+ Start collection

+ Add document

+ Start collection

incidents >

16XevfQe9IgwB09rzQtt >

+ Add field

users

5v3DC6IbGFRP76VCkURR

ai_insights

HrXLEt42KRAYuvMaHzUo

DxXxAGSehdJWbDFM5bBUndryzRXT1egXOxnG6-GceEnkmzGhluxxo0tgr5IbYIL6A3tVWeeMdOYVUBYc-

zC0OHGg67lmQjS2XPKQvjkVDOOQoukeBy1ESBT1A6KFA1y381fxNtoLkIdA8rLPYUWJfzbtHto2IPvfyNMHTCMUJ

K4TYXq42ZK3hSLO5Gf05

Ylbq-1eK_smfHb61eBbuiwh3nIBjFn1c93eYzryYbm-

6pbENi45g_0mOnG54yPXRTIMLF7_cm8qzmy9yn3Rkcl_U93zAbmZv5wudEieDOR5oB2GrIDmXLDJbLFAFA-

OhV9439yCHvRZTYS3Mue

BTqNjZizMe8P5nHJTzYr5eyRJmfrOvirYmDGYa4d7wbOBhGxqV1Ahum1AVrawzEL7ONi6boXKM1HAu_og11cZ7a

SrBunttULXsTSHaV_Jvsk9EXKvzoE2WPUnea7YbK2qvpIWE5sJb-

Oia1a2DYzbthkxwMRHER

LBQTSXMDIO6kr7EwhA7mLmuzgOPNsS_0EmOdkMpSzu7iqXQFSrUCJcle3TrO9qca_VYattSTWAh-

loIVmpl3oldptpVMsuuu7rZnh2IDS14TL0_JlOm_X_yyzs3RwHNouO4sP6GbnC3aN6jLEokBsZYUBWu64CdKde

PRKUBxtBs7cxuE5dFIx4

I3gnllqmvlI-o9qJmW9r2lCp7Duoo4MvYMRZID7FJL9KCL_aGG9d9Mk5ektIq13NAK2-

hPWvbiNcc4RkdMs166-WgV2Qf5Uto6MwhfBexyOw1.JejyoVuQLOWDXNH9nmkieh_vL6XzZEg8-EN-

VN68XSC0HvAaEIQ4orjC

RwNAU4Xc_eO1aZ1moTQrRHEIMgJpbBsdTo-

Gk3vawzWT7z5Gcu5ynfFXWPVmDrSKwfrR9F80OrMdhJ17N4emhHD7KyWfhcPns6KOhzUdq3PHZqzHBeSI

Vp6MDL0cUoCaeW5Y1eE

analysis_status: "AI Analysis Complete"

kwWmfq1z2lp3dkuZziz61dbHlQ3_ScDrFEDdNIRbn5aZe9WlQOWbS8sg==

ZzdNCDt1P1KbIH7dbJLa

data: "gAAAAABp2LOkMcgBUmoxbwn9_K0sxhZZ91J62.HelRah9c_M--

is_encrypted: true

cGJyYjYfJ40HxsEoXLIJj

NnxGQhztmtP3mPH8JW6OrbshkD3E0shADlrwq4hPdQXomVPkaEuMpBTfoD-

eRlaoKhGDzP19wCeS9A

cANFSj12yRHpZZ7SQuNbiKJlucy8BruIk-VQwc98P4UPEzj-

FFuMqJLhm5Z2M8ao8IZz

zCOOHGg67lmQjS2XPKQvjkVDOOQoukeBy1ESBT1A6KFA1y381fxNtoLkIdA8rLPYUWJfzbtHto2IPvfyNMHTCMUJ

ggkxm0zcvcPcPswCoSHd

6pbENi45g_0mOnG54yPXRTIMLF7_cm8qzmy9yn3Rkcl_U93zAbmZv5wudEieDOR5oB2GrIDmXLDJbLFAFA-

hbB6fptlirtXvEm74ubuV

SrBunttULXsTSHaV_Jvsk9EXKvzoE2WPUnea7YbK2qvpIWE5sJb-

loIVmpl3oldptpVMsuuu7rZnh2IDS14TL0_JlOm_X_yyzs3RwHNouO4sP6GbnC3aN6jLEokBsZYUBWu64CdKde

hPWvbiNcc4RkdMs166-WgV2Qf5Uto6MwhfBexyOw1.JejyoVuQLOWDXNH9nmkieh_vL6XzZEg8-EN-

Gk3vawzWT7z5Gcu5ynfFXWPVmDrSKwfrR9F80OrMdhJ17N4emhHD7KyWfhcPns6KOhzUdq3PHZqzHBeSI

kwWmfq1z2lp3dkuZziz61dbHlQ3_ScDrFEDdNIRbn5aZe9WlQOWbS8sg==

[Figure 2: INT 01 Test – Firestore Encrypted Data Received]

Security Incidents

Monitor and investigate security events

Generate Report

Filters

Showing 15 of 15 incidents

ID	SEVERITY	STATUS	ASSIGNEE	TIME	EVENT SUMMARY	INVESTIGATION
#782382ae	PENDING	PENDING	Unassigned	10/04/2026, 09:25:55	05/12-10:22:05.123456 [**] [1:2100469:7] GP...	INVESTIGATE
#9da9e9de	LOW	AI ANALYSIS COMPLETE	Unassigned	10/04/2026, 09:25:55	05/12-10:19:01.111222 [**] [1:2100469:7] GP...	INVESTIGATE
#89ebf15f	LOW	AI ANALYSIS COMPLETE	Unassigned	10/04/2026, 09:25:55	05/12-10:15:05.123456 [**] [1:2100469:7] GP...	INVESTIGATE
#4e5a9858	CRITICAL	AI ANALYSIS COMPLETE	Unassigned	10/04/2026, 09:25:54	05/12-10:12:16.111222 [**] [1:2013028:2] ET...	INVESTIGATE
#cF0bce4e	CRITICAL	AI ANALYSIS COMPLETE	Unassigned	10/04/2026, 09:25:54	05/12-10:12:15.555666 [**] [1:2013028:2] ET...	INVESTIGATE

[Figure 3: INT 02 Test – Frontend successfully rendering data on the screen, with a known issue from INT 01 being displayed.]

incidents

+

Add document

5IEIqn8eAu4sWPNoRZdo

69oMQu0tLB4BbDuWEh

CU2394PxFut7QvH6kppq

HotBaJcd6Boji75ipsL7

IGB6rEel1wcrjuJDdgFP

LxxdFxX2HPdbP4AW5RQY

PeR8vORKYVe2fbG2GYZF

WmVaJ6Pxd8ctx8Zmie02

WzWCT8t6jFsn7mvr4TW

YyThbT69sOSukrdCiL3Y

hMkyvn02L7AchF2HuxPv

k9NdVWT35dndkMEbR0ch

l9Zyep5euMcEsXHBrdIk

oLrKaEwcx979zOQKVaqw

rX2D6fJt5BVqsL4kQqOH

uiaA4DlqYgpiISfyJi0M

vhYwWtNxJxxPNeBf56z9

vA3rv7uvvDpnNkrdPaFlU1

PeR8vORKYVe2fbG2GYZF

+

Start collection

+

Add field

ZZ8lhWY9XppD11Ks98QZEJ9Ob4WSdsjzKZ3BnxyQbiJmoXRMSORuJismkiZj5wBJX2OqHYBNU

OPSOjIPJG2HYjbpnH-_f7ru5OCaRMNkuPPPBHwOACXTMSwpcE6HKH8joJtFwXJK215GaT5-

S2KslxEtmJ8NHYIYU96WndliKuOCucvMurQ7TM7q5tIK3dBOqCqWB3MdhVjUoZ9uRygDinkZV12a

LfhwPEsabrzcjmtNNPWTKPIUxicETOfBG2aeHLfibfqttYxwbvOCNwWztsVhtW3cpl-

7SEICAN9I9PBN9VzYmQPPm9Tn5ZTsd2tiOxxwOHfMmSkOcJH9mhwC7wftBzDCQQBF9mVfU8u

dWO4yOJzYAwRi5T7GzYwbFd4WzWSiJWNNettRLFyAGHkc65fft8Y7NGaH9_Sh7GCumbLAYJoCH

syMQiRK1idyXlCoKGZw5qZJWZKwCBZY5e-

fOoCvy5QgVzmKcXpurjDdMM4x2ctvbmIk6vfTejO5OpMQsNv9Z5UafG_VtGTpo7Ct6mZVAGem6

yn1V3EakB4IS2T15FJ320sti_b2JtPukHCOQqHXUJNHbNWxbLPSTw_oJcw1Zj6oBwv-

luttWyyYCBj8RT9P_g2f4iM8yiAJ2A3qulzyyOpUp9EQbnaY89NjoH6w-

OED_RQyeiAeYO48iCKA8ximagrDQBzrhZ4G1dDTkUePmVC1_Q_glhOuFOJUQdw5l-

_ZfVhzGfOqaRv_L5c8eQ-nJ7gcDD71sfyBROK0Zh1RK8SIVUKZ9Qhf6f_251n-

KLTYxoR3jiUqCfGu0qjhpN8s_OqQ_abEZMjnaDoOdleHzdEnH8q4ttghpbO4EkPvbx4WNSbeqY

ZMQDztXlIAh17FPDBgWJBA07hl7qsQggg_qebu_eXmxqOGM-FqQFy-NE_QmYz57ycmvvy5GA7iF

GSReFidic1X7uj4Zaq3yh_3GyDPLezjM6mCwwORa-

4elJUK9qKc_xazsKnOWYfjzU6GOjV3PwY9looHdopY1tutHjy4hes_w9WQDY4Palq3smJMK--

INpdlruGvhlpGazMx3cl1"

is_encrypted: true

risk_score: 2

timestamp: 10 April 2026 at 09:39:52 UTC+1

user_notes

0

"AAAAABp2Lkf1aZ_faQCeUyqiXyjCbaixlIK3_BoS0dOxkWq34u-VRdWffMT2j-f3rtW1AHuBJIYzDrtjmLm93TeTtYV86L1YU3Msl4a8B-_JWEP-Darg=

[Figure 4: INT 03 Test – Frontend successfully encrypted a note and added it to the database.]

POST

identitytoolkit.googleapis.com

accounts:lookup?key=AlzaSyDsyVPTWxaUW4gfUwb3JNOJFYwcKrYVlo

firebase_auth:is1105 (fetch) json

[Figure 5: INT 04 Test – User successfully logs in, however the firebase api key is exposed.]

Appendix H: Performance / Stress Tests

results_map = {res['event_id']: res for res in ai_results}

Monitoring c:\Users\mattr\Github\Matt-Fish-COMP3000-Computing-Project\source-code\backend\src\..\raw-logs for changes...

Processing 1500 new lines from stress_test.ids...

LLM Error: Unterminated string starting at: line 2802 column 26 (char 302095)

Batch processed and timer reset.

Shutdown complete. Goodbye!

PS C:\Users\mattr\Github\Matt-Fish-COMP3000-Computing-Project\source-code> & c:\Users\mattr\Github\Matt-Fish-COMP3000-Computing-Project\source-code\backend\src\..\raw-logs for changes...

[Figure 1: PERF-04 Test Fail.]

Security Incidents

Monitor and investigate security events

Generate Report

Q Search incidents by ID, summary, or notes...

Filters

Showing 1152 of 1152 incidents

ID	SEVERITY	STATUS	ASSIGNEE	TIME	EVENT SUMMARY	INVESTIGATION
#259e46a6	PENDING	PENDING	Unassigned	10/04/2026, 11:13:36	05/12-10:00:18.975 [1:2013028:2] ET EX...	INVESTIGATE
#bf9b0286	PENDING	PENDING	Unassigned	10/04/2026, 11:13:36	05/12-10:00:18.968 [1:2010937:3] ET WE...	INVESTIGATE
#39c386f5	PENDING	PENDING	Unassigned	10/04/2026, 11:13:36	05/12-10:00:18.930 [1:2100469:7] GPL I...	INVESTIGATE
#bda78009	PENDING	PENDING	Unassigned	10/04/2026, 11:13:36	05/12-10:00:18.855 [1:2013028:2] ET EX...	INVESTIGATE
#9f678665	PENDING	PENDING	Unassigned	10/04/2026, 11:13:36	05/12-10:00:18.795 [1:2010937:3] ET WE...	INVESTIGATE
#a6e27218	PENDING	PENDING	Unassigned	10/04/2026, 11:13:36	05/12-10:00:18.735 [1:2010937:3] ET WE...	INVESTIGATE

[Figure 2: PERF-01 and PERF-07 Test Pass.]

Security Incidents

Monitor and investigate security events

Generate Report

Q 259e4

Filters

Showing 1 of 1152 incidents

ID	SEVERITY	STATUS	ASSIGNEE	TIME	EVENT SUMMARY	INVESTIGATION
#259e46a6	PENDING	PENDING	Unassigned	10/04/2026, 11:13:36	05/12-10:00:18.975 [1:2013028:2] ET EXPLD...	INVESTIGATE

[Figure 3: PERF-02 Test Pass.]

Incident Report

Generated: 10/04/2026, 11:19:32

Executive Summary

Total Incidents: 1152

Critical Severity: 359

High Severity: 153

Resolved: 0

Pending: 493

Incident Details

ID	Severity	Status	Timestamp	Summary
259e46a6	PENDING	PENDING	10/04/2026, 11:13:36	N/A
bf9b0206	PENDING	PENDING	10/04/2026, 11:13:36	N/A
39c386f5	PENDING	PENDING	10/04/2026, 11:13:36	N/A
bda70009	PENDING	PENDING	10/04/2026, 11:13:36	N/A
9f678665	PENDING	PENDING	10/04/2026, 11:13:36	N/A

[Figure 4: PERF-03 Test Pass.]

```

INFO: 127.0.0.1:64513 - "GET /api/incidents HTTP/1.1" 200 OK
INFO: 127.0.0.1:64514 - "GET /api/users HTTP/1.1" 200 OK
INFO: 127.0.0.1:64514 - "GET /api/users HTTP/1.1" 200 OK
INFO: 127.0.0.1:64513 - "GET /api/incidents HTTP/1.1" 200 OK
INFO: 127.0.0.1:64517 - "GET /api/incidents HTTP/1.1" 200 OK
INFO: 127.0.0.1:64517 - "GET /api/incidents HTTP/1.1" 200 OK
INFO: 127.0.0.1:64514 - "GET /api/incidents HTTP/1.1" 200 OK
INFO: 127.0.0.1:64513 - "GET /api/users HTTP/1.1" 200 OK
INFO: 127.0.0.1:64514 - "GET /api/users HTTP/1.1" 200 OK
INFO: 127.0.0.1:64513 - "GET /api/incidents HTTP/1.1" 200 OK
INFO: 127.0.0.1:64514 - "GET /api/incidents HTTP/1.1" 200 OK
INFO: 127.0.0.1:64513 - "GET /api/users HTTP/1.1" 200 OK
INFO: 127.0.0.1:64514 - "GET /api/users HTTP/1.1" 200 OK
INFO: 127.0.0.1:64513 - "GET /api/incidents HTTP/1.1" 200 OK
INFO: 127.0.0.1:64517 - "GET /api/incidents HTTP/1.1" 200 OK
INFO: 127.0.0.1:64517 - "GET /api/incidents HTTP/1.1" 200 OK
INFO: 127.0.0.1:64514 - "GET /api/incidents HTTP/1.1" 200 OK
INFO: 127.0.0.1:64513 - "GET /api/users HTTP/1.1" 200 OK
INFO: 127.0.0.1:64513 - "GET /api/users HTTP/1.1" 200 OK
INFO: 127.0.0.1:64514 - "GET /api/incidents HTTP/1.1" 200 OK
INFO: 127.0.0.1:64517 - "GET /api/incidents HTTP/1.1" 200 OK
INFO: 127.0.0.1:64517 - "GET /api/incidents HTTP/1.1" 200 OK
INFO: 127.0.0.1:64521 - "GET /api/incidents HTTP/1.1" 200 OK
INFO: 127.0.0.1:64522 - "GET /api/users HTTP/1.1" 200 OK
INFO: 127.0.0.1:64521 - "GET /api/users HTTP/1.1" 200 OK
INFO: 127.0.0.1:64522 - "GET /api/incidents HTTP/1.1" 200 OK

```

[Figure 5: PERF-05 Test Pass.]

```

--- [ACTION] BATCH OF 50 READY FOR LLM ---
AI Persona Loaded: Analyze these logs for a non-technical Business Ow...
Waiting 60 seconds for API rate limits...
Consolidated alert sent to mattrfish@hotmail.co.uk
Success: Batch processed and notifications sent.
Batch processed and timer reset.

--- [ACTION] BATCH OF 50 READY FOR LLM ---
AI Persona Loaded: Analyze these logs for a non-technical Business Ow...
Waiting 60 seconds for API rate limits...
Consolidated alert sent to mattrfish@hotmail.co.uk
Success: Batch processed and notifications sent.
Batch processed and timer reset.

```

[Figure 5: PERF-06 Test Pass.]

S.I.R.A. User Acceptance Testing (UAT) Questionnaire

Thank you for testing S.I.R.A. (Security Incident Response Assistant). During this test, please complete the tasks given to you and rate how well the system performs on a scale of 1 to 5.

- **1** = Very Poor / Very Difficult
- **5** = Excellent / Very Easy

mattrobfish@gmail.com [Switch accounts](#)

 Not shared

* Indicates required question

Name *

Your answer

What is your current technical level or role? *

☐ Computer Science Student

☐ Business Owner / Non-Technical

☐ IT Support / Junior Sysadmin

☐ SOC Analyst / Senior Security Expert

Next

Clear form

[Figure 1: Blank UAT Page 1.]

S.I.R.A. User Acceptance Testing (UAT) Questionnaire

mattrobfish@gmail.com [Switch accounts](#)



Not shared

Section 1: Account Creation & Authentication

Task: Sign up for a new account and log in.

How easy and straightforward was the process of creating a new account?

	1	2	3	4	5	
Very Hard	☐	☐	☐	☐	☐	Very Easy

How smooth was the experience of logging into the application?

	1	2	3	4	5	
Very Complicated	☐	☐	☐	☐	☐	Very Smooth

[Figure 2: Blank UAT Section 1, Page 2.]

Section 2: Dashboard & Analytics

Task: Navigate the home dashboard, review the top-level statistics, and interpret the "Threat Activity Trend" and "Risk Breakdown" charts.

How clear and understandable are the top-level statistics (Total Events, Critical Threats, Resolved)?

1

2

3

4

5

Very Unclear

☐

☐

☐

☐

☐

Very Clear

How easy is it to interpret the Threat Activity Trend area chart?

1

2

3

4

5

Very Hard

☐

☐

☐

☐

☐

Very Easy

How well does the Risk Breakdown pie chart communicate the current threat severity?

1

2

3

4

5

Very Poor

☐

☐

☐

☐

☐

Very Well

How easy is it to use the time-range filters (e.g., "Last 7 Days", "Custom Range") to update the data?

1

2

3

4

5

Very Hard

☐

☐

☐

☐

☐

Very Easy

[Figure 3: Blank UAT Section 2, Page 3.]

Section 3: Incident Triage & Filtering

Task: Navigate to the "Incidents" tab. Search for a specific alert, and use the filters to show only "Critical" severity incidents.

How easy is it to navigate the main Incidents list and understand the table layout?

	1	2	3	4	5	
Very Hard	☐	☐	☐	☐	☐	Very Easy

How effectively does the Search bar find specific incidents or keywords?

	1	2	3	4	5	
Very Uneffective	☐	☐	☐	☐	☐	Very Effective

How simple is it to use the severity and status filter buttons?

	1	2	3	4	5	
Very Hard	☐	☐	☐	☐	☐	Very Simple

[Figure 4: Blank UAT Section 3, Page 4.]

Section 4: AI Investigation & Translation

Task: Click "Investigate" on an incident. Read the AI Summary and review the technical details.

How well does the AI Summary explain the threat without using confusing jargon?

	1	2	3	4	5	
Very Poor	☐	☐	☐	☐	☐	Very Well

How clearly does the Risk Score (1-10) communicate the urgency of the threat?

	1	2	3	4	5	
Very Unclear	☐	☐	☐	☐	☐	Very Clear

How useful is the presentation of the Raw Event Data and Technical Details (IPs, MAC addresses)?

	1	2	3	4	5	
Very Poor	☐	☐	☐	☐	☐	Very Useful

[Figure 5: Blank UAT Section 4, Page 5.]

Section 5: Incident Response & Mitigation

Task: Navigate to the "Mitigation Plan". Check off a mitigation step, assign the incident to a user, add a note, and mark it as resolved.

How clear and actionable are the step-by-step instructions in the AI Response Playbook?

	1	2	3	4	5	
Very Unclear	☐	☐	☐	☐	☐	Very Clear

How easy is it to assign a team member to the incident?

	1	2	3	4	5	
Very Hard	☐	☐	☐	☐	☐	Very Easy

How intuitive is the process of adding an investigation note?

	1	2	3	4	5	
Very Hard	☐	☐	☐	☐	☐	Very Easy

How seamless is the process of marking the incident as "Resolved"?

	1	2	3	4	5	
Very Hard	☐	☐	☐	☐	☐	Very Easy

[Figure 6: Blank UAT Section 5, Page 6.]

Section 6: Reporting & Notifications

Task: Check the notification bell in the top right, then generate a PDF report from the Incidents page.

How effective is the top navigation bar's notification bell at highlighting new alerts?

	1	2	3	4	5	
Very Uneffective	☐	☐	☐	☐	☐	Very Effective

How easy is it to generate an Incident Report (PDF)?

	1	2	3	4	5	
Very Hard	☐	☐	☐	☐	☐	Very Easy

How professional and readable is the layout of the generated PDF report?

	1	2	3	4	5	
Very Unreadable	☐	☐	☐	☐	☐	Very Readable

[Figure 7: Blank UAT Section 6, Page 7.]

Section 7: Settings & Preferences

Task: Navigate to the Settings page, change your AI Persona to "IT Support", and adjust your email notification preferences.

How easy was it to understand the options and change your AI Persona (Technical Level) to "IT Support"?

	1	2	3	4	5	
Very Hard	☐	☐	☐	☐	☐	Very Easy

How easy was it to find and adjust your email notification preferences?

	1	2	3	4	5	
Very Hard	☐	☐	☐	☐	☐	Very Easy

Overall, how smooth was the experience of configuring your dashboard settings?

	1	2	3	4	5	
Very Frustrating	☐	☐	☐	☐	☐	Very Smooth

[Figure 8: Blank UAT Section 7, Page 8.]

Section 8: Final Feedback

What did you like most about using the S.I.R.A. dashboard?

Your answer

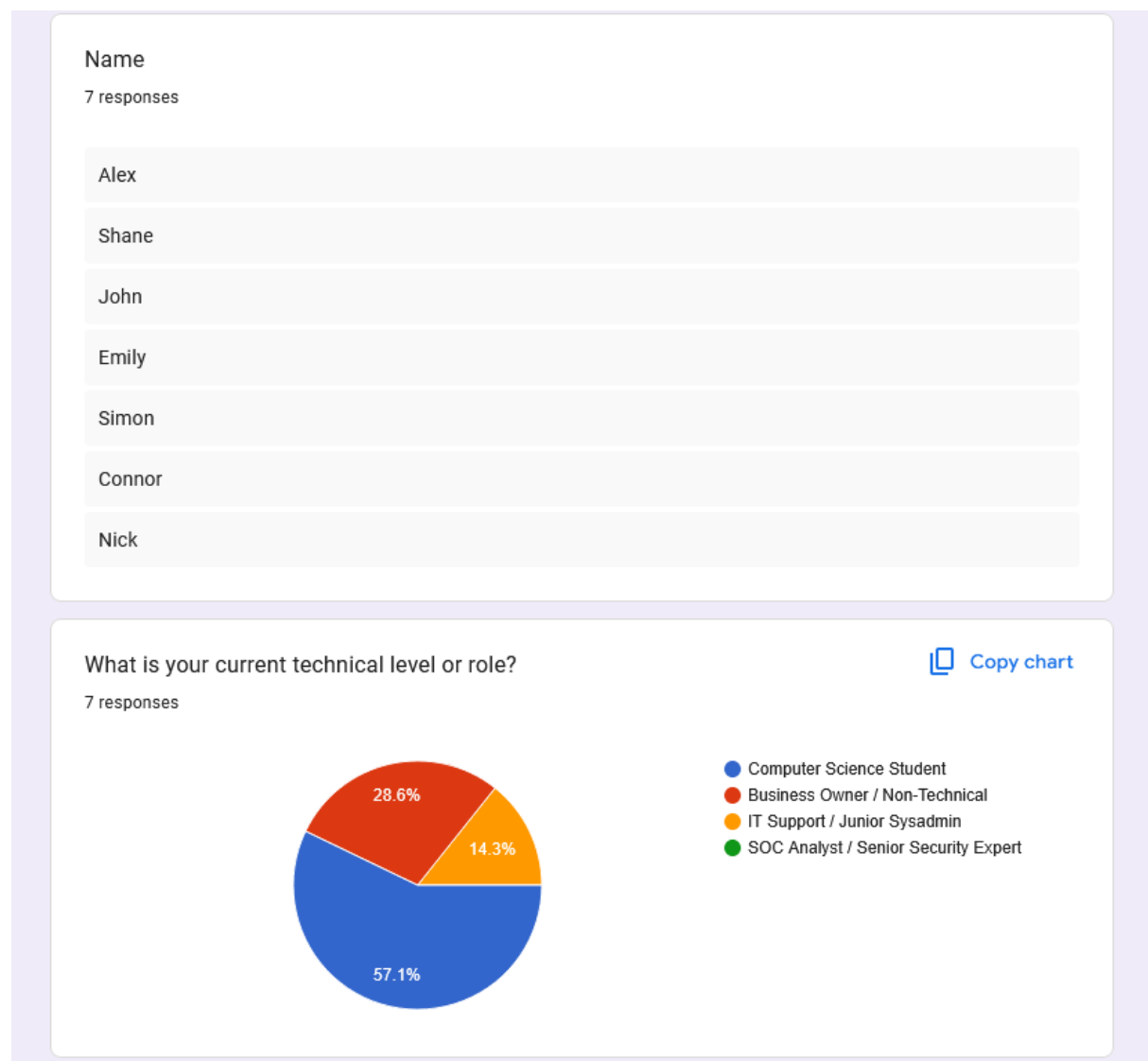
What was the most confusing or difficult part of the system?

Your answer

If you could add or change one feature to improve this tool for a business owner, what would it be?

Your answer

[Figure 9: Blank UAT Section 8, Page 9.]



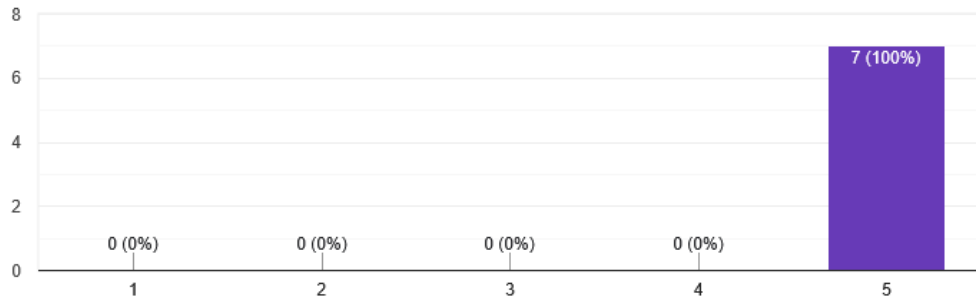
[Figure 10: Completed UAT.]

Section 1: Account Creation & Authentication

How easy and straightforward was the process of creating a new account?

 [Copy chart](#)

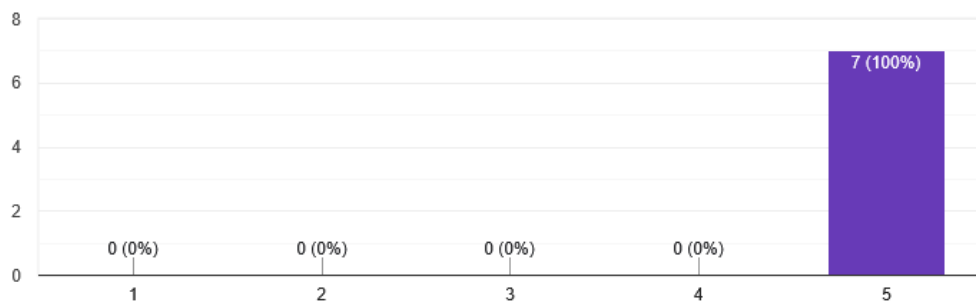
7 responses



How smooth was the experience of logging into the application?

 [Copy chart](#)

7 responses



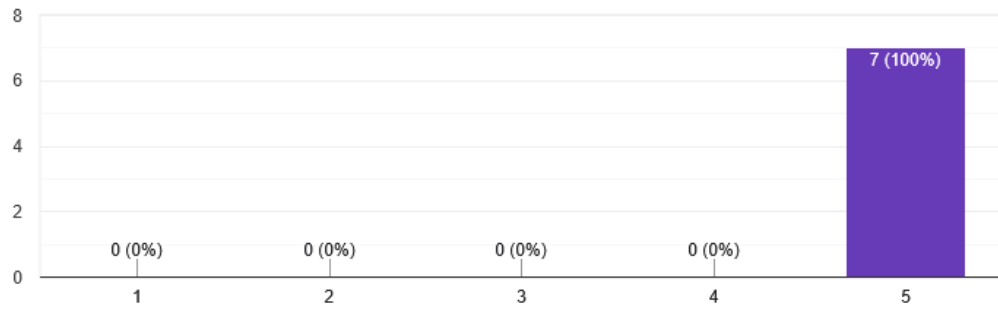
[Figure 11: Completed UAT Section 1, Q1+2.]

Section 2: Dashboard & Analytics

How clear and understandable are the top-level statistics (Total Events, Critical Threats, Resolved)?

 [Copy chart](#)

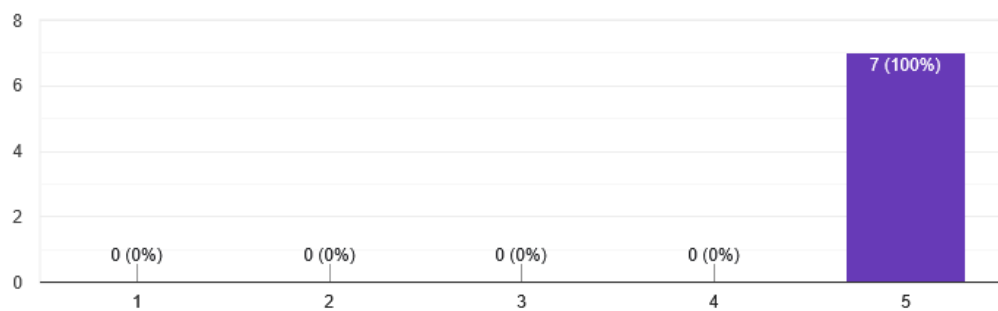
7 responses



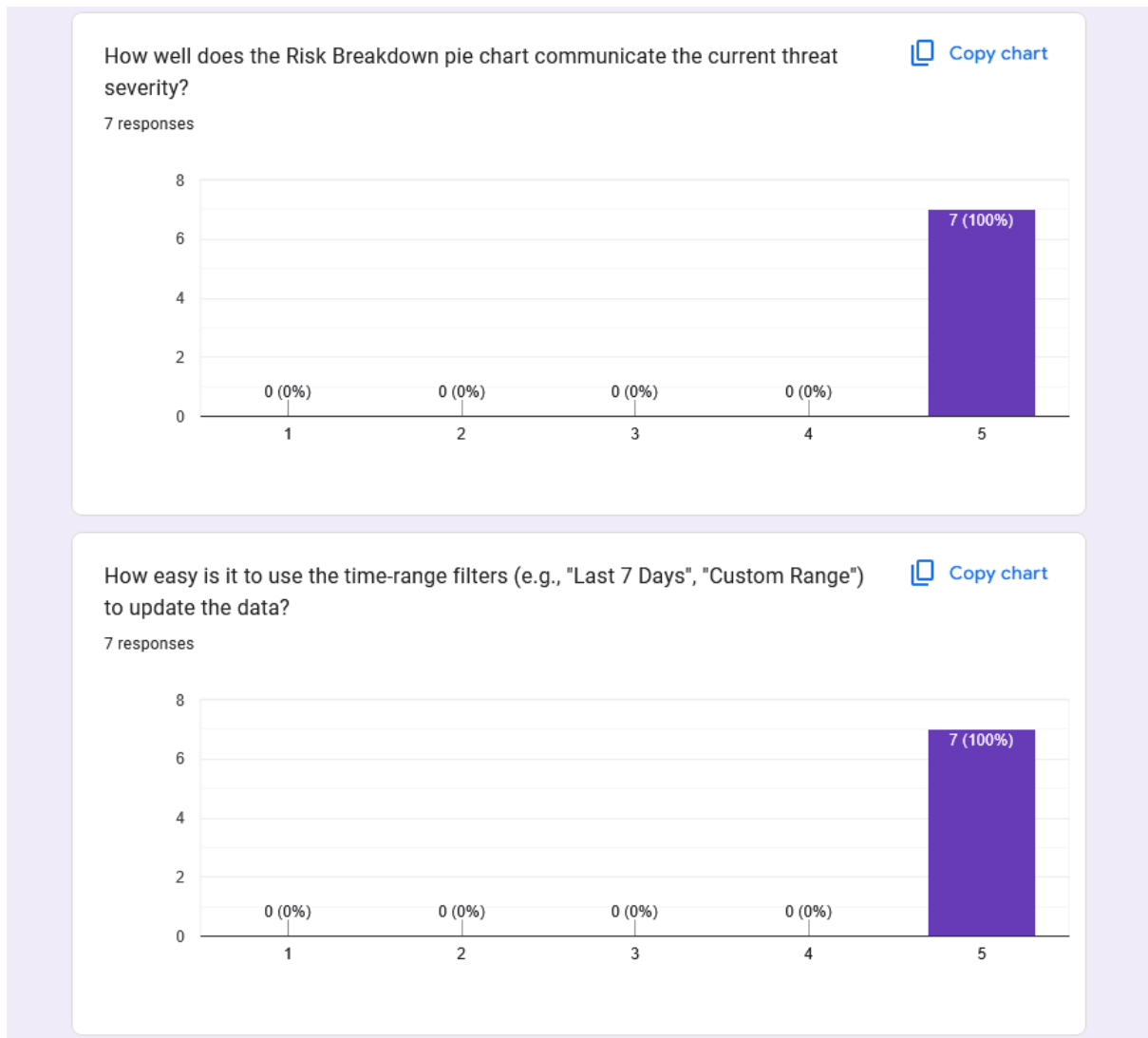
How easy is it to interpret the Threat Activity Trend area chart?

 [Copy chart](#)

7 responses



[Figure 12: Completed UAT Section 2, Q1+2.]



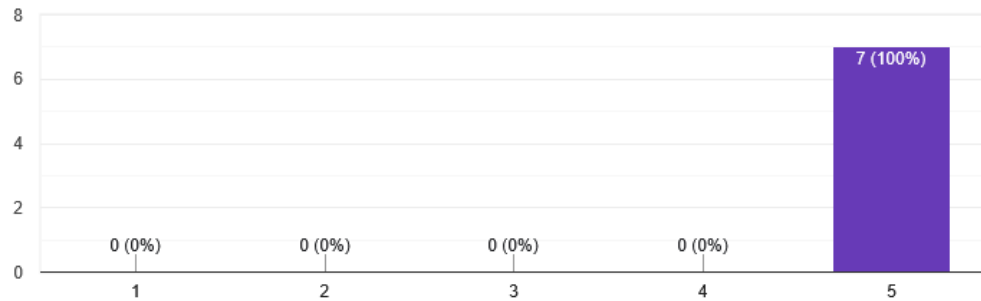
[Figure 13: Completed UAT Section 2, Q3+4.]

Section 3: Incident Triage & Filtering

How easy is it to navigate the main Incidents list and understand the table layout?

 Copy chart

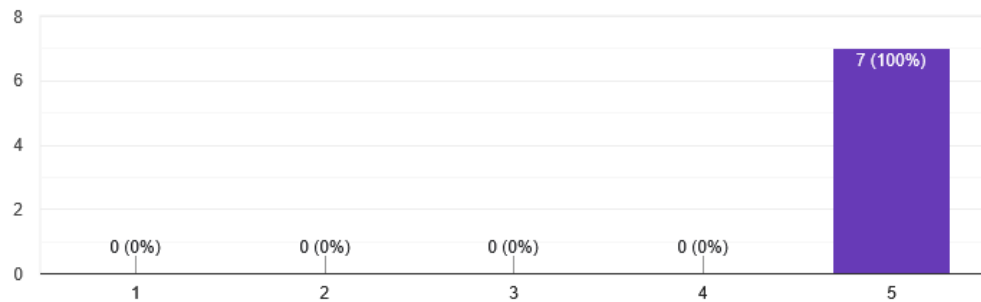
7 responses



How effectively does the Search bar find specific incidents or keywords?

 Copy chart

7 responses

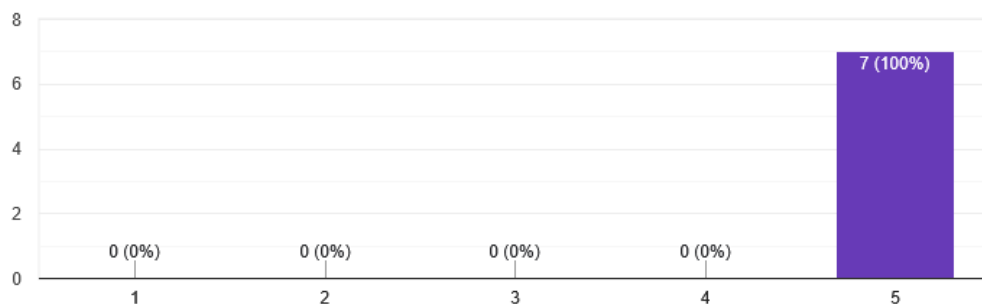


[Figure 14: Completed UAT Section 3, Q 1+2.]

How simple is it to use the severity and status filter buttons?

 Copy chart

7 responses



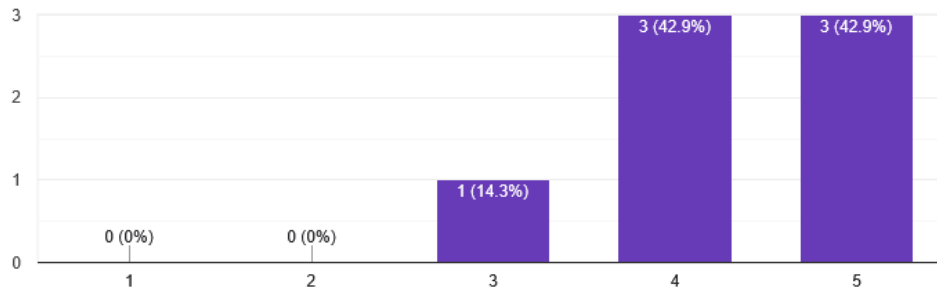
[Figure 15: Completed UAT Section 3, Q3.]

Section 4: AI Investigation & Translation

How well does the AI Summary explain the threat without using confusing jargon?

[Copy chart](#)

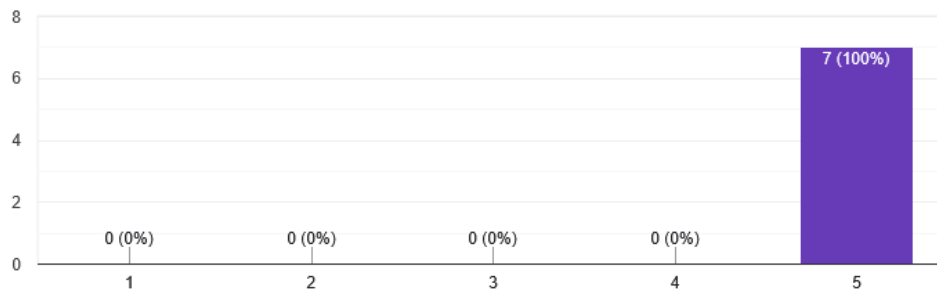
7 responses



How clearly does the Risk Score (1-10) communicate the urgency of the threat?

[Copy chart](#)

7 responses

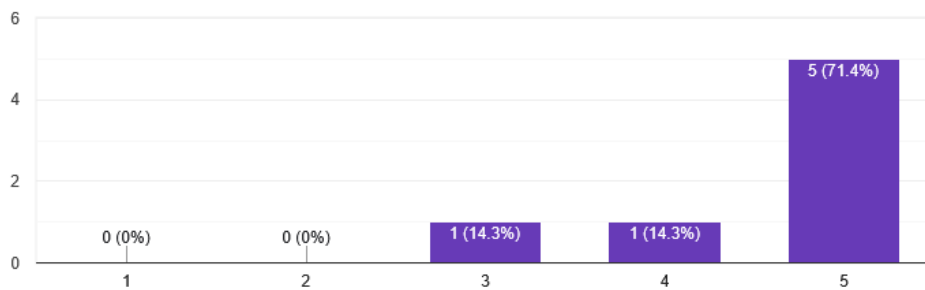


[Figure 16: Completed UAT Section 4, Q1+2.]

How useful is the presentation of the Raw Event Data and Technical Details (IPs, MAC addresses)?

[Copy chart](#)

7 responses



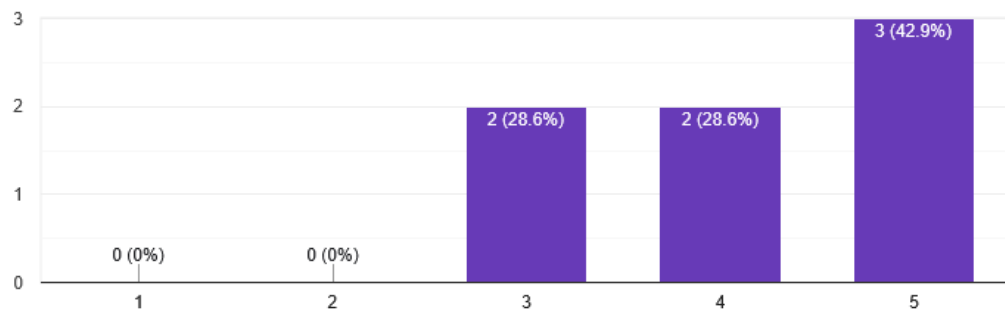
[Figure 17: Completed UAT Section 4, Q3.]

Section 5: Incident Response & Mitigation

How clear and actionable are the step-by-step instructions in the AI Response Playbook?

[Copy chart](#)

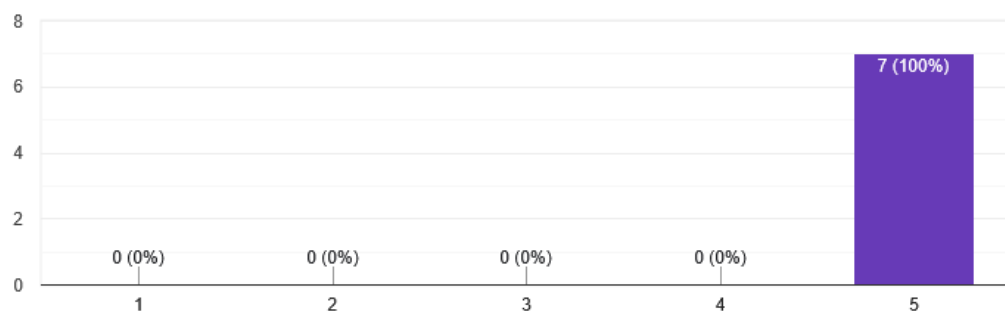
7 responses



How easy is it to assign a team member to the incident?

[Copy chart](#)

7 responses

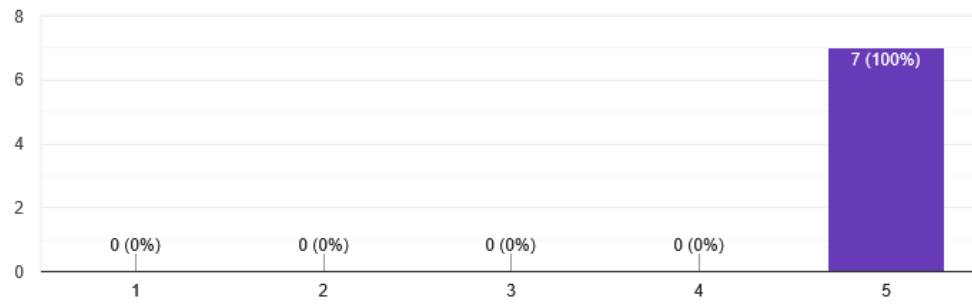


[Figure 18: Completed UAT Section 5, Q1+2.]

How intuitive is the process of adding an investigation note?

 [Copy chart](#)

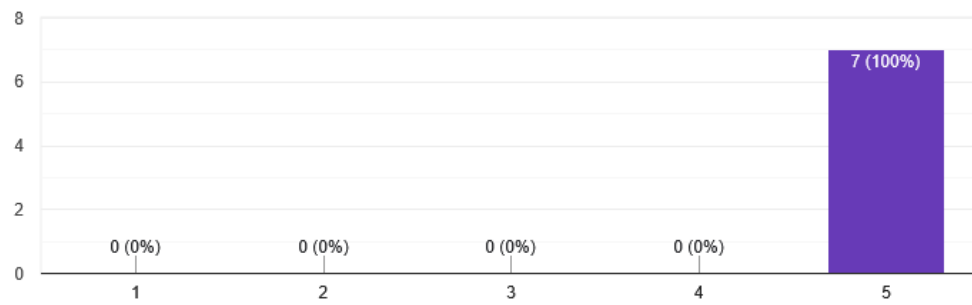
7 responses



How seamless is the process of marking the incident as "Resolved"?

 [Copy chart](#)

7 responses



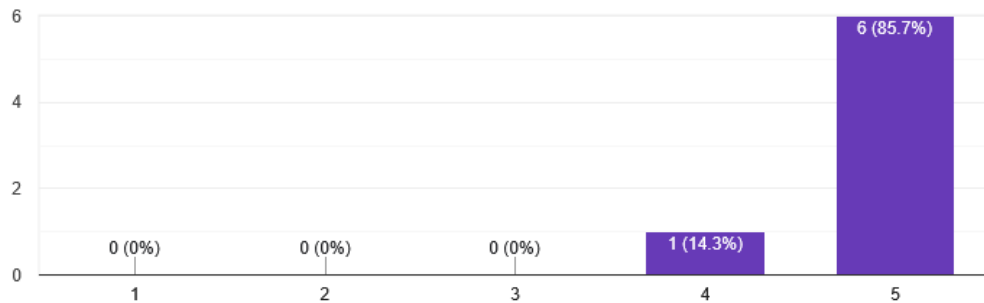
[Figure 19: Completed UAT Section 5, Q3+4.]

Section 6: Reporting & Notifications

How effective is the top navigation bar's notification bell at highlighting new alerts?

[Copy chart](#)

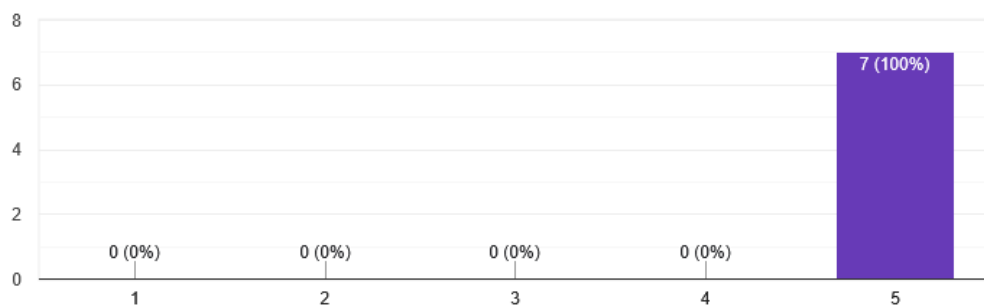
7 responses



How easy is it to generate an Incident Report (PDF)?

[Copy chart](#)

7 responses

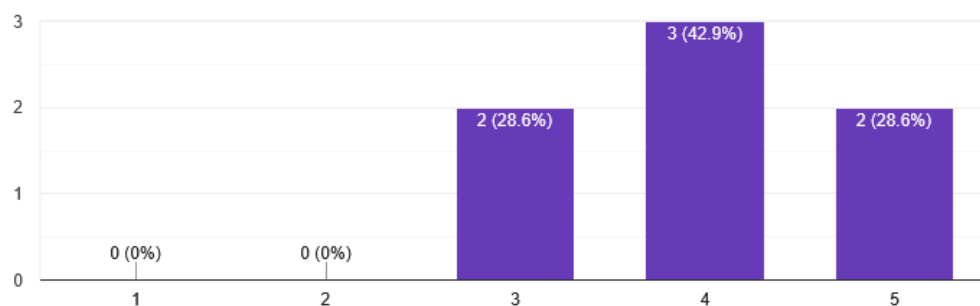


[Figure 20: Completed UAT Section 6, Q1+2.]

How professional and readable is the layout of the generated PDF report?

[Copy chart](#)

7 responses



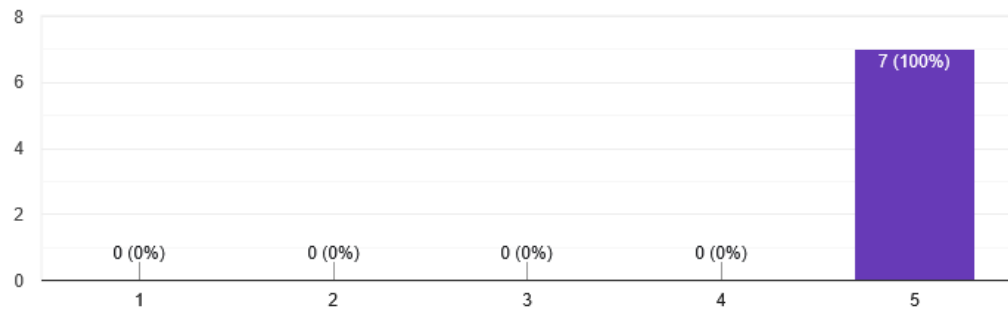
[Figure 21: Completed UAT Section 6, Q3.]

Section 7: Settings & Preferences

How easy was it to understand the options and change your AI Persona (Technical Level) to "IT Support"?

[Copy chart](#)

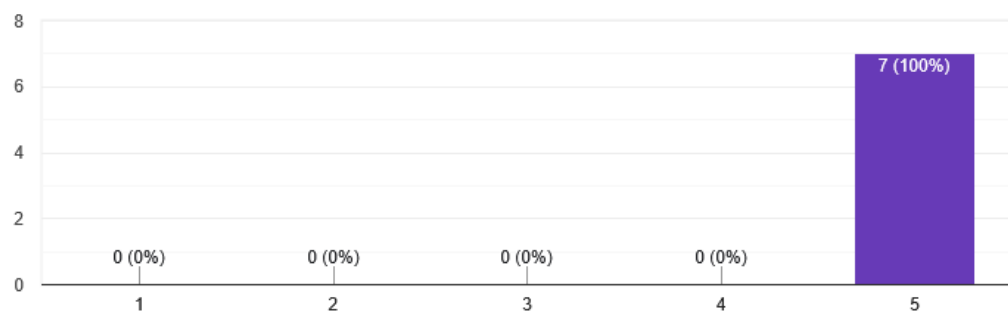
7 responses



How easy was it to find and adjust your email notification preferences?

[Copy chart](#)

7 responses

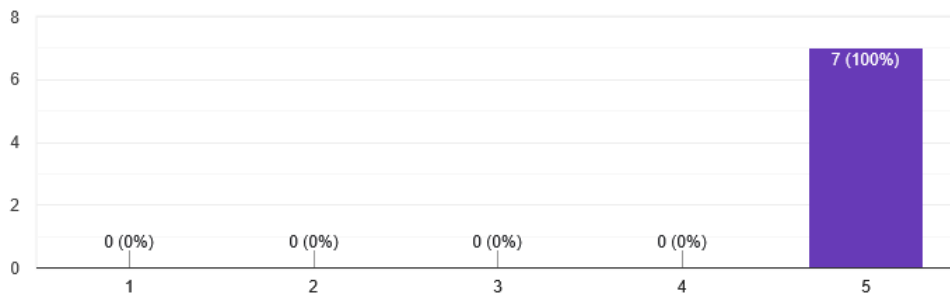


[Figure 22: Completed UAT Section 7, Q1+2.]

Overall, how smooth was the experience of configuring your dashboard settings?

[Copy chart](#)

7 responses



[Figure 23: Completed UAT Section 7, Q3.]

Section 8: Final Feedback

What did you like most about using the S.I.R.A. dashboard?

7 responses

◆ Summarise responses

I think that its very simplistic which is easy for a non tehcnical user to navigate and helps me undertand and identify mian weaknesses that hackers might be trying to find..

The use of AI to summarise alerts and give example mitigations is very unique. For a junior SOC this woudl be great to help them learn to mitigate threats they may have not seen, instead of just escalating it to higher ups and not leaning from it.

Modern Layout with some easy to use functionality and cool analytics.

The layout is simple and easy to navigate thorough lots of alerts.

Very good app for displaying incident threats compared to expensive SIEMs. I think the AI responses really make a different for filtering out noise and understanding which alerts should be prioritied.

Its simple and intuitive to use comapred to other siem dashboards.

AI generated summaries for alerts.

[Figure 24: Completed UAT Section 8, Q1.]

What was the most confusing or difficult part of the system?

7 responses

✦ Summarise responses

The AI summaries / mitigation dont make me fully understand the incident and can be quite brief. A lot of mitigations also involve escalating to a team which im unsure on what to do if i dont have an IT team.

the in app notifications were easy to navigate to the threats but only show a few threats at once unless you click clear all to dismiss those which brings up the next few. Maybe a scroll for all notifications would be better.

Confused on why the IPs and MACs are redacted and added underneath. If its for AI reasons, would be better to try and add them back into the technical alert instead. Also some of the redacted info (user names for example) isnt being shown underneath which would be useful to know if a certain system needed isolating.

Im not very technical so understanding some of the alerts but the AI does do a good job to help.

Id say overall it wasnt confusing or difficult to use. Could potentially be a bit tricky for absolute beginners to try and interpret but the AI does give some good output.

N/A

[Figure 25: Completed UAT Section 8, Q2.]

If you could add or change one feature to improve this tool for a business owner, what would it be?

7 responses

✦ Summarise responses

Slightly improve some of the AI responses.

Its missing a forgot password button which isnt a major issue but just a recommendation.

Personally, id like to see some more statistics like maybe searching for keywords (sql injection, nmap etc) and seeing the percentage of different types of attacks.

A variety of different analytics.

Id include some more analytics and maybe even put them into an LLM to further interpret what is going on over the course of weeks/months at a wider scale.

Add more analytics for analysis of incidents.

More features like analytics to analyse threats in other ways.

[Figure 25: Completed UAT Section 8, Q3.]

Appendix J: Code Snippets

```
# Define keywords for suspicious prediction
class ThreatDictionary:
    # 1. Web Application Attacks (SQLi, XSS, Path Traversal)
    WEB_ATTACKS = [
        "<script>", "alert(", "onerror=", "onload=", "eval(", "src=", # XSS
        "union select", "select *", "drop table", "insert into", "order by", "--", " ' or '1'='1", # SQL
        "../", "..\\", "etc/passwd", "windows/system32", "boot.ini", ".env", ".git", # Path Traversal /
        "jndi:ldap", "jndi:rmi", #Log4j exploits
        "wp-admin", "wp-login", # WordPress brute force/scanning
        "whoami", "ifconfig", "ipconfig" # Command execution output
    ]

    # 2. Authentication & Account Security (Brute Force)
    AUTH_ATTACKS = [
        "failed password", "invalid user", "authentication failure", "unauthorized",
        "login failed", "access denied", "bad password", "locked out", "user not found", "4625", # Windo
        "maximum authentication attempts exceeded", # Linux aggressive brute force
        "preauth", # SSH pre-authentication failures
        "4740" # Windows: User Account Locked Out
    ]

    # 3. System & Malware Indicators (Post-Exploitation)
    SYSTEM_ATTACKS = [
        "rm -rf", "sudo", "chmod", "chown", "wget ", "curl ", "netcat", "nc -e", # Command Injection
        "compromised", "unexpected service", "malicious", "backdoor", "shell",
        "powershell", "base64", "python -c", "perl -e", # Common script execution
        "4720", "4732", "1102", "7045", "vssadmin delete shadows",
        "lsass.exe", "invoke-expression", # Windows specific
        "certutil.exe -urlcache", "certutil.exe -split", # Often used by malware to download payloads
        "schtasks /create", "bitsadmin", # Windows persistence mechanisms
        "disableantispware", "reg add" # Tampering with Windows Defender/Registry
    ]

    # 4. Network Scanning & Reconnaissance (Probing)
    RECON = [
        "nmap", "masscan", "dirbuster", "nikto", "sqlmap", "iptables-dropped",
        "connection refused", "port scan", "icmp", "test packet",
        "zgrab", "nessus", "acunetix", "w3af" # Common automated vulnerability scanners
    ]

    @classmethod
    def get_all(cls):
        # combine everything into one massive list for initial filter
        return cls.WEB_ATTACKS + cls.AUTH_ATTACKS + cls.SYSTEM_ATTACKS + cls.RECON

SUSPICIOUS_KEYWORDS = ThreatDictionary.get_all()
```

[Figure 1: Code snippet defining the ThreatDictionary, used to categorise and aggregate suspicious keywords for initial log triage.]

```
# list of patterns that are noisy but safe
# Expanded to aggressively filter out normal Windows and Linux background noise
KNOWN_SAFE_PATTERNS = [
    # Linux / Unix Noise
    "session opened", "session closed", "systemd: started", "ntupdate",
    "authorized_keys", "cron[", "postfix/", "dovecot:", "crond[",
    "reached target", "pms-refresh", "status=sent (250 2.0.0 ok",
    "starting update inventory", "connection closed by authenticating user",
    "removed slice", "created slice", "starting session", "started session", # Systemd slice spam
    "pam_unix(cron:session)", "dhcpack", "dhcpdiscover", # DHCP and Cron spam

    # Windows Noise
    "4624", # Windows: Successful Logon (EXTREMELY NOISY)
    "4634", # Windows: Successful Logoff
    "4672", # Windows: Special privileges assigned to new logon (Noisy for admin accounts)
    "5140", # Windows: A network share object was accessed
    "5156", # Windows: Windows Filtering Platform has allowed a connection
    "service control manager", "system idle process",

    # Web / Network Noise
    "favicon.ico", "robots.txt", # Standard web crawler requests
    "\"get / http/1.1\" 200", "\"get / http/1.0\" 200" # Basic successful web loads
]
```

[Figure 2: Code snippet of the KNOWN_SAFE_PATTERNS array, used to filter benign background noise during initial log triage.]

```
# adding batching for brute force to reduce lines being parsed to llm at once
BATCH_LIMIT = 50 # Number of suspicious lines to collect before calling LLM
MAX_WAIT_SECONDS = 90 # 1 1/2 mins
last_batch_time = time.time() # Initialise the timer
suspicious_buffer = [] # Temporary list to hold lines
processed_files_announced = set() # stop the terminal spam for processed logs check
```

[Figure 3: Code snippet defining the core global variables used to control the LLM batching parameters.]

```

# batching logic
if event["is_suspicious"]:

    #encrypt files
    encrypted_token = encrypt_payload(event)

    # Firestore needs to receive a dictionary { "key": "value" }
    encrypted_payload = {
        "data": encrypted_token,
        "is_encrypted": True,
        "timestamp": firestore.SERVER_TIMESTAMP # used for sorting
    }

    # Push to Firestore
    doc_ref = db.collection("incidents").add(encrypted_payload)

    # Add doc ID for later LLM updates
    actual_doc_id = doc_ref[1].id
    event['doc_id'] = actual_doc_id # The Firestore UUID (e.g., "zX9yP...")

    # Add to the buffer for batching
    suspicious_buffer.append(event)

    # Trigger batch if limit reached
    if len(suspicious_buffer) >= BATCH_LIMIT:
        process_batch(suspicious_buffer) # send to LLM
        suspicious_buffer = [] # clear the buffer back to empty

# everything (including unsuspicious data) appended to processed_events so LOCAL JSON
# files remain a complete record of the whole log file
processed_events.append(event)

```

[Figure 4: Implementation of the batching pipeline, detailing how encrypted alerts are buffered locally and batched to optimize external API usage.]

```

# Batch Timeout Logic
time_since_last_batch = time.time() - last_batch_time
if len(suspicious_buffer) > 0 and time_since_last_batch >= MAX_WAIT_SECONDS:
    print(f"--- [TIMEOUT] Processing partial batch of {len(suspicious_buffer)} ---")
    process_batch(suspicious_buffer)
    suspicious_buffer = []

if not new_data_found:
    time.sleep(2) # Poll every 2 seconds

if __name__ == "__main__":
    try:
        log_watcher()
    except KeyboardInterrupt:
        print("\nScript stopped manually.")
        # Final flush before the script actually stops
        if len(suspicious_buffer) > 0:
            print(f"--- [FINAL FLUSH] Processing {len(suspicious_buffer)} remaining logs before exit ---")
            process_batch(suspicious_buffer)
        print("Shutdown complete. Goodbye!")
        sys.exit(0)

```

[Figure 5: Code snippet illustrating the batch timeout logic and shutdown sequence.]

```
# Define the 3 distinct personalities for ai prompt targeting
prompts = {
    "business_owner": (
        "Analyze these logs for a non-technical Business Owner (SME). "
        "Your goal is to provide absolute clarity without technical jargon. "
        "For the 'summary': Explain exactly what happened, if it is dangerous, and what the direct business impact is. "
        "For the 'mitigation_steps': Provide actionable, plain-English steps. You MUST explain exactly *what* each step does and *why* it is necessary. "
        "If a step requires IT help, tell them exactly what to ask their IT provider to do (e.g., 'Ask your IT team to block the attacker's IP address. This stops the hacker from attempting further logins.'). "
        "Note: Not all busienssers will have an IT team, so ensure that they will be able to understand and mitiage the threats to the best of their ability on their own."
    ),
    "it_support": (
        "Analyze these logs for a Junior IT Sysadmin. "
        "Use standard IT terminology. Focus on what service is failing, whether it's user error or a bug, and how to fix it. "
        "For the 'summary': Detail the technical issue clearly. "
        "For the 'mitigation_steps': Provide specific actions. Include exact CLI commands (e.g., 'sudo systemctl restart nginx') where applicable, but you MUST briefly explain what the command achieves so the junior admin learns from the process."
    ),
    "soc_analyst": (
        "Analyze these logs for a Senior Security Analyst (SOC). "
        "Be extremely technical. Focus on attack vectors, payload analysis (SQLi/XSS patterns), and Indicators of Compromise (IOCs). "
        "For the 'summary': Provide a deep dive into the threat mechanics. "
        "For the 'mitigation_steps': Provide exact, advanced remediation commands (e.g., specific iptables drops, patching CVEs) and long-term architectural recommendations."
    )
}

return prompts.get(level, prompts["business_owner"])
```

[Figure 6: Code snippet defining the dynamic, role-based AI persona prompts used to tailor LLM responses.]

```
# Define the 3 distinct personalities for ai prompt targeting
prompts = {
    "business_owner": (
        "Analyze these logs for a non-technical Business Owner (SME). "
        "Your goal is to provide absolute clarity without technical jargon. "
        "For the 'summary': Explain exactly what happened, if it is dangerous, and what the direct business impact is. "
        "For the 'mitigation_steps': Provide actionable, plain-English steps. You MUST explain exactly *what* each step does and *why* it is necessary. "
        "If a step requires IT help, tell them exactly what to ask their IT provider to do (e.g., 'Ask your IT team to block the attacker's IP address. This stops the hacker from attempting further logins.'). "
        "Note: Not all busienssers will have an IT team, so ensure that they will be able to understand and mitiage the threats to the best of their ability on their own."
    ),
    "it_support": (
        "Analyze these logs for a Junior IT Sysadmin. "
        "Use standard IT terminology. Focus on what service is failing, whether it's user error or a bug, and how to fix it. "
        "For the 'summary': Detail the technical issue clearly. "
        "For the 'mitigation_steps': Provide specific actions. Include exact CLI commands (e.g., 'sudo systemctl restart nginx') where applicable, but you MUST briefly explain what the command achieves so the junior admin learns from the process."
    ),
    "soc_analyst": (
        "Analyze these logs for a Senior Security Analyst (SOC). "
        "Be extremely technical. Focus on attack vectors, payload analysis (SQLi/XSS patterns), and Indicators of Compromise (IOCs). "
        "For the 'summary': Provide a deep dive into the threat mechanics. "
        "For the 'mitigation_steps': Provide exact, advanced remediation commands (e.g., specific iptables drops, patching CVEs) and long-term architectural recommendations."
    )
}

return prompts.get(level, prompts["business_owner"])

{
    "step_number": 1,
    "action_title": "A clear, concise title for this step.",
    "who_should_execute": "Specify who should do this (e.g., 'Business Owner', 'External IT Provider', 'Network Engineer').",
    "detailed_instructions": "Exact, step-by-step instructions. For IT/SOC, include specific CLI commands. For Business Owners, include exactly what to tell their IT team.",
    "why_this_is_necessary": "A deep explanation of what this specific action achieves and the risk of NOT doing it."
},
"risk_assessment": {
    "score": "integer between 1 and 10",
    "severity": "<Critical, High, Medium, or Low>",
    "justification": "A detailed reason why this specific score and severity were assigned based on the log evidence."
}
```

[Figure 7: Code snippet of the strict system prompt used to enforce structured JSON output from the LLM.]

```

/**
 * Fetches incident data from the local Python backend.
 */
const fetchData = async () => {
  setError(null);
  try {
    const res = await fetch("http://127.0.0.1:8000/api/incidents", {
      headers: { "X-API-Key": API_KEY }
    });

    if (!res.ok) {
      if (res.status === 403) throw new Error("403 Forbidden: Invalid API Key");
      throw new Error(`HTTP Error: ${res.status}`);
    }

    const data: Incident[] = await res.json();
    setIncidents(data);
    setLastSyncTime(new Date().toLocaleTimeString());
  } catch (err: any) {
    console.error("API Error:", err);
    setError(err?.message === "403 Forbidden: Invalid API Key" ? "Authentication Failed" : "API Offline");
  } finally {
    setLoading(false);
    setIsSyncing(false);
  }
};

```

[Figure 8: Implementation of the asynchronous data fetching logic, illustrating the dashboard's integration with the Python backend API.]

```

/* Manual Refresh / Retry Button */
<button
  onClick={() => { setIsSyncing(true); fetchData(); }}
  disabled={isSyncing}
  className={`flex-1 md:flex-none px-6 py-3 rounded-xl text-xs font-black transition-all active:scale-95 flex items-center justify-center gap-2 ${
    error ? 'bg-red-600 text-white hover:bg-red-700' : 'bg-slate-900 text-white hover:bg-indigo-600'
  } shadow-lg disabled:opacity-50`}
>
  <RefreshCw size={14} className={isSyncing ? "animate-spin" : ""} />
  {isSyncing ? "FETCHING DATA..." : error ? "RETRY CONNECTION" : "REFRESH ANALYTICS"}
</button>
</div>
</div>

```

[Figure 9: The manual refresh and synchronization interface, allowing users to forcefully trigger a state update from the backend.]

```

# START THE BACKGROUND LISTENERS
print("Starting Firestore Real-Time Listeners (0-Read Mode Active)...")
# 1. Watch Incidents
incident_query = db.collection("incidents").order_by("timestamp", direction=firestore.Query.DESENDING)
incident_watch = incident_query.on_snapshot(on_incident_snapshot)

# 2. Watch Users
users_query = db.collection("users")
users_watch = users_query.on_snapshot(on_users_snapshot)

```

[Figure 10: Code snippet demonstrating the initialization of Firestore on_snapshot listeners to establish a real-time data connection.]

```

    {/* risk score visualizer */}
    <div>
      <h3 className="text-sm font-black text-gray-400 uppercase tracking-wider mb-3">Risk Score</h3>
      <div className="flex items-center gap-4">
        <div className="flex-1 bg-gray-100 rounded-full h-6 overflow-hidden">
          <div
            className={`h-full rounded-full transition-all ${
              riskScore >= 8 ? 'bg-red-500' :
              riskScore >= 6 ? 'bg-orange-500' :
              riskScore >= 4 ? 'bg-yellow-500' : 'bg-green-500'
            }`}
            style={{ width: `${riskScore * 10}%` }}
          </div>
        </div>
        <span className="text-2xl font-black text-gray-800">{riskScore}/10</span>
      </div>
    </div>

    {/* ai generated summary */}
    <div>
      <h3 className="text-sm font-black text-gray-400 uppercase tracking-wider mb-3">AI Summary</h3>
      <p className="text-gray-700 leading-relaxed">{selectedIncident.ai_insights[0].summary}</p>
    </div>
  </div>

```

```

    <div className="space-y-3">
      {selectedIncident.ai_insights?.[0]?.mitigation_steps?.map((step, idx) => {
        // check if this step is completed
        const isChecked = selectedIncident.completed_steps?.includes(idx) || false;

        return (
          <label
            key={idx}
            className={`flex items-start gap-4 p-4 rounded-xl border-2 cursor-pointer transition-all shadow-sm ${
              isChecked
                ? 'bg-indigo-100 border-indigo-300'
                : 'bg-white border-indigo-50 hover:border-indigo-200'
            }`}
          >
            <input
              type="checkbox"
              checked={isChecked}
              onChange={() => handleToggleStep(selectedIncident.id, idx)}
              className="w-5 h-5 mt-0.5 rounded border-gray-300 text-indigo-600 focus:ring-indigo-500"
            />
            <span className={`text-sm font-medium leading-relaxed ${
              isChecked ? 'text-indigo-800 line-through decoration-indigo-400' : 'text-slate-700'
            }`}>
              {step}
            </span>
          </label>
        );
      })}
    </div>

```

[Figures 11 + 12: React frontend snippets demonstrating the dynamic rendering of the JSON AI insights into the Risk Score and Mitigation Playbook UI components.]

```

GLOBAL_INCIDENTS_CACHE = []
incident_cache_lock = threading.Lock()

def on_incident_snapshot(col_snapshot, changes, read_time):
    global GLOBAL_INCIDENTS_CACHE
    print(f"\n[SYNC] Firebase pushed an INCIDENT update! Updating RAM cache...")
    updated_list = []

    for doc in col_snapshot:
        data = doc.to_dict()
        decrypted_event = decrypt_payload(data.get("data"))
        if decrypted_event is None: continue

        ai_insights = None
        raw_insights = data.get("ai_insights")
        if raw_insights:
            if isinstance(raw_insights, list) and len(raw_insights) > 0:
                ai_insights = [decrypt_payload(raw_insights[0])]
            elif isinstance(raw_insights, str):
                ai_insights = [decrypt_payload(raw_insights)]

        raw_notes = data.get("user_notes", [])
        decrypted_notes = [decrypt_payload(n) for n in raw_notes if n is not None]

        updated_list.append({
            "id": doc.id,
            "event": decrypted_event,
            "ai_insights": ai_insights,
            "analysis_status": data.get("analysis_status", "pending"),
            "timestamp": data.get("timestamp"),
            "user_notes": decrypted_notes,
            "completed_steps": data.get("completed_steps", []),
            "assigned_to": data.get("assigned_to", "")
        })

    with incident_cache_lock:
        GLOBAL_INCIDENTS_CACHE = updated_list
        print("[SYNC] Incident Cache updated successfully.")

```

```

# FASTAPI ROUTE HANDLERS
def get_incidents():
    """Returns the decrypted incidents from local RAM."""
    with incident_cache_lock:
        return GLOBAL_INCIDENTS_CACHE

```

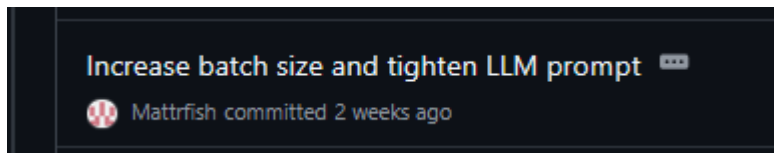
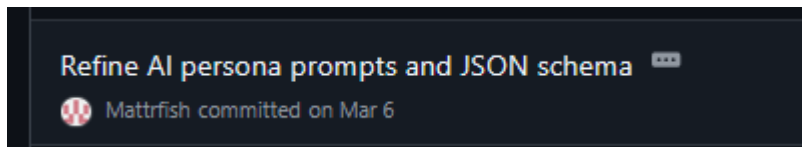
```

# API Endpoints

@router.get("/api/incidents") # retrieves all incidents
def fetch_incidents():
    return get_incidents()

```

[Figures 13 - 15: Implementation of the API caching layer, demonstrating how the FastAPI backend serves decrypted incident data directly from local memory to eliminate redundant database queries.]



[Figures 16 + 17: Screenshot of multiple prompt refinement evidence via github commits.]

```
# Define the 3 distinct personalities
prompts = {
    "business_owner": (
        "Analyze these logs for a non-technical business owner. "
        "Avoid jargon. Focus on: Is this dangerous? Do I need to panic? "
        "For the recommendation, tell them simply who to contact or if they can ignore it."
    ),
    "it_support": (
        "Analyze these logs for a Junior IT Sysadmin. "
        "Use standard IT terminology. Focus on: What service is failing? Is it a user error or a bug? "
        "For the recommendation, suggest specific actions like 'Reset User Password', 'Check Firewall', or 'Restart
Service'."
    ),
    "soc_analyst": (
        "Analyze these logs for a Senior Security Analyst (SOC). "
        "Be extremely technical. Focus on: Attack vectors, specific payload analysis (SQLi/XSS patterns), and Indicators
of Compromise (IOCs). "
        "For the recommendation, provide specific remediation commands (e.g., 'Block IP x via iptables', 'Patch CVE-2023-
xxx')."
    )
}
```

[Figure 18: Screenshot of the original persona prompts before iterative refinement.]

Appendix K: AI Declaration

Solo Work	S1 - Generative AI tools have not been used for this assessment.	☐
Assisted Work	A1 – Idea Generation and Problem Exploration Used to generate project ideas, explore different approaches to solving a problem, or suggest features for software or systems. Students must critically assess AI-generated suggestions and ensure their own intellectual contributions are central.	☐
	A2 - Planning & Structuring Projects AI may help outline the structure of reports, documentation and projects. The final structure and implementation must be the student's own work.	☒
	A3 – Code Architecture	☒

	AI tools maybe used to help outline code architecture (e.g. suggesting class hierarchies or module breakdowns). The final code structure must be the student's own work.	
	A4 – Research Assistance Used to locate and summarise relevant articles, academic papers, technical documentation, or online resources (e.g. Stack Overflow, GitHub discussions). The interpretation and integration of research into the assignment remain the student's responsibility.	☐
	A5 - Language Refinement Used to check grammar, refine language, improve sentence structure in documentation not code. AI should be used only to provide suggestions for improvement. Students must ensure that the documentation accurately reflects the code and is technically correct.	☒
	A6 – Code Review AI tools can be used to check comments within the code and to suggest improvements to code readability, structure or syntax. AI should be used only to provide suggestions for improvement. Students must ensure that the code accurately reflects their knowledge and is technically correct.	☒
	A7 - Code Generation for Learning Purposes Used to generate example code snippets to understand syntax, explore alternative implementations, or learn new programming paradigms. Students must not submit AI-generated code as their own and must be able to explain how it works.	☒
	A8 - Technical Guidance & Debugging Support AI tools can be used to explain algorithms, programming concepts, or debugging strategies. Students may also help interpret error messages or suggest possible fixes. However, students must write, test, and debug their own code independently and understand all solutions submitted.	☒
	A9 - Testing and Validation Support AI may assist in generating test cases, validating outputs, or suggesting edge cases for software testing. Students are responsible for designing comprehensive test plans and interpreting test results.	☒
	A10 - Data Analysis and Visualization Guidance AI tools can help suggest ways to analyse datasets or visualize results (e.g. recommending chart types or statistical methods). Students must perform the analysis themselves and understand the implications of the results.	☐

	A11 - Other uses not listed above Please specify:	☐
Partnered Work	P1 - Generative AI tool usage has been used integrally for this assessment Students can adopt approaches that are compliant with instructions in the assessment brief. Please Specify:	☐

Please provide details of AI usage and which elements of the coursework this relates to:

Generative AI was used in an assistive capacity for technical guidance and learning, functioning similarly to a resource like Stack Overflow (A7, A8). I prompted the AI to generate isolated example snippets and high-level pseudocode to help conceptualise my overall code architecture before writing my own implementation (A3). During development, I utilised AI to review my written code for syntax and readability improvements (A6), and to suggest edge cases and validation strategies for my testing phase (A9). Finally, AI assisted in planning and outlining the overall structure of my project report (A2), and was used to refine the grammar and sentence structure of the written documentation (A5). All AI outputs were critically assessed to ensure the final integrated codebase and report remain entirely my own independent work.

I understand that the ownership and responsibility for the academic integrity of this submitted assessment falls with me, the student.	☒
I confirm that all details provide above are an accurate description of how AI was used for this assessment.	☒

